

Reti di Calcolatori

Marco Tartaglione

7 ottobre 2026

Indice

1	Socket Java	9
1.1	Architettura e Comunicazione	9
1.2	Implementazione del Modello Sincrono	9
1.2.1	Evoluzione Storica	9
1.3	Tipi di comunicazione	9
1.3.1	Classi Java per Socket Internet	10
1.4	Sistema di nomi	10
1.4.1	Sistemi distribuiti e Naming	10
1.4.2	Binding	10
1.5	Socket Datagram	10
1.5.1	Scambio messaggi	11
1.5.2	Modello di comunicazione	11
1.5.3	Classi accessorie	12
1.5.4	Asincronismo	13
1.6	Multicast	13
1.6.1	MulticastSocket	13
1.6.2	Logica di funzionamento	13
1.7	Socket Stream (TCP)	14
1.7.1	Client	14
1.7.2	Server	14
1.7.3	Server Parallelo	15
1.7.4	Chiusura della Connessione	15
1.7.5	Opzioni Avanzate	16
2	Socket in C	17
2.1	Strumenti di comunicazione	17
2.2	Modello di uso	17
2.3	Primitive	17
2.4	Struttura	17
2.4.1	Nomi per la comunicazione	17
2.4.2	Tipo di comunicazione	18
2.5	Sistema di nomi	18
2.6	Tipi C	19
2.6.1	Funzioni di supporto	19
2.7	Primitive	20
2.7.1	socket	20
2.7.2	bind	20
2.8	Socket datagram	20
2.8.1	Primitive	20
2.8.2	Utilizzo	21
2.8.3	Proprietà	21
2.9	Rappresentazione dei dati e Endianness	21
2.10	Funzioni accessorie e manipolazione dati	22
2.10.1	Conversione degli Indirizzi IP	22
2.10.2	Gestione della Memoria (Buffer)	22
2.11	Socket Stream	22
2.11.1	Primitive	23
2.11.2	Sincronia e Gestione dei Buffer	24
2.11.3	Utilizzo e Gestione dei Messaggi	25
2.11.4	Terminazione della comunicazione	25
2.11.5	Processi Naive e Redirezione su Socket	27
2.12	API Socket in C	27

2.13	Modalità delle socket	28
2.13.1	Opzioni per le socket	28
2.13.2	Dimensioni dei buffer	29
2.13.3	Riutilizzo degli indirizzi	29
2.13.4	Gestione della terminazione (SO_LINGER)	29
2.14	Asincronismo	30
2.15	Non bloccanti	31
2.15.1	Effetti sulle primitive	31
2.16	Multiple Receive e Multiplexing	31
2.16.1	La primitiva select	31
2.16.2	Eventi della select	32
2.16.3	Maschere e bitset	32
2.17	Server Concorrente Monoprocesso	33
2.17.1	Gestione dei File Descriptor	33
2.17.2	Sincronizzazione della Maschera	33
2.18	Select in scrittura	33
2.19	Server Multifunzione	34
2.19.1	Configurazione di <code>inetd.conf</code>	34
2.20	Client Concorrente e Parallelo	34
2.20.1	Concorrente (Monoprocesso)	34
2.20.2	Parallela (Multiprocesso)	35
3	OSI	36
3.1	Caratteristiche dello Standard	36
3.2	Comitati	36
3.2.1	Comitati Internazionali Pubblici	36
3.2.2	Comitati Internazionali Privati e Professionali	36
3.3	Livelli	37
3.3.1	Protocollo	37
3.3.2	Interfaccia	37
3.3.3	Visione verticale ed orizzontale	37
3.3.4	Azioni di comunicazione	38
3.4	Connessioni e QoS	38
3.4.1	Connectionless	38
3.4.2	Connection-oriented	39
3.4.3	Quality of Service	39
3.5	Entità e Indirizzamento	39
3.5.1	Service Access Point	40
3.6	Comunicazione	40
3.7	Primitive	41
3.7.1	Esempio di Interazione	41
3.8	Differenze con TCP/IP	42
3.9	Livelli bassi	42
3.9.1	Network (Livello 3)	42
3.9.2	Trasporto	43
3.10	Livelli alti	44
3.10.1	Sessione	44
3.10.2	Presentazione	45
3.10.3	Applicazione	46
3.11	OSI vs Internet	47

4	Modello TCP/IP	48
4.1	L'architettura a livelli	48
4.2	Address Resolution Protocol (ARP)	48
4.2.1	ARP Cache	48
4.2.2	Ruoli	49
4.2.3	Realizzazione e Incapsulamento	49
4.2.4	RARP (Reverse ARP)	49
4.3	Dynamic Host Configuration Protocol (DHCP)	50
4.3.1	Soft-state e Lease Time	50
4.3.2	Bidding a 4 fasi (DORA)	50
4.3.3	Vantaggi Operativi	50
4.4	Network Address Translation (NAT)	51
4.4.1	Meccanismi di Traslazione	51
4.4.2	NAT a Porte (PAT - Port Address Translation)	51
4.4.3	Trattamento dei Datagrammi	51
4.5	Internet Control Message Protocol (ICMP)	51
4.5.1	Natura del Protocollo	52
4.5.2	Formato dell'Header	52
4.5.3	Livelli di Errore	52
4.5.4	Coordinamento e Gestione	52
4.6	User Datagram Protocol (UDP)	53
4.6.1	Caratteristiche del Servizio	53
4.6.2	Porte e Multiplexing	53
4.7	Transmission Control Protocol (TCP)	53
4.7.1	Modello di Servizio	54
4.7.2	Identificazione della Connessione	54
4.7.3	Struttura del Segmento TCP	54
4.7.4	Affidabilità e Trasmissione dei Dati	55
4.7.5	Controllo di Flusso: Sliding Window	55
4.7.6	Fasi della Connessione	56
4.7.7	Shutdown vs Close	57
4.7.8	Macchina a Stati del TCP	58
4.7.9	Timeout e Stima dell'RTT	59
4.7.10	Flow Control e Ottimizzazione	60
4.7.11	Eccezioni alla Trasparenza: PUSH e URG	60
4.7.12	Congestione	60
4.7.13	Meccanismi di Controllo: cwnd e ssthresh	61
4.7.14	Strategie principali riassunte	61
5	Servizi applicativi standard Internet	63
5.1	Servizi UNIX (livello applicativo)	63
5.1.1	Classificazione dei Servizi	63
5.2	Terminale remoto	63
5.2.1	Il Problema dell'Eterogeneità	64
5.2.2	Network Virtual Terminal (NVT)	64
5.3	Telnet	64
5.3.1	Operatività e Modalità di Lavoro	65
5.4	Rlogin	65
5.4.1	Caratteristiche Operative	65
5.4.2	Implementazione e Architettura	66
5.5	File Transfer Protocol (FTP)	66
5.5.1	Operazioni Comuni	66
5.5.2	Codifica e Risposte	67
5.5.3	Implementazione	67

5.5.4	Porte e Connessioni	68
5.5.5	Impegno risorse e continuità	68
5.5.6	Iniziativa delle connessioni (Modalità Attiva)	68
5.5.7	Protocollo vs Politiche	69
5.5.8	API standard	69
5.6	Posta elettronica	70
5.6.1	Architettura degli Agenti	70
5.6.2	Il Modello di Servizio	70
5.6.3	Dati e protocollo	70
5.6.4	Formato dati	70
5.6.5	Sistema di nomi e Indirizzamento	71
5.6.6	Componenti di servizio e MTA	71
5.6.7	DNS e Instradamento	71
5.7	Simple Mail Transfer Protocol (SMTP)	72
5.7.1	Struttura della Comunicazione	72
5.7.2	Codifica standard risposte	72
5.7.3	Mail Transaction	72
5.8	Usenet News	73
5.8.1	Gerarchie Principali	73
5.8.2	Architettura e Coordinamento	73
5.9	Network News Transfer Protocol (NNTP)	73
5.9.1	Meccanismo di Comunicazione	73
5.9.2	Codifica standard risposte	73
5.10	Servizi Sincroni vs Asincroni	74
5.10.1	Il modello USENET: Asincronicità e Distribuzione	74
5.10.2	Evoluzione verso la Sicurezza Infrastrutturale	74
6	Sistemi di Chiamate Remote	75
6.1	Semantiche	75
6.1.1	At-least-once	75
6.1.2	At-most-once	75
6.1.3	Exactly-once	75
6.2	Semantica protocolli TCP/IP	75
6.3	Remote Procedure Call (RPC)	76
6.3.1	Proprietà	76
6.4	Storia	76
6.5	Primitive	77
6.6	Tolleranza ai guasti	77
6.7	Open Network Computing (ONC)	78
6.8	Network Computing Architecture (NCA)	78
6.9	Modello con Stub	78
6.10	Passaggio di parametri	78
6.11	Eccezioni	79
6.12	Interface Definition Language (IDL)	79
6.12.1	Linguaggi più comuni	79
6.12.2	eXternal Data Representation (XDR)	79
6.12.3	Da IDL a Stub	80
6.13	Sviluppo e supporto RPC	80
6.13.1	Binding	80
6.14	Sistema di nomi	80
6.15	Controllo e asincronicità	81
6.15.1	RPC asincrone	81
6.16	RPC di SUN	81
6.17	RMI in JAVA	82

7	JAVA RMI	83
7.1	Accesso ad oggetti remoti	83
7.1.1	Stub e Skeleton	83
7.1.2	Differenze rispetto all'invocazione locale	83
7.1.3	Variabili Interfaccia e Disaccoppiamento	83
7.2	Architettura	84
7.2.1	Remote Reference Layer (RRL)	84
7.2.2	Transport Layer (TL)	84
7.2.3	Il Registro RMI (Registry)	84
7.3	Caratteristiche Fondamentali	85
7.4	Interfacce e Implementazioni	85
7.4.1	Interfaccia	85
7.4.2	Classe Server	85
7.5	Utilizzo	86
7.6	RMI Registry	86
7.6.1	Classe Naming e Attivazione Registry	87
7.7	Stub e Skeleton	87
7.8	Deployment	88
7.9	Serializzazione	88
7.9.1	Interazione con stream	88
7.9.2	Trasferimento remoto	88
7.10	Stub e riferimenti remoti	89
7.11	Implementazione del Registry	89
7.11.1	Registry Esterno	89
7.11.2	Registry Integrato (LocateRegistry)	89
7.12	Stub	90
7.12.1	Responsabilità Operative	90
7.12.2	Composizione Tecnica	90
7.12.3	Esempio	90
7.13	Skeleton	91
7.13.1	Esempio	92
7.14	Livello trasporto	92
7.14.1	Concorrenza	92
7.14.2	Comunicazione	93
7.15	Caricamento classi e applet	93
7.15.1	Le Applet	93
7.16	Class Loading	94
7.16.1	Codebase	94
7.17	Localizzazione del codice	95
7.17.1	Uso del codebase	95
7.18	Security Manager	95
7.18.1	RMISecurityManager	95
7.18.2	File di policy	96
7.18.3	Problema e Soluzione di Sicurezza del Registry	96
7.19	Problema del Bootstrap	96
7.20	Visione interna Memoria Java	97
7.20.1	Heap	97
7.21	Remote references	97

8	Implementazione RPC di SUN	99
8.1	Requisiti implementativi	99
8.2	Semantica di Invocazione	99
8.3	ONC (Open Network Computing)	99
8.4	Contratto	100
8.5	Programma	100
8.5.1	Definizione del servizio remoto	100
8.5.2	Implementazione del programma	101
8.6	Procedura di servizio	101
8.6.1	Implementazione del Client	102
8.7	Passi di Sviluppo	102
8.8	RPC in UNIX	103
8.9	RPC di SUN	103
8.9.1	Gestione della Concorrenza	103
8.9.2	Sincronicità	103
8.10	Rischi e problemi	104
8.10.1	Semantica e affidabilità	104
8.11	Identificazione di RPC	104
8.12	Supporto standard RPC	104
8.12.1	Anatomia dei messaggi: CALL e REPLY	105
8.12.2	Gestione degli Errori e Stati	105
8.13	Il Sistema di Nomi	105
8.13.1	Meccanismo di Registrazione e Struttura Dati	106
8.13.2	Interazione Client-Server	106
8.13.3	Servizi Portmapper	106
8.14	Primitiva svc_run	107
8.14.1	Vincoli di Trasporto e Compatibilità	107
8.15	Architettura RPC	107
8.16	Interface Definition Language: XDR	108
8.16.1	Vincoli dell'Interfaccia	108
8.16.2	Tipologie di Dati in XDR	108
8.16.3	Funzioni	108
8.16.4	Posizionamento nel modello OSI	109
8.16.5	Stream XDR	109
8.17	API di basso livello	109
8.17.1	Cliente	109
8.17.2	Server	109
8.18	Server	110
8.18.1	Gestore di trasporto	110
8.18.2	Dispatching	110
8.18.3	Procedure di conversione e gestione errori	111
8.19	Client	112
8.19.1	Gestore di trasporto	112
8.19.2	Chiamata procedura remota: clnt_call()	112
8.19.3	Analisi errori e Distruzione	113
8.20	Generazione Automatica delle RPC	113
8.21	Esempio	114
8.21.1	File .x (Definizione dell'Interfaccia)	114
8.21.2	Header file (Generato da rpcgen)	114
8.21.3	_xdr.c (Filtri di conversione)	115
8.21.4	Stub Client	116
8.21.5	Stub Server (Skeleton)	116
8.21.6	Implementazione del Client	117
8.21.7	Implementazione del Server	118

8.22	Utilizzo delle Variabili static nel Server	119
8.22.1	Necessità del valore di ritorno statico	119
8.23	Dichiarazione tipi XDR	119
8.23.1	Vettori a lunghezza fissa	120
8.23.2	Matrici	120
8.23.3	Vettori a lunghezza variabile	120
8.23.4	Puntatori	120
8.23.5	Struttura	120
8.23.6	Unione	120
8.23.7	Enumerazione e Costanti	120
8.24	Modalità asincrona	121
8.24.1	Batching	121
9	Sistemi di nomi	122
9.1	Nomi	122
9.1.1	Livelli di nomi	122
9.2	Globalità e Località	122
9.3	Spazi di nomi	123
9.3.1	Nomi di Gruppo	123
9.4	Componenti	123
9.4.1	Name server	124
9.4.2	Comunicazione	124
9.4.3	Gestione nomi	124
9.5	Sistema globale	125
9.6	Altri sistemi di nomi	125
9.7	Directory X.500	125
9.7.1	Organizzazione delle Informazioni	126
9.7.2	Caratteristiche distintive	126
9.7.3	Query e Ricerca	126
9.7.4	Attributi in X.500	127
9.7.5	Filtri e Operazioni	127
9.7.6	Organizzazione interna	127
9.8	Differenza con Database	128
9.9	Uso	128
9.10	Sviluppo	128
9.10.1	Protocolli di directory	129
9.10.2	Protocolli di discovery	129

1 Socket Java

La realizzazione di sistemi Client/Server (C/S) in Java sfrutta l'astrazione delle socket per implementare lo scambio di messaggi su reti eterogenee. Sebbene Java operi a un livello di astrazione superiore rispetto al C, i concetti derivati da Unix BSD 4.2 rimangono le fondamenta del sistema.

1.1 Architettura e Comunicazione

Le socket in Java rappresentano l'estremità locale (**endpoint**) di un canale di comunicazione bidirezionale. La comunicazione avviene tramite API che mascherano la complessità dei driver di rete sottostanti:

- **Astrazione dei Driver:** Le primitive Java vengono tradotte in chiamate di sistema che interagiscono con i driver di trasporto (TCP/UDP) e rete (IP).
- **Modello Punto a Punto:** La base della comunicazione rimane lo scambio di messaggi *send/receive* tra due entità identificate da un indirizzo IP e un numero di porta.

1.2 Implementazione del Modello Sincrono

Nonostante gli strumenti di base siano asincroni a livello di rete, l'obiettivo primario è spesso l'implementazione di un modello **sincrono bloccante** (Request-Response):

1. **Lato Cliente:** Il processo invia una richiesta e si sospende (*waiting*) in attesa della risposta, realizzando una sincronizzazione esplicita con il server.
2. **Lato Servitore:** Il processo riceve la richiesta, la elabora e invia la risposta specifica al mittente. Il server può operare in modo sequenziale (una richiesta alla volta) o concorrente (più richieste simultanee tramite Thread).

1.2.1 Evoluzione Storica

Le socket, nate nel 1983 con Unix BSD 4.2, sono diventate lo standard *de facto*. In Java, questa eredità si traduce in due classi principali contenute nel pacchetto `java.net`.

1.3 Tipi di comunicazione

In Java, la comunicazione via socket si divide in due grandi categorie, modellate sui principali protocolli della suite Internet (TCP/IP):

- **Comunicazione con connessione (Socket STREAM):** Richiede che venga stabilito un canale stabile e dedicato tra Client e Server prima dello scambio di dati.
 - **Protocollo:** TCP (*Transmission Control Protocol*).
 - **Caratteristiche:** Affidabile (garantisce l'arrivo dei dati nell'ordine corretto), orientata al flusso di byte.
- **Comunicazione senza connessione (Socket DATAGRAM):** Simile al sistema postale, dove ogni messaggio (pacchetto) è un'entità indipendente che contiene l'indirizzo del destinatario e può seguire percorsi diversi.
 - **Protocollo:** UDP (*User Datagram Protocol*).
 - **Caratteristiche:** Non affidabile (non garantisce l'ordine o la consegna), ma molto più veloce e leggera (assenza di handshake).

1.3.1 Classi Java per Socket Internet

Il linguaggio Java fornisce classi specifiche nel pacchetto `java.net` per gestire i due principali modelli di comunicazione:

- **DatagramSocket:** Utilizzata sia dal Client che dal Server per l'invio e la ricezione di pacchetti (`DatagramPacket`). È l'implementazione del protocollo **UDP**. In questo modello non c'è una distinzione gerarchica forte: ogni nodo è un endpoint che invia o riceve unità di dati indipendenti.
- **Socket:** Implementa la socket utilizzata dal **Client** per iniziare la connessione stream verso un server e, successivamente, da entrambi per lo scambio bidirezionale di dati. Rappresenta il canale attivo del protocollo **TCP**.
- **ServerSocket:** Utilizzata esclusivamente dal **Server**. Il suo compito è mettersi in ascolto su una porta specifica e accettare le richieste di connessione stream entranti. Quando riceve una richiesta, il metodo `accept()` "cristallizza" la connessione restituendo un oggetto `Socket` specifico per quel client.

1.4 Sistema di nomi

Un'applicazione distribuita è costituita da processi distinti per località che comunicano e cooperano attraverso lo scambio di messaggi per ottenere risultati coordinati. Il primo problema da affrontare riguarda la **identificazione reciproca** dei processi (Client o Server) nella rete, i cui nomi (come i PID) hanno solitamente solo validità locale al sistema operativo ospite.

Ogni processo deve essere associato a un **nome globale**, visibile in modo univoco e non ambiguo, che consenta ad altri nodi di raggiungerlo senza incertezze.

Gli endpoint di comunicazione (socket) sono locali al processo, ma il problema della loro visibilità globale viene risolto dai livelli bassi della pila ISO/OSI:

- **Livello Rete (IP):** Fornisce l'indirizzo univoco dell'host (il "contenitore" dei processi).
- **Livello Trasporto (TCP/UDP):** Fornisce il numero di **porta**, che identifica lo specifico endpoint all'interno dell'host.

1.4.1 Sistemi distribuiti e Naming

Nei sistemi distribuiti, la gestione dei nomi si articola su diverse dimensioni:

- **Sistemi di nomi visibili e non visibili:** I nomi visibili (es. `www.google.com`) sono user-friendly, mentre quelli non visibili (es. indirizzi IP a 32 o 128 bit) sono ottimizzati per il routing hardware.
- **Sistemi di nomi globali e locali:** I nomi locali identificano risorse interne a un nodo, mentre i nomi globali sono risolvibili da qualsiasi punto della rete.

1.4.2 Binding

Il **Binding** è l'operazione di associazione tra sistemi di nomi diversi (es. l'associazione tra un nome logico e un indirizzo fisico). In un'architettura distribuita, il binding permette di mappare l'insieme dei nomi globali sugli insiemi di nomi locali, garantendo che un messaggio inviato a un identificatore astratto raggiunga l'istanza corretta del processo.

1.5 Socket Datagram

Le socket **datagram** permettono a due thread di scambiarsi messaggi senza la necessità di stabilire una connessione preventiva. Ogni messaggio è incapsulato in un pacchetto indipendente che viaggia sulla rete.

La classe `DatagramSocket` definisce l'endpoint per la comunicazione basata su protocollo UDP. Sebbene esistano numerosi costruttori, l'obiettivo principale è legare (*bind*) la socket a un'interfaccia di rete e a una porta locale per poter ricevere messaggi.

```

1  /* Costruttore default:
2   * port:  Porta locale su cui restare in ascolto (0 per porta scelta dal sistema)
3   * laddr:  Indirizzo IP locale dell'interfaccia di rete (se multi-homed)
4   */
5  public DatagramSocket(int port, InetAddress laddr) throws SocketException;

```

Oltre alle primitive di I/O, la classe fornisce metodi per controllare il comportamento della comunicazione:

```

1  // Imposta un timeout massimo per la receive() (in millisecondi)
2  // Se scade, viene lanciata una SocketTimeoutException
3  public void setSoTimeout(int timeout) throws SocketException;
4
5  // Chiude la socket e rilascia le risorse del sistema operativo
6  public void close();
7
8  // Restituisce la porta locale a cui la socket è legata
9  public int getLocalPort();
10
11 // Imposta la dimensione del buffer di invio del kernel (SO_SNDBUF)
12 // Utile per inviare grandi volumi di dati senza perdere pacchetti localmente
13 public void setSendBufferSize(int size) throws SocketException;
14
15 // Imposta la dimensione del buffer di ricezione del kernel (SO_RCVBUF)
16 // Fondamentale in UDP per evitare il drop dei pacchetti se il processo è lento
17 public void setReceiveBufferSize(int size) throws SocketException;
18
19 // Abilita l'opzione SO_REUSEADDR. Permette a più socket di legarsi
20 // alla stessa porta (utile in multicast o per riavvii rapidi del server)
21 public void setReuseAddress(boolean on) throws SocketException;

```

1.5.1 Scambio messaggi

Lo scambio avviene tramite l'interazione con i driver locali utilizzando pacchetti utente. Una volta istanziata la `DatagramSocket`, le operazioni fondamentali sono:

```

1  // Consegna il pacchetto al kernel locale. L'operazione è ASINCRONA rispetto
2  // alla ricezione: il mittente non sa se o quando il destinatario riceverà
3  // il dato. La primitiva termina non appena i dati sono stati copiati nei
4  // buffer di sistema.
5  public void send(DatagramPacket p) throws IOException;
6
7  // Sospende il thread in attesa che il driver locale riceva un pacchetto.
8  // Ha una semantica SINCRONA BLOCCANTE per il ricevente (rispetto al proprio
9  // kernel), ma rimane asincrona rispetto all'azione del mittente.
10 public void receive(DatagramPacket p) throws IOException;

```

Le caratteristiche di flusso che caratterizzano il sistema datagram sono:

- **Consumo immediato:** È sufficiente l'arrivo di un singolo datagramma per sbloccare una `receive()` in attesa. Ogni chiamata a `receive()` consuma esattamente un pacchetto UDP.
- **Strutture di appoggio:** Entrambe le operazioni richiedono l'uso della classe `DatagramPacket`, che funge da contenitore sia per i dati (*payload*) sia per le informazioni di indirizzamento (IP e porta).

1.5.2 Modello di comunicazione

Perché lo scambio di messaggi avvenga correttamente, le socket datagram devono essere inizializzate e i nodi devono potersi identificare. Il modello si basa sulla specifica puntuale del destinatario per ogni invio:

- **Mittente:** Deve inserire nel pacchetto sia i dati che l'indirizzo del ricevente.
- **Ricevente:** Estrae dalla propria coda locale i messaggi uno alla volta (politica FIFO).

Le informazioni necessarie sono di due tipi:

- **Applicative:** Il contenuto del messaggio (payload) o lo spazio di memoria destinato a contenerlo.
- **Controllo:** L'indirizzo del nodo e la porta associata alla socket di destinazione.

1.5.3 Classi accessorie

InetAddress Questa classe gestisce l'identificazione degli host. Non avendo costruttori pubblici, l'istanziamento avviene tramite metodi statici che interagiscono con il sistema di naming:

```

1 // Risoluzione di un nome logico o indirizzo testuale
2 public static InetAddress getByName(String hostname) throws UnknownHostException;
3
4 // Restituisce un array di indirizzi per host multihomed
5 public static InetAddress[] getAllByName(String hostname) throws
    UnknownHostException;
6
7 // Identifica l'interfaccia di rete locale
8 public static InetAddress getLocalHost() throws UnknownHostException;
```

DatagramPacket Rappresenta l'unità dati (payload) unita alle informazioni di instradamento (IP e Porta). La struttura del costruttore riflette questa doppia natura:

```

1 /* buf:      array di byte (area di memoria applicativa)
2  * length:   dimensione dei dati da trattare
3  * address:  oggetto InetAddress (destinatario/mittente)
4  * port:     porta UDP (destinatario/mittente)
5  */
6 public DatagramPacket(byte[] buf, int length, InetAddress address, int port);
7
8 // In caso di sola ricezione
9 public DatagramPacket(byte[] buf, int length);
```

L'operatività differisce in base alla primitiva invocata:

- **Invio (sock.send):** L'utente deve aver già popolato il buffer con i dati applicativi e impostato i campi di controllo (IP e porta del destinatario).
- **Ricezione (sock.receive):** L'utente deve preparare un pacchetto con un buffer vuoto di dimensioni adeguate. Dopo la chiamata, il kernel riempirà il pacchetto non solo con i dati, ma anche con le informazioni di controllo di **chi ha inviato** il messaggio, permettendo al ricevente di identificare il mittente.

DataOutputStream La classe `DataOutputStream` agisce come un *wrapper* per un `OutputStream` esistente. Essa permette a un'applicazione di scrivere tipi di dati primitivi Java in un formato portabile e indipendente dall'architettura (formato *big-endian*).

```

1 // Creazione del wrapper su uno stream di output (es. di una socket)
2 DataOutputStream out = new DataOutputStream(socket.getOutputStream());
3
4 // Esempi di invio di tipi primitivi
5 out.writeInt(1024);           // Invia un intero (4 byte)
6 out.writeDouble(3.14);        // Invia un double (8 byte)
7 out.writeBoolean(true);       // Invia un booleano
8 out.writeUTF("Messaggio");    // Invia una stringa codificata in UTF-8
9 out.flush();                  // Svuota il buffer e forza l'invio
```

DataInputStream Simmetricamente, **DataInputStream** permette di leggere tipi primitivi da un **InputStream**. Perche la comunicazione avvenga correttamente, il ricevente deve invocare i metodi di lettura nello **stesso ordine** in cui il mittente ha eseguito le scritture.

```
1 // Creazione del wrapper su uno stream di input
2 DataInputStream in = new DataInputStream(socket.getInputStream());
3
4 // Esempi di ricezione
5 int numero = in.readInt();
6 double pi = in.readDouble();
7 boolean flag = in.readBoolean();
8 String testo = in.readUTF();
```

1.5.4 Asincronismo

Il sistema e asincrono tra i processi. La primitiva **send()** è non bloccante rispetto alla rete (consegna al kernel e prosegue), mentre la **receive()** è bloccante per il thread ricevente, ma non richiede alcuna sincronizzazione temporale con l'istante in cui il mittente ha eseguito la sua **send()**.

1.6 Multicast

La comunicazione multicast si discosta dal modello punto a punto. In questo scenario, i messaggi vengono recapitati a un insieme di host identificati da un unico indirizzo IP speciale.

Il multicast utilizza un range di indirizzi IP dedicato, noto come **Classe D**:

224.0.0.0 — 239.255.255.255

Questi indirizzi non identificano un singolo host, ma un "canale" logico a cui i processi possono iscriversi.

1.6.1 MulticastSocket

In Java, la gestione del multicast avviene tramite la classe **MulticastSocket**, che estende **DatagramSocket** aggiungendo le operazioni di gestione del gruppo.

```
1 // 1. Preparazione del gruppo (Indirizzo Classe D e Porta)
2 InetAddress gruppo = InetAddress.getByName("229.5.6.7");
3 int porta = 6666;
4
5 // 2. Creazione della socket legata alla porta del gruppo
6 MulticastSocket s = new MulticastSocket(porta);
7
8 // 3. Operazioni di gestione dell'appartenenza (per ricevere)
9 s.joinGroup(gruppo); // Il nodo si registra per ricevere i pacchetti
10 s.leaveGroup(gruppo); // Il nodo smette di ricevere i pacchetti dal gruppo
```

1.6.2 Logica di funzionamento

Il modello Java introduce una granularità basata sulle porte che lo rende estremamente selettivo:

- **Iscrizione (Join):** I nodi che desiderano ricevere devono manifestare esplicitamente il proprio interesse. Finché un processo non esegue la **joinGroup()**, i pacchetti multicast inviati a quell'indirizzo verranno scartati dal suo stack di rete.
- **Molteplicità dei gruppi:** È possibile avere più gruppi logici sullo stesso indirizzo IP di Classe D, distinguendoli tramite il numero di porta. Solo chi condivide sia l'IP che la porta farà parte dello stesso sotto-gruppo di ricezione.

- **Invio vs Ricezione:**

- Per **inviare** in multicast, non è strettamente necessario unirsi al gruppo; basta inviare un `DatagramPacket` all'indirizzo di Classe D.
- Per **ricevere**, è obbligatorio eseguire la `joinGroup()` e restare in attesa con una `receive()`.

Nota sulla codifica Sebbene le informazioni di controllo (IP e porta) siano gestite dalle classi `InetAddress` e `MulticastSocket`, i dati testuali all'interno dei pacchetti seguono la codifica **UTF-16** interna di Java, a meno di conversioni esplicite in array di byte (es. via UTF-8).

1.7 Socket Stream (TCP)

Le socket **stream** realizzano un canale virtuale bidirezionale basato su byte. A differenza del modello datagram, la comunicazione richiede una fase di configurazione (handshake) e garantisce una consegna affidabile e sequenziale (**FIFO**).

La connessione è identificata univocamente dalla quadrupla:

<IP Client, Porta Client, IP Server, Porta Server>

1.7.1 Client

Il Client utilizza la classe `Socket` per avviare una connessione "attiva". La creazione dell'oggetto implica l'esecuzione atomica delle fasi di *socket*, *bind* e *connect*.

```
1 // Connessione atomica a host remoto e porta
2 public Socket(InetAddress address, int port) throws IOException;
3
4 // Connessione tramite nome logico (DNS)
5 public Socket(String host, int port) throws IOException;
6
7 // Connessione con specifica dell'interfaccia locale e porta locale
8 public Socket(InetAddress remote, int rPort,
9               InetAddress local, int lPort) throws IOException;
10
11 // Restituisce l'indirizzo IP del nodo remoto
12 public InetAddress getInetAddress();
13 // Restituisce la porta remota a cui la socket è connessa
14 public int getPort();
15 // Restituisce l'indirizzo IP locale usato per la connessione
16 public InetAddress getLocalAddress();
17 // Restituisce la porta locale assegnata al Client
18 public int getLocalPort();
```

1.7.2 Server

Il Server utilizza la classe `ServerSocket` per mettersi in ascolto "passivo". Non scambia dati direttamente, ma accetta richieste di connessione.

```
1 // Crea socket in ascolto sulla porta locale (bind + listen)
2 public ServerSocket(int port) throws IOException;
3
4 // Specifica anche la lunghezza della coda di attesa (backlog/count)
5 public ServerSocket(int port, int count) throws IOException;
6
7 // Accetta una connessione dalla coda (BLOCCANTE).
8 // Restituisce un oggetto Socket per la comunicazione effettiva.
9 public Socket accept() throws IOException;
```

```

10
11 // Restituisce l'indirizzo locale su cui il server è in ascolto
12 // (0.0.0.0 se in ascolto su tutte le interfacce)
13 public InetAddress getInetAddress();
14 // Restituisce la porta locale su cui il server è in ascolto
15 public int getLocalPort();
16 // Restituisce l'indirizzo IP locale
17 public InetAddress getLocalAddress();
18 // Restituisce la porta remota
19 public int getPort();

```

1.7.3 Server Parallelo

Per evitare che il server diventi un collo di bottiglia, alla ricezione di una `Socket` tramite `accept()`, il server master genera un nuovo thread (o processo) affidandogli la gestione della comunicazione. Il master torna immediatamente in `accept()` per servire nuove richieste pendenti nella coda.

```

1 // Esempio logico di Server Parallelo
2 ServerSocket server = new ServerSocket(8080);
3 while (true) {
4     Socket clientSocket = server.accept(); // Bloccante fino a nuovo client
5     // Avvio di un thread dedicato (Slave)
6     new Thread(new ServiceHandler(clientSocket)).start();
7 }

```

In Java, grazie al modello di memoria condivisa, ogni socket è potenzialmente visibile da tutti i thread. Tuttavia, nel modello parallelo:

- **Isolamento logico:** Ogni thread "figlio" riceve il riferimento a una specifica `Socket` e lavora solo su quella, garantendo la separazione delle sessioni applicative.
- **Ciclo di vita:** La chiusura di una socket tramite `close()` ha effetto globale. Se un thread chiude la socket, essa viene chiusa definitivamente per tutti i thread che ne possiedono il riferimento.
- **Persistenza delle risorse:** Le risorse di supporto (buffer di sistema, descrittori) rimangono impegnate fino a quando non viene invocata esplicitamente la `close()` o finché il processo non termina.

È importante distinguere tra la chiusura della `Socket` (che termina una singola connessione client) e la chiusura della `ServerSocket` (che impedisce al server di accettare *qualsiasi* nuova connessione, ma non chiude necessariamente i canali già attivi gestiti dai thread figli).

1.7.4 Chiusura della Connessione

La gestione della chiusura è un aspetto critico per l'affidabilità del protocollo. Java offre due approcci: la chiusura definitiva (`close`) e la chiusura parziale (`shutdown`).

Chiusura Totale Il metodo `close()` rilascia tutte le risorse del sistema operativo associate alla socket.

```

1 public void close() throws IOException;

```

Il comportamento del kernel varia in base alla direzione del flusso:

- **Gestione Buffer OUT:** Per garantire l'affidabilità TCP, il kernel mantiene i dati nel buffer di uscita il tempo necessario per tentare la consegna al destinatario, anche dopo che l'applicazione ha chiamato la `close()`.
- **Gestione Buffer IN:** La memoria dedicata ai dati in entrata viene liberata immediatamente. Eventuali dati non ancora letti dall'applicazione vengono scartati.

Chiusura Parziale Questi metodi permettono di terminare la comunicazione in una sola direzione senza chiudere l'intera socket. È una pratica fondamentale per protocolli in cui un nodo deve segnalare la fine dell'invio dei dati pur rimanendo in attesa di una risposta.

```
1 // Chiude il flusso di input. Ulteriori letture restituiranno EOF.
2 public void shutdownInput() throws IOException;
3
4 // Chiude il flusso di output. Invia un pacchetto FIN al peer
5 // segnalando che non verranno inviati altri dati.
6 public void shutdownOutput() throws IOException;
```

1.7.5 Opzioni Avanzate

Le seguenti primitive consentono di modificare i parametri del protocollo TCP tramite la classe `Socket`:

```
1 // Imposta il comportamento alla chiusura (SO_LINGER)
2 // se true, la close() blocca per 'seconds' o finche i dati sono inviati
3 // seconds: tempo massimo di attesa prima di un reset forzato (RST)
4 public void setSoLinger(boolean on, int seconds) throws SocketException;
5
6 // Disabilita l'algoritmo di Nagle (TCP_NODELAY)
7 // se true, i dati vengono inviati immediatamente senza buffering.
8 // Indispensabile per applicazioni interattive o real-time (es. telnet, gaming)
9 public void setTcpNoDelay(boolean on) throws SocketException;
10
11 // Attiva il monitoraggio della connessione (SO_KEEPALIVE)
12 // se true, TCP invia pacchetti di controllo periodici per verificare
13 // che il peer sia ancora raggiungibile anche in assenza di traffico dati
14 public void setKeepAlive(boolean on) throws SocketException;
15
16 // Imposta un timeout per le operazioni di lettura (SO_TIMEOUT)
17 // timeout: tempo in millisecondi; se scade, read() lancia SocketTimeoutException
18 public void setSoTimeout(int timeout) throws SocketException;
```

2 Socket in C

In linguaggio C è possibile sviluppare sistemi che utilizzano le *API* di *UNIX* per realizzare meccanismi di comunicazione tra macchine distinte ed eterogenee, consentendo lo scambio di dati attraverso l'uso delle *socket*.

2.1 Strumenti di comunicazione

Nei sistemi *UNIX*, il modello basato su *socket* (così come i meccanismi di comunicazione e sincronizzazione) viene gestito attraverso l'utilizzo di:

- **Segnali:** un processo può generare un evento asincrono senza specificarne esplicitamente il mittente.
- **File:** i processi possono comunicare e sincronizzarsi condividendo lo stesso file system.

Tra processi coesistenti sullo stesso nodo è inoltre possibile utilizzare diversi strumenti di comunicazione interprocesso, tra cui:

- **Pipe**
- **Named Pipe**
- **Shared memory**

Per la comunicazione tra sistemi remoti, *UNIX* introduce le *Socket BSD* (*Berkeley Software Distribution*), che consentono di instaurare connessioni di rete indipendenti dall'hardware e dal sistema operativo delle macchine coinvolte.

2.2 Modello di uso

Le *socket UNIX* seguono la stessa struttura dei file. Il kernel mantiene una *tabella dei file aperti per processo* e associa a ciascuna socket un *file descriptor*. In questo modo è possibile eseguire le classiche operazioni di apertura, chiusura e manipolazione fornite dal sistema operativo per la gestione dei file. Internamente vengono poi definiti ulteriori parametri associati alla comunicazione, come indirizzi locale e remoto.

2.3 Primitive

È compito di *UNIX* fornire primitive atomiche che consentano la comunicazione attraverso le *socket*, completamente integrate con il sistema di gestione dei processi e dei file. *UNIX Berkeley* introduce le *socket standard*, ovvero un insieme di strumenti di comunicazione remota che permettono la lettura e la scrittura di messaggi o *stream* tramite *socket*.

Le principali caratteristiche di questo modello sono:

- **Eterogeneità:** supporto alla comunicazione tra processi che operano su architetture differenti.
- **Trasparenza:** i processi risultano indipendenti dalla loro localizzazione fisica.
- **Efficienza:** garantisce buone prestazioni in termini di comunicazione e utilizzo delle risorse.
- **Compatibilità:** i processi, anche già esistenti, possono operare tramite *socket* senza richiedere modifiche sostanziali.
- **Completezza:** supporto a protocolli diversi e specializzati per esigenze di comunicazione differenti.

2.4 Struttura

2.4.1 Nomi per la comunicazione

Il modello definisce anche il dominio della comunicazione, ovvero l'unione tra la semantica della comunicazione e il relativo standard di denominazione. La semantica della comunicazione include:

- L'affidabilità della trasmissione
- La possibilità di operare in modalità *multicast*

Con *naming* si intende il modo in cui vengono identificati i punti terminali della comunicazione. Il primo criterio di classificazione riguarda il dominio di appartenenza della comunicazione:

Dominio	Descrizione
PF_UNIX	Comunicazione locale tra processi sullo stesso sistema, tramite file o <i>pipe</i> .
PF_INET	Comunicazione tra sistemi distinti mediante protocolli di rete <i>TCP/IP</i> .

2.4.2 Tipo di comunicazione

I principali tipi di servizio disponibili sono:

- **Datagram:** scambio di messaggi non orientato alla connessione, senza garanzie di consegna, ordine o integrità.
- **Stream:** comunicazione bidirezionale orientata alla connessione, affidabile, ordinata e priva di duplicati; supporta flussi continui e dati *out-of-band*.
- **Seqpacket:** comunicazione basata su messaggi affidabili e ordinati, in cui ogni pacchetto mantiene i propri confini.
- **Raw:** accesso diretto ai protocolli di rete sottostanti, con gestione minima da parte del sistema operativo.

Ogni dominio di comunicazione specifica i propri protocolli di riferimento:

- *UNIX*: AF_UNIX
- *Internet*: AF_INET
- *Xerox*: AF_NS
- *CCITT*: AF_CCITT

Nello standard *Berkeley*, i protocolli più diffusi sono riportati nella seguente tabella:

Address Family	Socket Type	Protocol	Comunicazione
AF_INET	<i>Stream</i>	IPPROTO_TCP	TCP
AF_INET	<i>Datagram</i>	IPPROTO_UDP	UDP
AF_INET	<i>Raw</i>	IPPROTO_ICMP	ICMP
AF_INET	<i>Raw</i>	IPPROTO_RAW	Raw
AF_NS	<i>Stream</i>	NSPROTO_SPP	SPP
AF_NS	<i>Seqpacket</i>	NSPROTO_SPP	SPP
AF_NS	<i>Raw</i>	NSPROTO_ERROR	Error Protocol
AF_NS	<i>Raw</i>	NSPROTO_RAW	Raw
AF_UNIX	<i>Stream</i>	—	Locale (file/socket)
AF_UNIX	<i>Datagram</i>	—	Locale (file/socket)

2.5 Sistema di nomi

I nomi utilizzati nel modello di comunicazione si suddividono in:

- **Logici:** nomi locali, ovvero l'indirizzo della *socket* all'interno del dominio di comunicazione.
- **Fisici:** nomi globali, corrispondenti a una porta identificata sul nodo di rete.

Ogni *socket* richiede un legame, detto *binding*, tra il nome logico e quello fisico. Si definiscono quindi tre principali domini di *half-association*, ovvero coppie costituite da un nome logico e da un nome fisico:

- **Internet:** la *socket* è associata a una porta locale identificata da un indirizzo IP.
- **UNIX:** la *socket* è legata a un percorso nel file system locale.
- **CCITT:** la *socket* è indirizzata tramite il protocollo di rete X.25.

2.6 Tipi C

Le librerie del linguaggio C offrono diversi tipi di dato per rappresentare in modo flessibile gli indirizzi di rete.

```
1 struct sockaddr {
2     u_short sa_family;    // Famiglia di indirizzi
3     char sa_data[14];    // Dati specifici della famiglia
4 };
5
6 // Famiglia Internet
7 struct in_addr {
8     u_long s_addr;       // Indirizzo IP a 32 bit
9 };
10
11 struct sockaddr_in {
12     u_short sin_family;   // Famiglia di indirizzi (AF_INET)
13     u_short sin_port;     // Numero di porta
14     struct in_addr sin_addr; // Indirizzo IP
15 };
16
17 struct sockaddr_in mySocketAddress;
```

2.6.1 Funzioni di supporto

Le funzioni di supporto messe a disposizione dalle librerie di rete permettono di ottenere gli indirizzi fisici a partire da quelli logici.

gethostbyname La funzione `gethostbyname` ricerca l'host specificato all'interno del file `/etc/hosts`. Se l'host non viene trovato, la funzione restituisce NULL.

```
1 #include <netdb.h>
2
3 struct hostent {
4     char* h_name;         // Nome ufficiale dell'host
5     char** h_aliases;     // Lista degli alias
6     int h_addrtype;       // Tipo di indirizzo (es. AF_INET)
7     int h_length;         // Lunghezza dell'indirizzo
8     char** h_addr_list;   // Lista degli indirizzi IP
9     #define h_addr h_addr_list[0] // Indirizzo IP principale
10 };
11
12 struct hostent* gethostbyname(const char* name);
```

getservbyname In modo analogo a `gethostbyname`, la funzione `getservbyname` consente di ottenere il numero di porta associato a un servizio, a partire dal suo nome logico, consultando il file `/etc/services`.

```
1 #include <netdb.h>
2
3 struct servent {
4     char* s_name;         // Nome ufficiale servizio
5     char** s_aliases;     // Lista degli alias
6     int s_port;           // Numero della porta
7     char* s_proto;        // Protocollo da utilizzare
8 }
9
10 struct servent* getservbyname(const char* name, const char* protocol);
```

2.7 Primitive

2.7.1 socket

Per creare una *socket* con nome logico **locale** è necessario utilizzare la primitiva `socket`:

```
1 #include <sys/socket.h>
2
3 int socket(int dominio, int tipo, int protocollo);
```

Questa funzione crea una nuova *socket* e restituisce il relativo *file descriptor*. I parametri principali sono:

- **dominio**: specifica la famiglia di indirizzi (es. `AF_INET`, `AF_UNIX`);
- **tipo**: definisce la semantica di comunicazione (es. `SOCK_STREAM`, `SOCK_DGRAM`);
- **protocollo**: indica il protocollo di trasporto (generalmente posto a 0 per la scelta automatica).

2.7.2 bind

Per associare una *socket* a un nome fisico **globale**, si utilizza invece la funzione `bind`:

```
1 #include <sys/socket.h>
2
3 int bind(int socket, const struct sockaddr *address, socklen_t address_length);
```

La funzione `bind` collega la *socket* creata a un indirizzo specificato dalla struttura `sockaddr`. In caso di successo restituisce 0, mentre in caso di errore ritorna -1 e imposta la variabile globale `errno`.

2.8 Socket datagram

Le *socket* di tipo **datagram** rappresentano punti terminali della comunicazione utilizzabili per interagire con qualsiasi processo appartenente allo stesso dominio. Esse supportano due principali ruoli di processo:

- **Cliente**: definisce le variabili di riferimento alla *socket*, conosce l'indirizzo Internet del nodo remoto e la porta del servizio.
- **Server**: definisce le variabili di riferimento alla *socket*, conosce la porta del servizio da offrire e può ricevere da qualsiasi indirizzo IP locale (server **multiporta**).

Le *socket* datagram permettono operazioni dirette di invio e ricezione verso più destinatari, senza necessità di una connessione persistente.

2.8.1 Primitive

sendto La funzione `sendto` consente di inviare un messaggio a un destinatario specifico, identificato dalla struttura di indirizzo passata come parametro. Viene utilizzata per le comunicazioni non connesse, tipiche delle *socket* di tipo datagram.

```
1 #include <sys/socket.h>
2
3 ssize_t sendto(int socket, const void *buf, size_t len, int flags,
4               const struct sockaddr *dest_addr, socklen_t addrlen);
```

recvfrom La funzione `recvfrom` riceve un messaggio da una *socket* datagram e, opzionalmente, memorizza l'indirizzo del mittente nella struttura fornita. È l'equivalente di `recv` per le comunicazioni non orientate alla connessione.

```
1 #include <sys/socket.h>
2
3 ssize_t recvfrom(int socket, void *buf, size_t len, int flags,
4                 struct sockaddr *src_addr, socklen_t *addrlen);
```

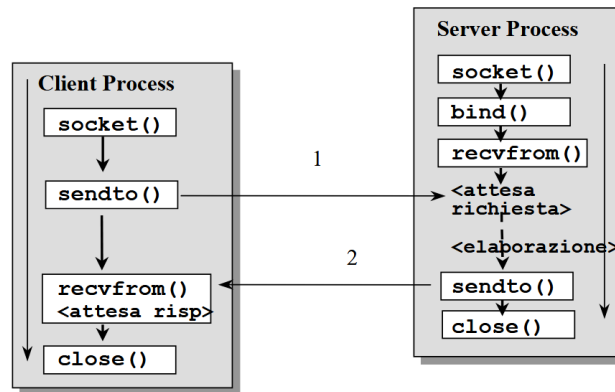
2.8.2 Utilizzo

Prima di iniziare la comunicazione, mittente e destinatario devono inizializzare le rispettive *socket* e allocare i buffer di memoria necessari. I dati vengono scambiati sotto forma di **datagrammi**: messaggi di lunghezza limitata e privi di meccanismi di affidabilità nativi (modello *best-effort*).

Questi messaggi utilizzano la pila protocollare **UDP/IP** e hanno un limite teorico di un di dimensione pari a 9KB.

A livello applicativo (lato utente), è possibile implementare logiche personalizzate per sopperire alle mancanze del protocollo, come:

- **Protocolli Stop-and-Wait:** per limitare l'invio a una singola unità alla volta;
- **Meccanismi di ACK (Acknowledge):** per introdurre servizi di conferma della ricezione e gestione delle ritrasmissioni.



2.8.3 Proprietà

Poiché il protocollo UDP non è affidabile, in caso di perdita di un datagramma il client potrebbe rimanere in attesa della risposta per un tempo indeterminato. Inoltre, non essendo previsto un meccanismo di *flow control* nativo, un server in stato di congestione potrebbe scartare automaticamente i datagrammi in ingresso senza inviare alcuna notifica al mittente.

Per mitigare questi problemi o ottimizzare le prestazioni, è possibile intervenire sulla dimensione dei buffer di sistema dedicati alla trasmissione e alla ricezione utilizzando le opzioni di socket `SO_SNDBUF` e `SO_RCVBUF`.

2.9 Rappresentazione dei dati e Endianness

Nelle architetture dei calcolatori, gli interi composti da più byte possono essere memorizzati secondo due diverse convenzioni di ordinamento dei byte:

- **Little-endian:** il byte meno significativo (*Least Significant Byte - LSB*) viene memorizzato all'indirizzo di memoria più basso.
- **Big-endian:** il byte più significativo (*Most Significant Byte - MSB*) viene memorizzato all'indirizzo di memoria più basso.

Per garantire l'interoperabilità tra sistemi con architetture diverse (eterogeneità), il software di rete distingue tra:

- **Network Byte Order:** l'ordinamento standard utilizzato dai protocolli della suite Internet (corrisponde al **Big-endian**).
- **Host Byte Order:** l'ordinamento nativo utilizzato dalla CPU del sistema locale (può essere *Little-endian* come su x86/x64 o *Big-endian* su alcune architetture RISC).

La conversione tra questi due formati viene gestita attraverso le funzioni standard della libreria **sockets** (`htons`, `htonl`, `ntohs`, `ntohl`), che assicurano che i dati numerici (come numeri di porta e indirizzi IP) siano interpretati correttamente da ogni nodo della rete.

2.10 Funzioni accessorie e manipolazione dati

Per facilitare lo sviluppo di applicazioni di rete, il sistema *socket* e le librerie standard del C mettono a disposizione diverse funzioni di supporto per la conversione degli indirizzi e la gestione dei buffer di memoria.

2.10.1 Conversione degli Indirizzi IP

Queste funzioni permettono di passare dalla rappresentazione testuale (stringa "dotted decimal") a quella binaria utilizzata dalle strutture di rete (*Network Byte Order*):

- `inet_addr()`: Converte una stringa contenente un indirizzo IPv4 nella sua rappresentazione binaria in *network byte order*.
- `inet_ntoa()`: Esegue l'operazione inversa, convertendo un indirizzo di rete `in_addr` in una stringa ASCII (Network to ASCII).

2.10.2 Gestione della Memoria (Buffer)

Le funzioni per la manipolazione dei blocchi di memoria sono essenziali per azzerare le strutture (`sockaddr_in`) o copiare dati tra i buffer:

- **Funzioni standard ANSI C:**
 - `memset()`: Inizializza un'area di memoria con un valore specifico (comunemente usata per azzerare le strutture).
 - `memcpy()`: Copia un numero specificato di byte da un'area di memoria a un'altra.
 - `memcmp()`: Confronta byte per byte due aree di memoria.
- **Funzioni legacy (BSD):**
 - `bzero()`, `bcopy()`, `bcmp()`: Funzioni storiche derivate da BSD. Sebbene ancora presenti in molti sistemi, sono oggi considerate deprecate in favore delle controparti `mem*`.

2.11 Socket Stream

Le *socket stream* implementano il concetto di **connessione virtuale** tra due *endpoint*. A differenza del modello a datagrammi, la comunicazione *stream* richiede una fase preliminare di instaurazione della connessione, che vede il coinvolgimento di due ruoli distinti:

- **Entità Passiva (Server):** si pone in ascolto su un indirizzo e una porta specifici, restando in attesa di richieste di connessione. Utilizza le primitive `bind()`, `listen()` e `accept()`.
- **Entità Attiva (Client):** inizializza la comunicazione indirizzando una richiesta verso l'entità passiva tramite la primitiva `connect()`.

Questo processo di *handshake* (nello specifico il *three-way handshake* di TCP) richiede l'utilizzo di primitive differenziate a seconda del ruolo. Tuttavia, una volta stabilita con successo la connessione, la comunicazione diventa **completamente simmetrica** e **full-duplex**: entrambi gli interlocutori possono inviare e ricevere dati contemporaneamente utilizzando le funzioni `read()/write()` o `recv()/send()`.

La *socket stream* garantisce inoltre che i dati arrivino a destinazione in ordine, senza duplicazioni e senza errori, gestendo internamente la frammentazione e il controllo di flusso.

Una connessione permane fino alla chiusura esplicita tramite la primitiva `close()` o `shutdown()`. Poiché una connessione TCP è composta da due flussi unidirezionali indipendenti (denominati **half-association**), è possibile chiudere la connessione in una sola direzione (lasciando l'altra aperta) o terminare l'intera associazione attraverso il processo di *four-way handshake*.

2.11.1 Primitive

socket La funzione `socket()` crea un endpoint per la comunicazione, restituendo un descrittore di file. È necessario specificare la famiglia di indirizzamento (es. `AF_INET` per IPv4), il tipo di servizio (es. `SOCK_STREAM` per TCP) e il protocollo specifico (solitamente 0 per quello di default).

```
1 #include <sys/socket.h>
2
3 int socket(int domain, int type, int protocol);
```

bind Una socket deve essere legata a un indirizzo nel dominio di comunicazione scelto. La `bind()` associa la socket a un indirizzo locale (IP e porta). A seguito di questa operazione nasce una **half-association** (una tripla composta da protocollo, indirizzo locale e porta locale).

```
1 #include <sys/socket.h>
2
3 int bind(int sockfd, const struct sockaddr *addr,
4         socklen_t addrlen);
```

Mentre i server devono sempre eseguire la `bind()` per essere raggiungibili su una porta nota, i client raramente la chiamano esplicitamente: in tal caso, il sistema operativo assegna automaticamente una porta effimera al momento della connessione.

connect Il client avvia la procedura di connessione verso il server tramite la `connect()`. Questa chiamata è **sincrona**: il processo chiamante si blocca fino a quando la connessione non viene stabilita (inserimento nella coda del server) o si verifica un errore.

```
1 #include <sys/socket.h>
2
3 int connect(int sockfd, const struct sockaddr *addr,
4            socklen_t addrlen);
```

La funzione può comportare tempi di completamento elevati a causa della latenza di rete. In caso di errore, è possibile consultare `errno` per identificare la causa:

- `ECONNREFUSED`: Il server non è in ascolto o ha rifiutato la connessione.
- `ETIMEDOUT`: Nessuna risposta ricevuta entro il timeout previsto.
- `ECONNABORTED`: La connessione è stata interrotta durante l'handshake.

listen Attraverso la funzione `listen()`, il server crea una coda locale per le richieste di connessione in ingresso. È una funzione non bloccante che fallisce solo se eseguita su socket non predisposte.

```
1 #include <sys/socket.h>
2
3 int listen(int sockfd, int backlog);
```

Il parametro `backlog` definisce la lunghezza massima della coda. Se la coda è piena, le successive `connect()` dei client falliranno.

accept Il server estrae la prima richiesta dalla coda tramite `accept()`. È una primitiva bloccante che, in caso di successo, produce una **nuova connessione**.

```
1 #include <sys/socket.h>
2
3 int accept(int sockfd, struct sockaddr *_Nullable restrict addr,
4           socklen_t *_Nullable restrict addrlen);
```

La nuova socket restituita:

- Eredita la semantica di comunicazione della socket originale;
- È associata alla stessa porta locale del server;
- È connessa univocamente alla socket del client (formando una *association* completa).

La socket originale utilizzata per la `listen()` rimane inalterata e pronta per nuove richieste. L'interazione tra `accept()` e `connect()` realizza un **rendez-vous**: il livello applicativo opera in modo asimmetrico, mentre il livello di trasporto coordina le entità in modo sincrono.

read e write Le primitive `read()` e `write()` costituiscono l'interfaccia di base per il trasferimento dei dati su una socket dopo che la connessione è stata stabilita. A differenza delle varianti specifiche per le socket, queste funzioni non prevedono il parametro `flags`.

```
1 #include <unistd.h>
2
3 ssize_t read(int fd, void *buf, size_t count);
4 ssize_t write(int fd, const void *buf, size_t count);
```

La `write()` copia i dati dal buffer dell'applicazione al buffer di invio della socket gestito dal kernel; l'operazione è considerata conclusa con successo non appena i dati sono stati trasferiti nel buffer di sistema. La `read()` preleva i dati dal buffer di ricezione; se il numero di byte disponibili è inferiore a `count`, la funzione restituisce immediatamente i byte presenti senza attendere il completamento della dimensione richiesta.

send Per inviare dati su una connessione attiva si utilizza la primitiva `send()`, che estende le funzionalità della `write`.

```
1 #include <sys/socket.h>
2
3 ssize_t send(int sockfd, const void *buf, size_t len, int flags);
```

Il parametro `flags` permette di specificare comportamenti particolari, come `MSG_OOB` per l'invio di dati *out-of-band* (prioritari). La funzione restituisce il numero di byte effettivamente inviati, che può essere inferiore a `len` in caso di saturazione dei buffer.

recv La primitiva `recv()` permette di leggere i dati dal buffer di ricezione della socket. A differenza della `read()` standard, offre un controllo granulare tramite il parametro `flags`.

```
1 #include <sys/socket.h>
2
3 ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

La primitiva ha comportamento **bloccante**, il quale prevede che, se il buffer è vuoto, il processo venga sospeso fino all'arrivo di nuovi dati o alla chiusura della connessione. Il valore di ritorno indica i byte letti (0 segnala una *orderly shutdown* del peer, -1 indica un errore) e tramite il parametro `flags` si possono attivare opzioni come `MSG_PEEK` (legge i dati senza rimuoverli dal buffer), `MSG_WAITALL` (attende il riempimento completo di `len`) o `MSG_DONTWAIT` (rende l'operazione non bloccante).

2.11.2 Sincronia e Gestione dei Buffer

La lettura e la scrittura a livello applicativo sono operazioni tipicamente **sincrone** rispetto al driver TCP sottostante. Questo significa che il kernel coordina lo stato di avanzamento del processo chiamante in base alla disponibilità delle risorse di rete (buffer).

Il comportamento del driver influenza direttamente l'esecuzione del programma:

- **Sospensione in invio (send/write):** Se il buffer di trasmissione del kernel è pieno (ad esempio a causa di una congestione di rete o di un ricevitore lento), il driver può sospendere la `send` o la `write`. Il processo chiamante viene messo in stato di attesa (*waiting*) finché non si libera spazio sufficiente per ospitare i dati da inviare.

- **Sospensione in ricezione (recv/read):** Se il buffer di ricezione del kernel è vuoto, il driver sospende la `recv` o la `read`. Il processo rimane in attesa finché il driver non riceve dal network almeno un byte da consegnare all'applicazione.

Questo meccanismo di *backpressure* assicura che l'applicazione non proceda ignorando lo stato reale della connessione, mantenendo l'integrità del trasferimento dei dati nel modello a stream.

2.11.3 Utilizzo e Gestione dei Messaggi

L'invocazione di una primitiva di invio non comporta necessariamente l'invio immediato di un pacchetto sulla rete. Il protocollo TCP opera una **bufferizzazione** dei dati: i byte vengono raggruppati e trasmessi solo quando il driver TCP lo ritiene opportuno (ad esempio al raggiungimento della dimensione MSS o alla scadenza di un timer), ottimizzando l'efficienza del trasporto.

Poiché TCP è un protocollo orientato al **flusso di byte** e non ai messaggi, non implementa marcatori di fine messaggio. In ricezione, una `read` o una `recv` restituisce semplicemente i byte disponibili nel buffer locale, che potrebbero corrispondere a una frazione di messaggio o a più messaggi concatenati.

Per risolvere questo problema di *frammentazione e riassetblaggio*, si adottano diverse strategie a livello applicativo:

- **Messaggi a lunghezza fissa:** Client e server concordano una dimensione fissa per ogni pacchetto. È la soluzione più semplice, ma inefficiente se i dati reali variano molto in dimensione.
- **Opzione Watermark:** Si configura il driver per segnalare la disponibilità dei dati solo quando il buffer di ricezione ha raggiunto una determinata soglia (*low-water mark*).
- **Descrittori espliciti di lunghezza (Framing):** Per gestire messaggi a lunghezza variabile, si utilizza una tecnica a due fasi:
 1. Si invia un'intestazione (*header*) a lunghezza fissa che contiene esclusivamente il valore numerico della lunghezza del messaggio successivo.
 2. Si effettua una seconda lettura, utilizzando il valore appena ricevuto per sapere esattamente quanti byte leggere per completare il corpo del messaggio.

Questo metodo di "letture in successione" è lo standard de facto nelle comunicazioni via socket stream per garantire che l'applicazione ricevente possa ricostruire correttamente i dati inviati.

2.11.4 Terminazione della comunicazione

In una connessione TCP, i dati fluiscono come uno stream continuo fino a quando uno dei due interlocutori non segnala esplicitamente la fine del file (*EOF*). Poiché la connessione è **full-duplex**, la chiusura può essere gestita in modo asimmetrico o simmetrico, seguendo un protocollo rigoroso per evitare la perdita di dati residui.

Il meccanismo di terminazione si basa sui seguenti principi:

- **Segnalazione di Fine Flusso:** Quando un'autorità (client o server) ha terminato l'invio dei dati, invoca una chiusura (`close` o `shutdown`). Questa operazione invia un segmento speciale (FIN) che viene interpretato dal ricevente come un'indicazione di **fine file**.
- **Consumo dei dati:** La chiusura non è istantanea. Chi chiude il flusso deve aspettare che tutti i dati già inviati e presenti nei buffer di rete siano stati effettivamente consumati dal destinatario.
- **Protocollo degli Endpoint:** Per una terminazione pulita, ogni endpoint deve osservare due regole fondamentali:
 1. Deve continuare a leggere l'input dal *source* finché la primitiva di ricezione non restituisce 0 (indicazione di fine flusso).
 2. Deve inviare a sua volta una notifica di fine flusso quando intende cessare l'invio dei dati sulla propria direzione di uscita.

close La funzione `close()` viene utilizzata per interrompere il riferimento di un processo a un descrittore di socket. La sua esecuzione a livello applicativo è immediata, ma innesca una serie di operazioni complesse a livello di kernel.

```
1 #include <unistd.h>
2
3 int close(int fd);
```

- **Gestione dei riferimenti:** La `close()` decrementa il contatore dei processi che referenziano il *socket descriptor*. Il controllo viene restituito immediatamente al chiamante (**primitiva passante e senza attesa**), che non potrà più utilizzare quel descrittore per alcuna operazione.
- **Rilascio delle risorse:** Il processo segnala al sistema operativo di liberare le risorse locali. Tuttavia, il rilascio delle risorse remote gestite dal driver TCP avviene con tempi non prevedibili, poiché dipende dallo svuotamento dei buffer e dai messaggi di controllo (*FIN/ACK*) scambiati sulla rete.
- **Opzione SO_LINGER:** Di default, la chiusura è istantanea per l'applicazione ma "pigra" per il kernel. Attraverso l'opzione avanzata `SO_LINGER`, è possibile forzare la `close()` a diventare bloccante finché tutti i dati nel buffer non sono stati spediti o finché non scade un timeout predefinito.

Ogni socket a stream è associata a buffer di memoria per i dati in ingresso e in uscita. Alla chiamata della `close()`, il sistema segue una politica differenziata:

1. **Dati in uscita:** Ogni messaggio rimasto nel buffer di trasmissione deve essere spedito (impiegando il tempo necessario secondo le dinamiche del protocollo TCP).
2. **Dati in ingresso:** Qualsiasi dato già ricevuto dal driver ma non ancora letto dall'applicazione tramite `read/recv` viene scartato, liberando la memoria della porta.

Una volta che la chiusura (e la relativa notifica di *EOF*) raggiunge il peer connesso:

- Se il peer esegue una **lettura**, otterrà come risultato **0** (indicazione di fine file).
- Se il peer tenta una **scrittura**, dopo alcuni tentativi falliti riceverà dal sistema il segnale **SIGPIPE** (*broken pipe*), indicante che la connessione non è più esistente.

La durata totale della persistenza della memoria nel driver TCP può essere molto lunga (anche nell'ordine dei minuti, come nello stato *TIME_WAIT*), impegnando le risorse di sistema ben oltre la terminazione del processo applicativo.

shutdown La primitiva `shutdown()` permette di terminare la connessione gestendo separatamente i due versi del flusso (*half-association*), ottimizzando l'uso delle risorse e il coordinamento tra i peer. A differenza della `close()`, che agisce sull'intero descrittore, la `shutdown()` agisce direttamente sullo stato della connessione TCP.

```
1 #include <sys/socket.h>
2
3 int shutdown(int s, int how);
```

Il comportamento della funzione dipende dal parametro `how`:

- `how = 0` (`SHUT_RD`): Chiude la direzione di **input**. L'applicazione non può più ricevere dati. Se il peer tenta una `send()`, riceverà un errore (`-1`) e potenzialmente il segnale **SIGPIPE**.
- `how = 1` (`SHUT_WR`): Chiude la direzione di **output**. L'applicazione non può più trasmettere dati, ma può ancora riceverne. Questa è la modalità più utilizzata per segnalare la fine della trasmissione (*EOF*) senza interrompere la capacità di ascolto.
- `how = 2` (`SHUT_RDWR`): Chiude entrambi i versi della comunicazione, combinando gli effetti precedenti.

L'utilizzo tipico della `shutdown` permette di implementare un protocollo di terminazione robusto:

1. **Segnalazione:** Un peer decide di terminare la propria trasmissione e invoca `shutdown`. Questo invia un pacchetto `FIN` al destinatario.
2. **Ricezione EOF:** Il peer ricevente legge i dati residui finché la `read()` non restituisce 0 (*end-of-file*). A questo punto, il ricevente sa che non arriveranno più dati da quel mittente.
3. **Asimmetria:** Nonostante il primo peer abbia chiuso l'output, il secondo peer può continuare a utilizzare il proprio verso di invio per tutto il tempo necessario, finché non deciderà a sua volta di invocare la `shutdown()` o la `close()`.

Questo approccio garantisce che tutte le richieste vengano completate e che le risorse dei buffer siano deallocate solo quando entrambi i flussi unidirezionali sono stati effettivamente conclusi.

2.11.5 Processi Naive e Redirezione su Socket

I processi cosiddetti **naive** sono programmi progettati per funzionare come filtri: leggono i dati dallo *standard input* (`stdin`) e producono risultati sullo *standard output* (`stdout`), senza essere consapevoli della natura del canale di comunicazione sottostante.

Poiché in ambiente Unix "tutto è un file", se un processo utilizza le primitive `read()` e `write()`, queste possono operare indistintamente su file regolari, pipe o **socket**. È quindi possibile connettere un processo locale a un host remoto seguendo un protocollo di attivazione specifico:

1. **Preparazione:** Il processo padre crea le socket necessarie e stabilisce la connessione con il nodo remoto.
2. **Generazione del processo (fork):** Il padre crea un processo figlio tramite la `fork()`. Il figlio eredita i descrittori delle socket aperte.
3. **Redirezione (dup2):** All'interno del figlio, i descrittori delle socket vengono duplicati sui descrittori 0 (`stdin`), 1 (`stdout`) e, se necessario, 2 (`stderr`). In questo modo, le socket "sostituiscono" i canali standard.
4. **Esecuzione (exec):** Il figlio esegue il comando locale tramite una funzione della famiglia `exec()`. Il nuovo codice mantiene i descrittori di file aperti dal processo precedente.

Il risultato è che il processo naive, pur continuando a leggere e scrivere sui canali standard, comunica automaticamente con processi remoti attraverso la rete. Questa tecnica è alla base del funzionamento di storici servizi come **inetd** (il super-server Internet) e delle moderne implementazioni di **SSH** per l'esecuzione di comandi remoti.

2.12 API Socket in C

Riepilogando le principali funzioni fornite dall'interfaccia *socket* per la comunicazione di rete e la gestione del sistema:

```
1  /* Creazione e Connessione */
2  int socket(int domain, int type, int protocol);
3  int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
4
5  /* I/O Generico (POSIX) */
6  ssize_t write(int fd, const void *buf, size_t count);
7  ssize_t read(int fd, void *buf, size_t count);
8  int close(int fd);
9
10 /* Configurazione Server */
11 int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
12 int listen(int sockfd, int backlog);
```

```

13 int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
14
15 /* I/O Specifico per Socket (TCP/Connessi) */
16 ssize_t send(int sockfd, const void *buf, size_t len, int flags);
17 ssize_t recv(int sockfd, void *buf, size_t len, int flags);
18
19 /* I/O per Datagram (UDP/Non connessi) */
20 ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
21               const struct sockaddr *dest_addr, socklen_t addrlen);
22 ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
23                 struct sockaddr *src_addr, socklen_t *addrlen);
24
25 /* I/O Avanzato (Vettoriale) */
26 ssize_t sendmsg(int sockfd, const struct msghdr *msg, int flags);
27 ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);
28
29 /* Gestione Stato e Opzioni */
30 int shutdown(int sockfd, int how);
31 int getsockname(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
32 int getpeername(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
33 int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen
34               );
35 int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t
36               optlen);
37
38 /* Diagnostica e Logging */
39 void perror(const char *s);
40 void syslog(int priority, const char *format, ...);

```

2.13 Modalità delle socket

Le primitive UNIX standard sono intrinsecamente **sincrone e bloccanti**: l'esecuzione del processo viene sospesa dal kernel in attesa che il driver di autorità (il driver di rete o del file system) completi l'operazione richiesta.

Per ottenere flessibilità e prestazioni superiori, è possibile alterare questo comportamento predefinito attraverso due meccanismi principali:

- **Opzioni delle socket:** Agiscono su parametri specifici e limitati (es. `SO_REUSEADDR`, `SO_RCVBUF`).
- **Funzioni di controllo del kernel (ioctl / fcntl):** Permettono cambiamenti strutturali nel comportamento della socket, rendendo possibile la gestione di I/O asincrono o non bloccante.

2.13.1 Opzioni per le socket

Le funzioni `getsockopt()` e `setsockopt()` sono primitive di utilità fondamentali per leggere e modificare i parametri operativi delle socket. Molte di queste opzioni agiscono direttamente sul comportamento dello stack TCP/IP nel kernel.

```

1 #include <sys/types.h>
2 #include <sys/socket.h>
3
4 int getsockopt(int s, int level, int optname, void *optval,
5               socklen_t *optlen);
6
7 int setsockopt(int s, int level, int optname, const void *optval,
8               socklen_t optlen);

```

- **level**: Specifica il livello dello stack a cui l'opzione appartiene (es. `SOL_SOCKET` per opzioni generiche, `IPPROTO_TCP` per opzioni specifiche del protocollo TCP).
- **optname**: L'identificativo costante dell'opzione che si desidera manipolare.

Opzione	Descrizione
<code>SO_DEBUG</code>	Abilita il debugging nel kernel per i pacchetti inviati/ricevuti.
<code>SO_REUSEADDR</code>	Permette il riutilizzo immediato di un indirizzo locale (IP e porta) anche se la socket è in stato <i>TIME_WAIT</i> . Essenziale per il riavvio rapido dei server.
<code>SO_DONTROUTE</code>	Invia i pacchetti direttamente all'interfaccia, ignorando le tabelle di routing.
<code>SO_LINGER</code>	Controlla il comportamento della <code>close()</code> : se abilitata, la chiusura attende la consegna dei messaggi pendenti.
<code>SO_BROADCAST</code>	Permette l'invio di messaggi broadcast (solo per protocolli datagram).
<code>SO_SNDBUF</code> / <code>RCVBUF</code>	Imposta le dimensioni massime in byte dei buffer di invio e ricezione del sistema operativo.
<code>SO_SNDTIMEO</code> / <code>RCVTIMEO</code>	Definisce un timeout per le operazioni di input/output; se superato, la primitiva ritorna un errore.
<code>SO_KEEPALIVE</code>	Abilita l'invio periodico di messaggi di controllo per verificare se il peer è ancora attivo (non presente in tabella originale ma fondamentale).

- **optval**: Puntatore a un buffer che contiene il valore dell'opzione. Tipicamente è un intero utilizzato come booleano.

2.13.2 Dimensioni dei buffer

Le opzioni `SO_SNDBUF` e `SO_RCVBUF` permettono di intervenire sulla memoria allocata dal kernel per la trasmissione e ricezione. Aumentare questi valori (fino al limite teorico di 65535 byte, salvo estensioni del kernel) consente di gestire messaggi di grandi dimensioni riducendo le sospensioni del processo dovute a buffer pieni.

2.13.3 Riutilizzo degli indirizzi

Di norma, il sistema operativo garantisce l'unicità dell'associazione indirizzo locale/porta. Se un processo tenta di legarsi (`bind`) a una porta ancora impegnata (anche solo nello stato di chiusura *TIME_WAIT*), la chiamata fallisce. L'opzione `SO_REUSEADDR` è fondamentale per i server che devono garantire alta disponibilità:

- Permette il **riavvio immediato** dopo un crash o una chiusura, senza attendere i tempi di deallocazione della memoria della porta (che possono durare decine di secondi).
- Deve essere impostata **prima** della chiamata a `bind()`.

```

1 int optval = 1;
2 int s = socket(AF_INET, SOCK_STREAM, 0);
3 setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));
4 bind(s, &sin, sizeof(sin));

```

2.13.4 Gestione della terminazione (`SO_LINGER`)

L'opzione `SO_LINGER` modifica il comportamento della `close()` agendo sulla struttura `linger`:

```

1 struct linger {
2     int l_onoff;    /* 0 = off, 1 = on */
3     int l_linger;   /* tempo di attesa in secondi */
4 };

```

Configurazioni possibili:

1. **Default** (`l_onoff = 0`): La `close()` ritorna subito; il sistema tenta di inviare i dati residui in background (impegnativo per le risorse TCP).
2. **Chiusura "Hard"** (`l_onoff = 1, l_linger = 0`): Ogni dato ancora presente nel buffer di invio viene scartato immediatamente. La connessione viene interrotta bruscamente con un segnale di *Reset* (RST).
3. **Chiusura "Graceful"** (`l_onoff = 1, l_linger > 0`): La `close()` diventa una primitiva **bloccante**. Il processo attende fino alla trasmissione di tutti i dati o alla scadenza del tempo `l_linger`. Questo garantisce l'allineamento tra lo stato dell'applicazione e quello del trasporto.

2.14 Asincronismo

Il design dei kernel moderni permette di gestire le socket con comportamenti non standard per ottimizzare l'efficienza, specialmente in scenari di I/O intensivo. Mentre il modello di lavoro normale è **sincrono** (il processo attende il risultato dal kernel), è possibile configurare le socket per operare in modo asincrono o non bloccante utilizzando le primitive di controllo `fcntl` e `ioctl`.

Queste funzioni, originariamente progettate per l'interazione diretta con i driver di dispositivo, permettono di gestire i file descriptor (e quindi le socket) senza attendere il completamento delle operazioni, rimandando il recupero del risultato a un momento successivo.

In una socket configurata come asincrona, il kernel avvisa il processo di un cambiamento di stato (come l'arrivo di nuovi dati) tramite l'invio di un segnale specifico: **SIGIO**.

```

1 #include <sys/ioctl.h>
2 #include <unistd.h>
3
4 int on = 1;
5 /* Abilitazione della modalita asincrona sulla socket */
6 ioctl(sockfd, FIOASYNC, &on);
7
8 /* Impostazione del proprietario della socket per ricevere i segnali */
9 fcntl(sockfd, F_SETOWN, getpid());

```

Perché il kernel possa recapitare correttamente il segnale **SIGIO** in risposta a un evento di rete, non basta attivare la modalità asincrona; è necessario associare esplicitamente la socket a un processo o a un gruppo di processi destinatari.

Mentre `FIOASYNC` abilita la generazione del segnale, la primitiva `ioctl()` con l'attributo `SIOCSPGRP` viene utilizzata per impostare il *Process Group ID* che deve ricevere la notifica. In particolare, il comportamento cambia in base al segno del valore passato:

- **Valore negativo:** Il valore (preso in modulo) viene interpretato come un **Process ID (PID)**. Il segnale **SIGIO** verrà inviato esclusivamente al singolo processo specificato.
- **Valore positivo:** Il valore viene interpretato come un **Process Group ID (PGID)**. Il segnale **SIGIO** verrà inviato a tutti i processi appartenenti a quel gruppo, permettendo una gestione coordinata dell'evento.

2.15 Non bloccanti

Sebbene si possano ottenere comportamenti simili attraverso diverse strade, la modalità **non bloccante** pura si distingue perché non utilizza segnali (come l'asincronismo), ma modifica direttamente il comportamento delle primitive di I/O (`read`, `write`, `accept`, `connect`).

Il non blocco può essere attivato tramite la primitiva `ioctl()` utilizzando il parametro `FIONBIO` (*File I/O Non-Blocking*):

- **Valore 0:** Modalità **default bloccante**. La primitiva non ritorna finché l'operazione non è completata.
- **Valore 1:** Modalità **non bloccante**. La primitiva ritorna immediatamente.

2.15.1 Effetti sulle primitive

Quando una socket è impostata come non bloccante, le primitive cambiano comportamento nel caso in cui l'operazione non possa essere soddisfatta istantaneamente:

- **In lettura (`read/recv`):** Se il buffer è vuoto, la funzione non attende l'arrivo di dati ma restituisce immediatamente `-1`, impostando `errno` a `EWOULDBLOCK` o `EAGAIN`.
- **In scrittura (`write/send`):** Se il buffer di invio è pieno, la funzione restituisce errore invece di sospendere il processo.
- **In accettazione (`accept`):** Se non ci sono richieste di connessione pendenti nella coda, la funzione fallisce subito.

Questa modalità impone al programmatore l'adozione di una tecnica di **polling** (interrogazione ciclica) o, più efficientemente, l'uso di meccanismi di multiplexing per sapere quando riprovare l'operazione.

2.16 Multiple Receive e Multiplexing

Le primitive su socket, essendo sincrone e bloccanti, impediscono a un processo di gestire contemporaneamente più sorgenti di dati. Se un server deve servire più client o monitorare diversi eventi (es. tastiera e rete), una `read()` su un canale vuoto bloccherebbe il processo, rendendolo incapace di servire altri canali dove i dati sono già pronti.

Per risolvere questo problema si utilizza il **multiplexing**, che permette di tenere aperte più azioni senza scegliere a priori quale servire. La gestione avviene tramite una primitiva che blocca il processo non su una singola operazione, ma sull'attesa di **almeno un evento** tra quelli configurati.

2.16.1 La primitiva select

La `select()` è una primitiva di attesa multipla sincrona che include un meccanismo di **timeout intrinseco** per evitare attese infinite.

```
1 #include <sys/select.h>
2 #include <sys/time.h>
3
4 int select(int nfds, fd_set *readfds, fd_set *writefds,
5           fd_set *exceptfds, struct timeval *timeout);
6
7 /*
8  * nfds:      Valore del descrittore pi alto + 1 (es. max_fd + 1)
9  * readfds:   Insieme di bit per monitorare la leggibilità (dati pronti/nuove
10              connessioni)
11 * writefds:  Insieme di bit per monitorare la scrivibilità (buffer di invio
12              libero)
13 * exceptfds: Insieme di bit per condizioni eccezionali (dati Out-of-Band)
14 * timeout:   Puntatore a struct timeval (NULL=attesa infinita, 0=polling, >0=
15              tempo max)
```

```

13  * Ritorna:    n > 0 (n. descrittori pronti), 0 (timeout), -1 (errore)
14  */

```

2.16.2 Eventi della select

La `select()` sospende il processo finché non si verifica una delle seguenti condizioni su uno o più descrittori monitorati:

- **Eventi di lettura (Readability):** Rendono la `recv()` o la `accept()` non bloccanti.
 - Presenza di dati nel buffer di ricezione.
 - Nuova richiesta di connessione su socket passiva (pronta per `accept()`).
 - Chiusura della connessione (EOF) o errore.
- **Eventi di scrittura (Writability):** Segnalano che il buffer di invio ha spazio.
 - Connessione completata con successo (`connect()`).
 - Possibilità di spedire altri dati senza sospensione.
 - Chiusura del pari (SIGPIPE) o errore.
- **Eventi eccezionali:** Segnalano condizioni anomale.
 - Arrivo di dati *out-of-band* (prioritari).
 - Inutilizzabilità della socket o errori gravi.

2.16.3 Maschere e bitset

La `select()` opera su tre maschere di bit (`fd_set`), dove ogni bit corrisponde a un file descriptor (FD). La posizione del bit coincide con il valore numerico del descrittore: ad esempio, il bit in posizione 4 controlla il file descriptor 4. Operativamente:

- **In ingresso:** Il processo imposta a **1** i bit corrispondenti ai descrittori che desidera monitorare. Il parametro `nfds` indica al kernel l'ampiezza della scansione (da 0 a `nfds-1`).
- **In uscita:** Al ritorno della primitiva, il kernel ha **sovrascritto** le maschere. Solo i bit relativi ai descrittori dove l'evento si è effettivamente verificato rimangono a **1**, tutti gli altri vengono azzerati (**0**).

Se, ad esempio, monitoriamo i descrittori 4, 5, 7, 9 (con `nfds = 10`) e solo il 5 e il 7 ricevono dati, la maschera restituita avrà bit accesi solo in posizione 5 e 7.

Macro di gestione Poiché la struttura interna delle maschere dipende dall'architettura (solitamente array di `long` o `int`), l'accesso ai bit avviene tramite macro standard:

```

1  /* Inizializza la maschera azzerando tutti i bit */
2  void FD_ZERO(fd_set *fdset);
3
4  /* Imposta a 1 il bit corrispondente al descrittore fd */
5  void FD_SET(int fd, fd_set *fdset);
6
7  /* Resetta (porta a 0) il bit corrispondente al descrittore fd */
8  void FD_CLR(int fd, fd_set *fdset);
9
10 /* Verifica se il bit per fd è impostato a 1 (usata dopo la select) */
11 int FD_ISSET(int fd, fd_set *fdset);

```

2.17 Server Concorrente Monoprocesso

Un server concorrente monoprocesso mira a servire molteplici client simultaneamente senza i ritardi tipici di un approccio sequenziale e senza l'overhead di memoria della creazione di nuovi processi o thread. L'intero coordinamento si basa sulla gestione dinamica dei descrittori di file (*socket descriptors*) all'interno della maschera della `select()`.

2.17.1 Gestione dei File Descriptor

In ambiente UNIX, i descrittori vengono assegnati dal kernel seguendo una logica predicibile:

- I descrittori **0**, **1**, **2** sono riservati a *stdin*, *stdout* e *stderr*.
- La prima socket creata dal server per l'ascolto (*listening socket*) occuperà solitamente il **fd 3**.
- Ogni nuova connessione accettata tramite `accept()` occuperà il valore libero più basso disponibile (es. 4, 5, 6...).
- Alla chiusura (`close()`) di una connessione, il descrittore viene rilasciato e il kernel lo riassegnerà alla prossima `accept()` seguendo l'ordine crescente dal basso.

2.17.2 Sincronizzazione della Maschera

Il cuore del server è il ciclo principale che deve mantenere una maschera (`fd_set`) consistente con lo stato attuale delle connessioni aperte.

- **Socket di Listen:** Deve essere sempre presente nella maschera di lettura. Quando la `select()` segnala un evento su questo descrittore, significa che c'è una nuova richiesta di connessione pronta per la `accept()`.
- **Aggiornamento Dinamico:** Ogni volta che una `accept()` apre un nuovo descrittore (es. il 5), questo deve essere aggiunto immediatamente alla maschera dei descrittori da monitorare tramite `FD_SET(5, master_set)`.
- **Rimozione al termine:** Quando un client chiude la connessione (ricezione di EOF), il server deve invocare la `close(5)` e rimuovere il bit corrispondente dalla maschera tramite `FD_CLR(5, master_set)`.

Poiché la `select()` blocca il processo solo quando *nessun* client ha dati pronti, il server può processare le richieste non appena arrivano. Se più client inviano dati contemporaneamente, la `select()` ritorna segnalandoli tutti, permettendo al server di iterare sui descrittori pronti e servirli uno dopo l'altro senza mai bloccarsi su una singola lettura vuota.

2.18 Select in scrittura

L'utilizzo della `select()` con la maschera di scrittura (`wrtefds`) differisce concettualmente dall'uso in lettura. Se in lettura il processo attende un evento causato da entità remote (arrivo di datagrammi o dati), in scrittura l'evento dipende dalla disponibilità di risorse locali (lo spazio nel buffer di invio del kernel). In modalità standard (sincrona bloccante), l'uso della `select` in scrittura è pressoché inutile:

- Se una `send()` blocca il processo, questo non può invocare la `select()` per monitorare l'evento.
- L'evento di "disponibilità alla scrittura" si verifica quasi sempre, a meno che i buffer del kernel non siano completamente saturi.

La maschera di scrittura diventa fondamentale quando si opera con socket in modalità **asincrona** o **non bloccante**. In questo scenario, la `select()` svolge due funzioni critiche:

1. **Completamento di operazioni differite:** Quando si usa una `connect()` non bloccante, la funzione ritorna immediatamente. Il processo usa la `select()` in scrittura per sapere quando la connessione con il peer è stata effettivamente stabilita.

2. **Controllo del flusso (Flow Control):** Se una `send()` non bloccante restituisce un errore (`EWOULDBLOCK`) perché il buffer è pieno, il processo si sospende sulla `select()` (maschera di scrittura). La `select()` si sbloccherà solo quando il kernel avrà inviato sufficienti dati sulla rete, liberando spazio nel buffer locale per la successiva operazione di scrittura.

In sintesi, la `select` in scrittura permette di gestire il "dopo-azione" in contesti non bloccanti, segnalando al processo che è possibile procedere con il comando successivo senza rischiare la sospensione.

2.19 Server Multifunzione

Un server multifunzione agisce come un punto di ingresso unico per diversi servizi di rete. Questo approccio evita la proliferazione di processi "dormienti" in attesa di connessioni, centralizzando la gestione delle socket in un unico processo master.

Il caso più celebre è `inetd` (*Internet Daemon*), noto come il "super-server" di BSD UNIX. Il suo funzionamento si basa su pochi passaggi chiave:

1. Legge un file di configurazione (`/etc/inetd.conf`) per sapere quali servizi deve gestire.
2. Crea una socket per ogni servizio elencato e si mette in attesa su tutte contemporaneamente utilizzando la `select()`.
3. Quando arriva una richiesta, `inetd` esegue una `fork()` e delega l'esecuzione del servizio a un processo figlio specializzato tramite `exec()`.

2.19.1 Configurazione di `inetd.conf`

Il comportamento del demone è definito da un file di testo che specifica, per ogni servizio, come gestire la comunicazione e quale processo attivare. La sintassi standard è la seguente:

```
<service_name> <socket_type> <proto> <flags> <user> <server_pathname> <args>
```

- **socket_type:** Solitamente `stream` (per TCP) o `dgram` (per UDP).
- **proto:** Il protocollo di trasporto (`tcp`, `udp`, o `rpc/udp`).
- **flags:**
 - **nowait:** Tipico dei servizi `stream`. `inetd` accetta la connessione e lancia subito un altro server, tornando immediatamente in ascolto sulla porta principale.
 - **wait:** Tipico dei servizi `dgram`. `inetd` attende che il server figlio termini prima di riprendere il monitoraggio di quella socket.
- **server_pathname:** Il percorso dell'eseguibile da lanciare. Se indicato come `internal`, il servizio è gestito direttamente dal codice di `inetd` (es. `echo`, `daytime`).

2.20 Client Concorrente e Parallelo

Anche il lato client può trarre enormi benefici dalla gestione della concorrenza, specialmente quando deve interagire con più server o gestire flussi di dati multipli senza bloccarsi. Si distinguono due approcci principali:

2.20.1 Concorrente (Monoprocesso)

In questa configurazione, un unico processo client gestisce più interazioni contemporaneamente sfruttando l'asincronismo. Il client non aspetta la risposta di una singola richiesta prima di inviare la successiva.

- **Meccanismo:** Il client invia una serie di richieste in sequenza rapida (senza attesa) e poi si mette in ascolto tramite la `select()`.

- **Vantaggi:** Minore overhead di sistema rispetto alla creazione di processi; gestione centralizzata dello stato di tutte le connessioni.
- **Gestione:** Richiede politiche di servizio opportune per ricomporre le risposte che potrebbero arrivare in ordine diverso rispetto alle richieste inviate.

2.20.2 Parallela (Multiprocesso)

Questo approccio prevede la generazione di più processi "slave" (tramite `fork()`), dove ogni processo si occupa di una singola interazione specifica con un server.

- **Meccanismo:** Ogni processo slave è indipendente e può utilizzare primitive sincrone bloccanti, poiché il suo eventuale blocco non influisce sugli altri processi paralleli.
- **Multicast e Interazione Multi-Server:** È particolarmente utile per interagire con più server contemporaneamente. Ad esempio, in un contesto di **multicast**, un processo padre può coordinare diversi figli che attendono risposte da nodi differenti della rete.
- **Vantaggi:** Semplicità di programmazione (si usano le primitive standard); isolamento dei guasti (se un figlio crasha, gli altri continuano).

3 OSI

Il modello **OSI** (*Open Systems Interconnection*) è uno standard di comunicazione per sistemi aperti, sviluppato dall'ISO per garantire l'interoperabilità tra architetture eterogenee. OSI definisce un quadro concettuale per la gestione dei sistemi remoti, un ambito noto come **Systems Management** o **Network Management**. L'obiettivo principale del modello è fornire le basi per controllare, coordinare e monitorare sistemi interconnessi indipendentemente dalla loro natura hardware o software, eliminando i vincoli legati alla località fisica o alla coresidenza dei componenti.

Il modello si articola in sette livelli gerarchici, ciascuno dei quali offre servizi al livello superiore e utilizza quelli del livello inferiore, garantendo una separazione netta delle responsabilità (incapsulamento).

3.1 Caratteristiche dello Standard

Il modello OSI non nasce come un protocollo implementativo (come la suite TCP/IP), bensì come uno standard di **schemi di progetto astratti**. Il suo scopo è razionalizzare, inquadrare e abilitare l'interoperabilità tra diverse tecnologie di comunicazione. Le sue caratteristiche principali sono:

- **Organizzazione gerarchica:** Il sistema è strutturato in sette livelli indipendenti; ogni livello comunica logicamente con il suo omologo sul sistema remoto (*peer-to-peer*) e fisicamente solo con i livelli adiacenti.
- **Descrizione orientata agli oggetti:** Le risorse e i servizi sono modellati come oggetti, facilitando la definizione di interfacce standardizzate.
- **Astrazione realizzativa:** Essendo un modello di riferimento, non impone specifiche implementazioni hardware o software, ma definisce *cosa* un livello deve fare, non *come* debba essere realizzato.
- **Scenario di riferimento omnicomprensivo:** È progettato per coprire ogni aspetto della comunicazione, dalla trasmissione dei bit fisici fino alle interfacce utente finali.

L'architettura OSI si basa sulla separazione tra **servizio** (l'insieme di funzioni offerte a un livello superiore), **protocollo** (le regole per lo scambio di dati tra entità dello stesso livello) e **interfaccia** (il punto di accesso attraverso cui il livello superiore utilizza i servizi del livello inferiore).

3.2 Comitati

La definizione e il mantenimento del modello OSI sono affidati a comitati internazionali che collaborano per garantire l'uniformità delle specifiche. Si distinguono generalmente tra organismi pubblici (istituzionali) e privati (industriali).

3.2.1 Comitati Internazionali Pubblici

- **ISO (International Organization for Standardization):** Il principale ente sviluppatore del modello OSI, responsabile della standardizzazione a livello mondiale in quasi tutti i settori tecnici.
- **IEC (International Electrotechnical Commission):** Focalizzata sugli standard per le tecnologie elettriche ed elettroniche, collabora strettamente con l'ISO.
- **ITU-T (già CCITT):** L'Unione Internazionale delle Telecomunicazioni (settore Standardizzazione). È l'organismo governativo che definisce i protocolli per le reti pubbliche di telecomunicazione.

3.2.2 Comitati Internazionali Privati e Professionali

Questi organismi sono composti da esperti del settore industriale e accademico e definiscono standard tecnici che vengono spesso adottati dall'ISO:

- **IEEE (Institute of Electrical and Electronics Engineers):** Fondamentale per i livelli bassi del modello OSI (Fisico e Data Link), come gli standard 802.3 (Ethernet) e 802.11 (Wi-Fi).

- **ECMA International:** Organismo europeo dedicato alla standardizzazione di sistemi informativi e di comunicazione.
- **EIA (Electronic Industries Alliance):** Nota per la definizione di interfacce fisiche e standard di cablaggio (come la RS-232).

3.3 Livelli

L'architettura OSI mira a fornire una descrizione rigorosa e sistematica delle comunicazioni ICT complesse. Per raggiungere questo obiettivo, il modello si basa sul principio della **modularità** e dell'**astrazione a livelli**: un problema di comunicazione complesso viene scomposto in sottoproblemi più semplici e gestibili. Ogni livello è progettato per essere indipendente; esso nasconde i propri dettagli implementativi (*information hiding*) e offre un insieme definito di servizi al livello superiore attraverso un'interfaccia standard. I livelli descritti dal modello OSI, dal più vicino all'utente al più vicino al mezzo fisico, sono:

1. **Applicazione:** Interfaccia diretta per i processi applicativi (es. HTTP, FTP, DNS).
2. **Presentazione:** Gestisce la sintassi e la semantica dei dati (traduzione, cifratura e compressione).
3. **Sessione:** Organizza e sincronizza il dialogo tra applicazioni remote (gestione dei turni e checkpoint).
4. **Trasporto:** Fornisce il trasferimento dei dati end-to-end, garantendo (ove richiesto) affidabilità e controllo di flusso.
5. **Network:** Responsabile dell'instradamento (*routing*) dei pacchetti attraverso nodi intermedi.
6. **Data Link:** Gestisce il trasferimento affidabile di frame su un collegamento fisico tra due nodi adiacenti e il controllo dell'accesso al mezzo (MAC).
7. **Fisico:** Si occupa della trasmissione dei singoli bit sul canale di comunicazione (tensioni, frequenze, connettori).

I livelli sono generalmente prescrittivi e non è quindi possibile bypassarli.

3.3.1 Protocollo

Ogni livello della gerarchia mira a comunicare esclusivamente con il proprio *peer* (entità paritaria) situato su un sistema remoto. Questa comunicazione logica è regolata da un **protocollo**: un insieme di regole, sintassi e procedure che definiscono lo scambio di dati.

Nella pratica, il protocollo di livello N realizza le proprie funzioni non attraverso un collegamento fisico diretto con l'altro sistema, ma utilizzando il **servizio** messo a disposizione dal livello immediatamente sottostante ($N - 1$).

3.3.2 Interfaccia

La definizione astratta dei servizi offerti dal livello corrente al livello superiore è detta **interfaccia**. Essa stabilisce i punti di accesso (SAP - *Service Access Points*) e le primitive attraverso cui il livello $N + 1$ può invocare le funzionalità del livello N , garantendo il totale mascheramento dei dettagli implementativi interni al livello sottostante.

3.3.3 Visione verticale ed orizzontale

In questa architettura si distinguono due direttrici fondamentali:

- **Visione Orizzontale:** riguarda il protocollo, ovvero la specifica di implementazione dello scambio di dati e informazioni tra due entità paritarie su sistemi distanti.

- **Visione Verticale:** riguarda l'interazione locale tra livelli adiacenti, dove ogni livello utilizza il servizio sottostante per realizzare il proprio protocollo e, a sua volta, offrire un servizio più evoluto al livello superiore.

Ogni protocollo è dunque lo strumento necessario per trasformare il servizio offerto dal livello $N - 1$ nel servizio richiesto dal livello $N + 1$.

3.3.4 Azioni di comunicazione

Ogni azione di invio informazioni coinvolge un mittente, un destinatario e una serie di eventuali nodi intermedi. Il flusso di comunicazione segue un percorso specifico attraverso la pila OSI:

- **Mittente:** I dati vengono generati al livello *Applicazione* e scendono progressivamente fino al livello *Fisico*. Durante questa discesa, ogni livello aggiunge le proprie informazioni di controllo (*header* e talvolta *trailer*) attraverso il processo di **incapsulamento**.
- **Nodi Intermedi:** La comunicazione attraversa i nodi della rete risalendo la pila esclusivamente fino al livello di **Rete (Network)**. Qui il nodo analizza l'indirizzamento logico per determinare il percorso successivo (*routing*) e poi re-incapsula il pacchetto verso il livello fisico per l'inoltro.
- **Destinatario:** Una volta raggiunto il sistema finale, il messaggio risale l'intera pila OSI, dal livello *Fisico* fino a quello *Applicazione*. In questa fase avviene il **decapsulamento**, in cui ogni livello rimuove e interpreta l'header di propria competenza per consegnare infine il dato puro al processo utente.

3.4 Connessioni e QoS

Il modello OSI classico è intrinsecamente orientato alla connessione e al supporto della **Qualità del Servizio (QoS)**. In questo paradigma:

- **Instradamento Statico:** Tutti i messaggi appartenenti a una sessione seguono il medesimo percorso tra mittente e destinatario, stabilito durante la fase di setup della connessione.
- **Prenotazione delle Risorse:** Il percorso impegna risorse dedicate (buffer, banda, CPU) sia negli endpoint che nei nodi intermedi, garantendo parametri di performance costanti.

Esiste tuttavia un approccio alternativo, adottato dal modello Internet (TCP/IP), che sposta l'intelligenza della rete verso gli **endpoint**:

- **Instradamento Dinamico (Datagram):** I singoli messaggi (pacchetti) possono seguire strade diverse a seconda dello stato della rete e delle decisioni dei router intermedi.
- **Modello Best-effort:** I nodi intermedi non mantengono informazioni sullo stato della connessione né riservano risorse fisse. L'affidabilità e il riordinamento dei dati vengono gestiti esclusivamente dagli host finali tramite protocolli come il TCP.

3.4.1 Connectionless

Nel modello **Connectionless**, ogni unità di dati è trasferita in modo indipendente dalle altre, essendo autocontenuta (contiene tutte le informazioni di indirizzamento necessarie).

- **Caratteristiche:** Non è garantito l'ordine di consegna e non è prevista una fase di negoziazione preventiva tra le entità.
- **Vantaggi/Svantaggi:** Presenta costi di gestione limitati (basso overhead) ma offre scarse garanzie di consegna (assenza di QoS nativa).

3.4.2 Connection-oriented

Il modello **Connection-oriented** prevede che si stabilisca una connessione logica tra entità paritarie prima dello scambio dei dati. Tale connessione segue un ciclo di vita rigoroso:

1. **Apertura (Setup):** Fase di negoziazione dei parametri e allocazione delle risorse.
2. **Trasferimento dati:** Scambio di informazioni con garanzia di sequenza.
3. **Chiusura (Release):** Rilascio delle risorse impegnate.

Questo approccio comporta costi elevati in termini di latenza di setup e overhead di gestione, ma garantisce una comunicazione bidirezionale coordinata e affidabile.

3.4.3 Quality of Service

OSI definisce la **QoS** come l'insieme delle caratteristiche di un servizio che influiscono sulla sua capacità di soddisfare esigenze espresse o implicite. Ogni servizio connection-oriented deve poter negoziare e mantenere determinati parametri, tra cui:

- **Affidabilità:** Probabilità che i dati arrivino integri.
- **Throughput:** Quantità di dati trasferiti nell'unità di tempo.
- **Latenza (Delay):** Tempo di transito tra mittente e destinatario.
- **Jitter:** Variazione del ritardo temporale tra i pacchetti.
- **Garanzia di sequenza:** Consegna dei dati nello stesso ordine in cui sono stati inviati.

3.5 Entità e Indirizzamento

Per permettere la comunicazione tra un'applicazione sull'Host A e una sull'Host B, l'applicazione mittente deve essere in grado di identificare univocamente il proprio *peer* remoto attraverso tutta la gerarchia della pila protocollare.

Il processo di identificazione segue una struttura gerarchica:

- **Identificazione Orizzontale:** L'applicazione A deve identificare il suo corrispondente paritario (livello 7) su B. Tuttavia, in un sistema stratificato, l'accesso a un'entità di livello N avviene esclusivamente attraverso il punto di accesso al servizio (**SAP - Service Access Point**) del livello $N - 1$.
- **Addressing Multilivello:** Se un livello presenta più entità attive (ad esempio, più processi applicativi o più protocolli di trasporto), è necessario specificare un identificatore preciso per ogni passaggio della pila.
- **Strategie di Selezione:** Per giungere all'applicazione B, il flusso deve essere incanalato attraverso entità specifiche in ogni livello (es. un particolare indirizzo IP a livello Rete e un numero di porta specifico a livello Trasporto).

In sintesi, la comunicazione richiede che ogni livello conosca i "nomi" (indirizzi) delle entità paritarie del sistema remoto, garantendo che i dati risalcano la pila verso l'entità corretta tramite la corretta sequenza di SAP.

3.5.1 Service Access Point

Un **Service Access Point (SAP)** definisce il punto di accesso logico attraverso cui un servizio di livello N viene offerto al livello superiore ($N + 1$). Esso può essere inteso come:

- **Interfaccia logica:** Il punto di connessione tra due livelli adiacenti.
- **API:** L'insieme di funzioni e primitive che permettono l'accesso al servizio.
- **N-SAP address:** L'indirizzo specifico che identifica un'entità all'interno del livello N .

Per garantire una comunicazione non ambigua, ogni nome deve essere unico. Teoricamente, per identificare un'entità remota in modo assoluto, sarebbe necessario nominare tutti i SAP di ogni livello, dal livello fisico (livello 1) fino al livello dell'entità di interesse.

Nella pratica, per semplicità e scalabilità, si adotta un approccio gerarchico che parte dal livello di rete (livello 3):

- **Indirizzo di Rete:** Identifica il nodo (host) in modo univoco all'interno dell'interrete (es. indirizzo IP).
- **Identificazione a stack:** Un'entità (un *peer*) viene identificata univocamente combinando il SAP di rete con la sequenza degli identificatori dei livelli superiori.

Questa struttura a stack (che comprende, ad esempio: indirizzo IP, numeri di porta, socket e ID di processo) permette di specificare in modo non ambiguo e abbreviato qualsiasi entità applicativa remota, evitando di dover gestire i dettagli dei livelli inferiori alla rete (come l'indirizzo MAC) una volta che il pacchetto è stato correttamente instradato.

È importante sottolineare come nello standard **non** si faccia mai riferimento a singole implementazioni hardware o software (processi, procedure, registri, ecc.), ma esclusivamente a **concetti astratti** (entità, flussi di informazioni, attività), garantendo così la massima interoperabilità tra sistemi eterogenei.

3.6 Comunicazione

Le comunicazioni nel modello OSI sono sempre avviate su iniziativa del **mittente**. Il flusso informativo è di tipo verticale e discendente: ogni livello N riceve i dati dal livello superiore, vi integra le proprie esigenze specifiche (funzioni di controllo, indirizzamento, rilevamento errori) e passa il risultato al **SAP** del livello sottostante ($N - 1$).

Poiché un protocollo è progettato per avere un ciclo di vita molto lungo, lo standard si limita a definire i **meccanismi** di comunicazione (il "come" si comunica), lasciando le **politiche** di utilizzo (il "cosa" o "quando" comunicare) non standardizzate e soggette a decisioni locali. Ogni livello viene quindi descritto esclusivamente come un puro meccanismo, ovvero come uno standard di messaggi scambiati tra entità paritarie.

Questo processo si basa su due componenti fondamentali che interagiscono attraverso l'interfaccia:

- **SDU (Service Data Unit):** Rappresenta l'unità di dati "pura" ricevuta dal livello superiore ($N + 1$). La SDU è l'oggetto del servizio: ciò che il livello N deve trasportare per conto del livello sovrastante.
- **PCI (Protocol Control Information):** È l'insieme di informazioni di controllo, comunemente chiamato **header** (ed eventualmente *trailer*), aggiunto dal protocollo del livello corrente per gestire la comunicazione con il *peer* remoto.

L'unione di PCI e SDU forma la **PDU (Protocol Data Unit)**. Questa è l'unità di dati completa scambiata logicamente tra i due livelli N remoti. Attraverso questo meccanismo di **incapsulamento**, ogni livello aggiunge uno "strato" di informazioni che verrà interpretato e rimosso (*decapsulamento*) esclusivamente dal livello paritario corrispondente sul nodo ricevente.

3.7 Primitive

Le entità di livelli adiacenti e le entità paritarie cooperano tramite le **primitive di servizio**, chiamate astratte che permettono di implementare le funzionalità del livello di appartenenza. Le primitive base, che definiscono l'azione da compiere, sono generalmente:

- **Connect:** Per stabilire una connessione tra entità paritarie.
- **Data:** Per il trasferimento effettivo delle SDU.
- **Disconnect:** Per il rilascio di una connessione esistente.

Per ogni azione (es. una *Connect*), il modello OSI definisce quattro forme (o eventi) che descrivono il flusso della comunicazione tra il fornitore del servizio (livello N) e l'utente del servizio (livello $N + 1$):

- **Request (Richiesta):** L'utente del servizio invoca il servizio per richiedere l'inizio di un'azione.
- **Indication (Indicazione):** Il fornitore del servizio notifica all'utente remoto che è stata richiesta un'azione.
- **Response (Risposta):** L'utente remoto risponde alla notifica ricevuta (usata nei servizi confermati).
- **Confirm (Conferma):** Il fornitore del servizio notifica all'utente originario che l'azione è stata completata con successo.

Le primitive possono essere classificate in base alla sincronizzazione:

- **Sincrone:** Il mittente attende il completamento dell'operazione (spesso associato a servizi confermati).
- **Asincrone:** Il mittente prosegue l'esecuzione dopo l'invio della richiesta, senza attendere l'esito immediato.

3.7.1 Esempio di Interazione

La sequenza di primitive descrive l'interazione temporale tra due sistemi (Mittente e Ricevente) attraverso i vari livelli. Un tipico scenario di instaurazione connessione e trasferimento dati si articola come segue:

1. Fase di Setup (Servizio Confermato):

- **CONNECT.request:** Il mittente richiede l'apertura di una connessione al livello inferiore.
- **CONNECT.indication:** Il livello inferiore del ricevente notifica l'entità paritaria della richiesta in arrivo.
- **CONNECT.response:** L'entità ricevente accetta la connessione.
- **CONNECT.confirm:** Il mittente riceve la conferma definitiva; la connessione è ora stabilita.

2. Fase di Trasferimento (Servizio Non Confermato):

- **DATA.request:** Il mittente invia un'unità di dati.
- **DATA.indication:** Il ricevente riceve i dati. In questo caso non segue una **response** immediata a livello di primitiva di servizio, a meno che il protocollo non preveda un riscontro (ACK) esplicito ad alto livello.

Questo meccanismo garantisce che il livello superiore sia sempre informato sullo stato della comunicazione, mantenendo l'astrazione necessaria tra i livelli adiacenti.

3.8 Differenze con TCP/IP

A differenza del modello OSI, che rimane un quadro di riferimento teorico e astratto, il modello **TCP/IP** adotta un approccio pragmatico orientato all'implementazione. In questo standard:

- **Meccanismi e Politiche:** Oltre alla definizione dei meccanismi di comunicazione, TCP/IP include specifiche sull'uso pratico e sull'implementazione dei protocolli, accettate dalla comunità degli utenti attraverso le **RFC** (*Request for Comments*).
- **Evoluzione dal basso:** Mentre OSI è stato progettato "a tavolino" da comitati internazionali, TCP/IP è nato dall'esperienza operativa e si è evoluto attraverso realizzazioni software concrete che sono diventate standard *de facto*.
- **Accorpamento dei livelli:** TCP/IP semplifica la struttura OSI riducendo i sette livelli a quattro o cinque (secondo la letteratura), accorpando in particolare i livelli superiori (Sessione e Presentazione) direttamente nel livello *Applicativo*.

Le RFC rappresentano quindi non solo lo standard tecnico, ma anche la documentazione di un'implementazione specifica che garantisce l'interoperabilità reale sulla rete globale.

3.9 Livelli bassi

I livelli al di sotto del trasporto (Fisico, Data Link e Rete) sono definiti **livelli di rete** o *network-oriented*. Essi forniscono un meccanismo di trasferimento trasparente per il livello Trasporto, occupandosi della movimentazione effettiva dei dati attraverso l'infrastruttura. Le loro funzioni principali includono:

- **Controllo degli errori:** Rilevamento e correzione dei bit errati durante la trasmissione.
- **Controllo del flusso di dati:** Gestione della velocità di invio per non sovraccaricare il ricevente.
- **Modelli di indirizzamento:** Identificazione univoca dei nodi nella rete.
- **Strategie di routing:** Selezione del percorso ottimale tra mittente e destinatario.

3.9.1 Network (Livello 3)

Il livello di rete tiene conto della presenza di eventuali **nodi intermedi** in una comunicazione e gestisce le diverse realizzazioni di instradamento (*routing*) tra reti eterogenee, oltre a definire il sistema di nominazione delle entità.

L'obiettivo fondamentale è garantire il passaggio delle informazioni interferendo il meno possibile con il comportamento locale dei sistemi. In particolare, il livello si occupa di:

- **Indirizzamento logico:** Fornire un sistema di nomi universale (es. indirizzo IP) indipendente dall'hardware.
- **Controllo di flusso tra le parti:** Gestione del transito dei dati tra nodi adiacenti.
- **Controllo di congestione:** Monitoraggio e gestione del carico complessivo del sistema per evitare colli di bottiglia nei nodi intermedi.

L'applicazione del **principio di separazione** permette di isolare i livelli superiori dalla complessità tecnologica dei nodi intermedi, migliorando l'efficienza complessiva e la scalabilità dell'intera architettura.

3.9.2 Trasporto

Il livello di Trasporto è il primo livello **end-to-end**: esso non si occupa più del transito tra nodi adiacenti, ma considera esclusivamente la struttura e la comunicazione tra i due nodi finali partecipanti.

Ogni livello dispone di proprie entità (spesso multiple). Una caratteristica fondamentale del Trasporto è la gestione della cardinalità tra SAP di diversi livelli: un nodo può avere molteplici **T-SAP** (identificatori di trasporto per diversi processi applicativi) mappati su un'unica (o poche) **N-SAP** di rete. Questo principio di astrazione si applica in modo ricorsivo a tutti i livelli superiori.

Durante la fase di instaurazione, la primitiva **CONNECT** coinvolge le entità di tutti i nodi interessati nel percorso, negoziando la qualità del servizio e mantenendo le risorse impegnate (buffer, tabelle di stato) per l'intera durata della sessione.

Funzioni principali Il livello di Trasporto svolge compiti critici di adattamento e ottimizzazione:

- **Segmentazione e Riasssemblaggio:** Il Trasporto può spezzare dati di grandi dimensioni provenienti dal livello superiore in unità più piccole (segmenti) e ricostruirli nel corretto ordine a destinazione.
- **Multiplexing:** Più connessioni di trasporto possono condividere la stessa connessione di rete per ottimizzare i costi (multiplexing verso il basso).
- **Splitting:** Una singola connessione di trasporto può essere suddivisa su più connessioni di rete per aumentare il throughput (multiplexing verso l'alto).
- **Gestione SAP:** L'unione e la decomposizione dei SAP rispetto a quelli di rete permette di indirizzare correttamente i flussi ai vari processi applicativi.

Connessione L'obiettivo primario del livello di Trasporto è garantire il trasferimento dei dati tra i due endpoint con **correttezza**, tempi di risposta accettabili e il rispetto della qualità del servizio (QoS) richiesta dal livello superiore. Esso agisce come un "filtro di affidabilità" sopra i servizi spesso imprevedibili della rete. In una modalità *connection-oriented*, il livello gestisce l'intero ciclo di vita del canale logico:

- **Apertura e Chiusura:** Procedure di *handshaking* per sincronizzare le entità e rilasciare le risorse.
- **Trasferimento Dati Ordinario:** Flusso di dati standard soggetto ai normali meccanismi di controllo di flusso e sequenziamento.
- **Dati Privilegiati (*Expedited*):** Messaggi urgenti che bypassano le code dei dati ordinari. Questi dati sono soggetti a meccanismi di controllo di flusso indipendenti e prioritari, garantendo che le informazioni di controllo o di emergenza raggiungano la destinazione anche in caso di congestione del flusso principale.

Primitive Le primitive T-* (Transport) permettono l'interazione tra il livello Trasporto e il livello Sessione (o l'Applicazione nel modello TCP/IP):

- **T-CONNECT:** Utilizzata per negoziare e stabilire la connessione end-to-end.
- **T-DATA:** Utilizzata per il trasferimento trasparente delle SDU ordinarie.
- **T-EXPEDITED-DATA:** Utilizzata per l'invio prioritario di piccole quantità di dati (fuori banda).
- **T-DISCONNECT:** Utilizzata per la chiusura della connessione, che può essere di tipo *abrupt* (perdita dei dati in transito) o coordinata.

3.10 Livelli alti

3.10.1 Sessione

Il livello di Sessione ha il compito di organizzare e strutturare il dialogo tra entità applicative, supportando diverse dimensioni e qualità della comunicazione. Esso determina i meccanismi per l'interazione tra entità remote, gestendo anche scenari di comunicazione eterogenea. Il dialogo può essere:

- **Bidirezionale:** Simultaneo (Full-Duplex) o alternato (Half-Duplex).
- **Strutturato:** Organizzato in entità separate e attività logiche.
- **Gestito:** Con monitoraggio delle risorse impegnate e garanzie di affidabilità e correttezza del flusso.

Sincronia e Gestione dello Stato A differenza dei livelli inferiori, la Sessione permette il salvataggio di uno "stato" della comunicazione. In caso di errore di rete, questo consente di risincronizzarsi e riprendere la trasmissione dagli ultimi dati validi senza ricominciare da capo. Tale meccanismo si basa sui **punti di sincronizzazione**:

- **Punti maggiori (Major Sync Points):** Suddividono il dialogo in unità logiche complete. Il mittente attende in modo bloccante la conferma (*Confirm*) prima di procedere. Una volta confermato, un punto maggiore non può essere annullato.
- **Punti minori (Minor Sync Points):** Utilizzati per marcare il flusso all'interno di un'unità logica. Non richiedono necessariamente una conferma immediata; la conferma di un punto successivo convalida implicitamente tutti i punti minori precedenti.

Token Il livello di Sessione utilizza i **Token** per gestire i diritti di esecuzione e prevenire conflitti durante il dialogo. Solo l'entità che possiede il token specifico in un determinato istante ha il diritto di compiere l'azione associata. I token principali sono:

- **Data Token:** Regola il diritto di invio dati in modalità Half-Duplex.
- **Release Token:** Necessario per iniziare la chiusura della sessione.
- **Sync-minor/major Token:** Necessari per inserire i rispettivi punti di sincronizzazione.

L'intersezione dei requisiti negoziati durante la primitiva **S-CONNECT** determina le caratteristiche finali della S-connezione.

Unità Funzionali e Primitive Il servizio è strutturato in **unità funzionali**, ovvero gruppi logici di primitive (58 in totale) e parametri. Le principali sono:

- **Kernel:** Funzioni base (apertura, chiusura, trasferimento dati semplice).
- **Half-Duplex:** Gestione del dialogo alternato tramite token.
- **Typed Data:** Trasferimento di dati non soggetti al controllo del token.
- **Basic Sync:** Supporto ai punti di sincronizzazione.
- **Basic Activity:** Permette di suddividere il dialogo in "Attività" indipendenti che possono essere sospese e riprese.

Gestione del Recovery In caso di guasto, il livello di Sessione utilizza i punti di sincronizzazione per il *recovery* automatico o negoziato. Le strategie previste includono l'abbandono della sessione, il ripristino (*Resynchronize*) a uno stato precedente o l'intervento diretto dell'utente applicativo.

3.10.2 Presentazione

Il livello di Presentazione garantisce che le informazioni inviate dal livello Applicativo di un sistema siano leggibili dal livello Applicativo di un altro sistema. Esso funge da traduttore universale, normando strumenti e protocolli per la gestione di dati in sistemi aperti.

Oltre all'interoperabilità, il livello si occupa di servizi a valore aggiunto quali:

- **Compressione:** Ottimizzazione del volume di dati da trasmettere.
- **Crittografia:** Garanzia di riservatezza dei dati durante il transito.

Eterogeneità e Formati Comuni Per risolvere le disomogeneità tra ambienti di programmazione, linguaggi, sistemi operativi e architetture (es. *Big-Endian* vs *Little-Endian*), il modello OSI scarta l'approccio delle $N \times (N - 1)$ conversioni dirette a favore di un **formato di rappresentazione comune** (canale standard intermedio). Ogni nodo necessita quindi solo di due trasformazioni:

- Da formato locale a formato standard (mittente).
- Da formato standard a formato locale (ricevente).

Sintassi Astratta e Concreta In mancanza di un accordo preventivo, il livello deve negoziare sia il "cosa" che il "come" comunicare:

- **Sintassi Astratta:** Definizione formale dei dati indipendentemente dalla loro rappresentazione fisica (es. definire che si sta trasmettendo un "Intero" o un "Record").
- **Sintassi di Trasferimento (Concreta):** La codifica effettiva dei bit sul canale (es. quanti bit usare, l'ordine dei byte).

Efficienza e Descrittori (TLV) La rappresentazione dei valori può variare in base al costo di comunicazione e alla robustezza richiesta. Si utilizza spesso lo schema **TLV (Type, Length, Value)**:

- **Tipo:** Identifica la natura del dato (es. stringa).
- **Lunghezza:** Indica quanti byte occupa il valore.
- **Valore:** Il contenuto informativo effettivo.

Negoziazione e Bidding In un sistema aperto, la scelta della sintassi di trasferimento non è sempre predeterminata. Il processo di **Bidding** (derivato dal *Contract Net Protocol*) permette di stabilire un accordo dinamico tra le entità:

1. **Announcement:** Il mittente (Manager) pubblica i requisiti del dato da trasmettere (es. tipo di dato, necessità di cifratura).
2. **Bidding:** I potenziali riceventi (Contractors) rispondono con le loro capacità di decodifica e i costi associati.
3. **Awarding:** Il mittente seleziona l'offerta più idonea e invia una conferma.
4. **Contract:** Viene stabilito il contesto di presentazione comune per la durata della sessione.

Standard di Riferimento: ASN.1 e BER Il livello di presentazione formalizza questi concetti attraverso due standard principali:

- **ASN.1 (Abstract Syntax Notation One):** Un linguaggio di specifica per definire i tipi di dati in modo astratto, indipendente dall'hardware.

```
UserAge ::= INTEGER
currentAge UserAge ::= 26
```

- **BER (Basic Encoding Rules):** L'insieme di regole che specifica come i tipi definiti in ASN.1 devono essere codificati in una sequenza di ottetti per la trasmissione (sintassi di trasferimento).

Campo	Esadecimale	Descrizione
T (Type)	02	Identifica univocamente il tipo <i>INTEGER</i>
L (Length)	01	Indica che il valore è contenuto in 1 ottetto
V (Value)	1A	Rappresentazione binaria del valore 26 ($1A_{16} = 26_{10}$)

Tabella 1: Codifica BER dell'intero 26

3.10.3 Applicazione

Il livello di Applicazione è responsabile dell'interfacciamento diretto con l'utente o con i processi software. L'obiettivo ultimo è l'astrazione massima per nascondere la complessità dei livelli sottostanti, coordinando le applicazioni distribuite attraverso servizi standardizzati e indipendenti dal sistema:

- **MHS (Message Handling System):** Servizi di posta elettronica (X.400).
- **X.500:** Servizi di directorio e gestione dei nomi.
- **X.700 / CMISE & CMIP:** Gestione della rete e monitoraggio.
- **FTAM (File Transfer Access and Management):** Accesso e gestione file remoti.
- **VT (Virtual Terminal):** Emulazione di terminale remoto.
- **DTP (Distributed Transaction Processing):** Gestione di transazioni distribuite.

Approccio Object-Oriented A differenza di altri modelli, OSI adotta un approccio rigorosamente orientato agli oggetti per modellare le risorse di rete:

- **Template e Package:** Definizione di strutture dati e funzionalità riutilizzabili.
- **Ereditarietà statica:** Possibilità di derivare nuovi oggetti da classi base standardizzate.
- **Manipolazione:** Gli oggetti sono gestiti tramite pacchetti di operazioni ben definite, garantendo l'integrità del sistema.

Identificazione e Gerarchie (OID) Per identificare univocamente ogni entità o protocollo, si utilizza una gerarchia globale basata su **Object Identifiers (OID)**. Un OID è una sequenza di numeri che rappresenta un percorso in un albero decisionale unico:

Esempio: 1.3.6.1.2.1.4 identifica il sottoalbero relativo al protocollo IP per scopi di gestione (MIB-II).

Le proprietà non coperte dal directorio X.500 vengono negoziate dinamicamente tramite ASN.1.

Sotto-livelli di Applicazione Il livello 7 è internamente strutturato in elementi di servizio (**ASE - Application Service Elements**):

- **ACSE (Association Control)**: Fondamentale per stabilire e rilasciare le associazioni tra applicazioni.
- **RTSE (Reliable Transfer)**: Garantisce il trasferimento affidabile di unità dati applicative.
- **ROSE (Remote Operations)**: Supporta l'invocazione di operazioni remote (stile RPC).
- **CCR (Commitment, Concurrency, and Recovery)**: Gestisce l'integrità nelle transazioni distribuite.

3.11 OSI vs Internet

Il modello OSI si distingue per un approccio **Object-Oriented** puro, con una separazione netta tra interfacce ed implementazioni, offrendo un progetto completo e standardizzato per la Qualità del Servizio (QoS) e le connessioni.

Al contrario, il modello **TCP/IP** (Internet) presenta alcuni limiti strutturali:

- **Documentazione**: Gli standard RFC spesso offrono descrizioni meno rigorose rispetto alle specifiche ISO.
- **Accoppiamento**: Protocollo e implementazione sono spesso fusi insieme, rendendo difficile l'astrazione.
- **Pragmatismo**: Il progetto è cresciuto in modo incrementale per soddisfare esigenze pratiche immediate, prediligendo la diffusione del prodotto rispetto alla perfezione architettonica del modello.

4 Modello TCP/IP

A differenza del modello OSI, gli standard della suite TCP/IP sono nati dalla necessità di una realizzazione immediata e funzionale. L'obiettivo primario di Internet è la creazione di un **sistema globale** capace di connettere reti eterogenee. Oggi, Internet si configura come un'interrete (*internetwork*) di reti private e pubbliche che adottano i protocolli standard definiti dall' **IETF** (Internet Engineering Task Force).

Il successo di questo modello risiede nell'adozione di protocolli a basso costo e ridotto overhead, ottimizzati per la scalabilità su scala mondiale.

4.1 L'architettura a livelli

Sebbene il modello concettuale preveda più strati, l'intelligenza del sistema si concentra in due livelli fondamentali che definiscono il comportamento della rete:

- **Livello di Trasporto (Host-to-Host):** Gestisce la comunicazione tra le applicazioni sugli endpoint.
 - **TCP (Transmission Control Protocol):** Fornisce un servizio affidabile, orientato alla connessione, con controllo di flusso e congestione.
 - **UDP (User Datagram Protocol):** Un servizio *connectionless* ridotto all'osso, ideale per applicazioni che prediligono la bassa latenza (es. streaming, DNS).
- **Livello di Internet (Network):** Responsabile dell'instradamento dei pacchetti attraverso la rete.
 - **IP (Internet Protocol):** Il protocollo di indirizzamento universale.
 - **ICMP (Internet Control Message Protocol):** Utilizzato per scambiare messaggi di controllo e diagnostica (es. *ping*, *traceroute*).

4.2 Address Resolution Protocol (ARP)

I protocolli devono rispettare la visione verticale della pila ISO/OSI. Per l'invio effettivo di pacchetti su un segmento fisico, è necessario risolvere l'indirizzo logico del destinatario in un indirizzo del livello *Data Link*. Sebbene esista una corrispondenza tra i due, non è possibile estrarre deterministicamente l'indirizzo fisico (MAC) dall'indirizzo logico (IP), poiché ciò violerebbe il principio di indipendenza dei livelli.

A tal fine nascono i protocolli **ARP** (Address Resolution Protocol) e **RARP** (Reverse ARP), protocolli operanti localmente che si basano sul meccanismo del **broadcast**:

- **Richiesta:** Il mittente invia un pacchetto in broadcast a tutti gli host della sottorete chiedendo: "Chi ha l'indirizzo IP X.X.X.X? Comunica il tuo indirizzo fisico a Y.Y.Y.Y".
- **Risposta:** Tutti gli host ricevono il pacchetto, ma solo il legittimo proprietario dell'indirizzo IP richiesto risponde (in unicast) comunicando il proprio indirizzo fisico.

4.2.1 ARP Cache

Per ottimizzare i costi di esercizio ed evitare di saturare la rete con messaggi broadcast, ogni host gestisce una **ARP Cache** (una tabella locale di associazione IP-MAC):

- Ogni volta che viene ricevuta una risposta, l'associazione viene salvata e consultata per le comunicazioni successive.
- **Apprendimento Passivo:** Durante una richiesta broadcast, non solo il destinatario, ma anche gli altri host della rete possono aggiornare le proprie tabelle leggendo l'associazione del mittente contenuta nel pacchetto.
- **Gratuitous ARP:** Quando una macchina si connette alla rete, può inviare un pacchetto broadcast spontaneo con la propria associazione per aggiornare preventivamente le cache degli altri host ed evitare conflitti di indirizzi.

4.2.2 Ruoli

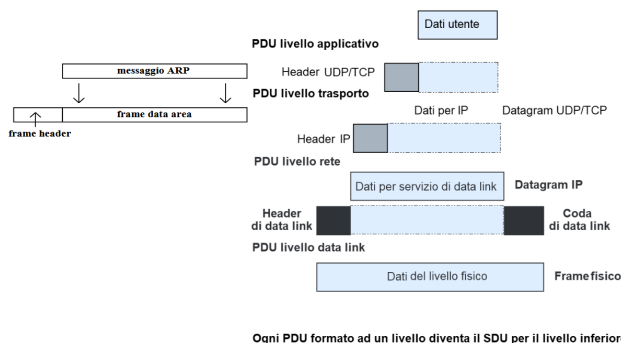
Il protocollo ARP definisce due comportamenti distinti:

- **Ruolo Attivo (Richiedente):** Esamina la cache locale prima di generare traffico. In caso di *cache miss*, avvia il broadcast e deve gestire i *timeout* qualora non ricevesse risposta (host spento o indirizzo inesistente).
- **Ruolo Passivo (Rispondente):** Monitora le richieste broadcast. Se il proprio IP corrisponde a quello cercato, genera una risposta unicast. Indipendentemente dalla corrispondenza, analizza l'intestazione ARP per aggiornare la propria cache con i dati del mittente.

4.2.3 Realizzazione e Incapsulamento

Il pacchetto ARP non utilizza un'intestazione IP, ma viene incapsulato direttamente all'interno di un frame Ethernet. La struttura completa del messaggio, partendo dai campi del livello Data Link, è la seguente:

- **Campi Ethernet:**
 - Preambolo e Delimitatore di inizio frame (SFD).
 - **Destinazione:** Indirizzo MAC (fisico). Per le richieste è `FF:FF:FF:FF:FF:FF` (Broadcast).
 - **Sorgente:** Indirizzo MAC del mittente.
 - **Tipo (EtherType):** Impostato a `0x0806` per identificare il protocollo ARP.
- **Header ARP (Payload del Frame):**
 - **Hardware Type:** Tipo di rete fisica (es. Ethernet = 1).
 - **Protocol Type:** Protocollo di rete per cui si chiede l'indirizzo (es. IPv4 = `0x0800`).
 - **Hardware/Protocol Length:** Lunghezza in byte degli indirizzi (MAC = 6, IP = 4).
 - **Operation:** Identifica il tipo di messaggio (1 per *Request*, 2 per *Reply*).
 - **Sender Hardware Address:** MAC del mittente.
 - **Sender IP Address:** IP del mittente.
 - **Target Hardware Address:** MAC del destinatario (nullo nelle richieste).
 - **Target IP Address:** IP del destinatario cercato.
- **FCS (Frame Check Sequence):** Controllo di errore CRC.



4.2.4 RARP (Reverse ARP)

Il Protocollo RARP (oggi deprecato) implementa una ricerca inversa per quei nodi che conoscono il proprio indirizzo MAC ma non il proprio indirizzo IP. Questa situazione è tipica delle macchine *diskless* che devono richiedere la propria identità logica a un server di rete all'avvio.

RARP opera direttamente sulla rete fisica e gestisce la logica di ritrasmissione. Per garantire la *fault tolerance*, sono solitamente previsti più server RARP per ogni LAN.

Gestione dei Server RARP Il modello multi-server richiede strategie per evitare che le risposte simultanee sovraccarichino la rete:

- **Modello Statico:** Ogni server possiede una tabella fissa e risponde solo a un sottoinsieme specifico di client. È un approccio rigido ma previene le interferenze.
- **Modello Dinamico (Attivo/Passivo):** I server sono organizzati gerarchicamente o utilizzano tempi di attesa differenziati. In caso di mancata risposta del server primario (*timeout*), i server secondari intervengono per soddisfare la richiesta.

4.3 Dynamic Host Configuration Protocol (DHCP)

Il **DHCP** è il protocollo standard per l'attribuzione dinamica degli indirizzi IP e dei parametri di configurazione. Rispetto all'indirizzamento statico, il DHCP permette un utilizzo efficiente dello spazio di indirizzamento, assegnando gli IP solo agli host effettivamente attivi (risparmio basato sulla contemporaneità d'uso).

Oltre all'indirizzo IP, il protocollo consente di distribuire automaticamente parametri critici quali:

- Maschera di sottorete (*Subnet Mask*).
- Gateway predefinito (*Default Gateway*).
- Indirizzi dei server DNS.
- Parametri specifici per la mobilità e il wireless.

4.3.1 Soft-state e Lease Time

Il DHCP adotta un approccio **soft-state**: l'assegnazione dell'indirizzo non è permanente ma ha una durata limitata nel tempo, definita **Lease Time**.

- Una volta scaduto il lease, il server può riassegnare l'indirizzo ad altri host.
- Il client può richiedere il rinnovo del lease prima della scadenza per mantenere lo stesso indirizzo.
- Il client può inviare un messaggio di **Release** per rilasciare volontariamente l'indirizzo prima della scadenza.

4.3.2 Bidding a 4 fasi (DORA)

Il protocollo si basa su un'interazione client-server che segue un processo di negoziazione in quattro fasi, spesso identificato dall'acronimo **DORA**:

1. **DHCP Discovery:** Il client invia un pacchetto broadcast per individuare i server DHCP disponibili nella rete locale.
2. **DHCP Offer:** I server che ricevono la richiesta rispondono con un'offerta contenente un indirizzo IP disponibile e i relativi parametri.
3. **DHCP Request:** Il client seleziona un'offerta (solitamente la prima ricevuta) e invia una richiesta formale di utilizzo di quell'indirizzo, notificando implicitamente agli altri server che le loro offerte sono state rifiutate.
4. **DHCP Acknowledgment (ACK):** Il server conferma l'assegnazione e invia i parametri di configurazione finali.

4.3.3 Vantaggi Operativi

Il DHCP elimina la necessità di configurazione manuale (*setup*), riducendo drasticamente gli errori umani (come la duplicazione degli indirizzi IP) e facilitando la gestione di host mobili che si spostano tra reti diverse.

4.4 Network Address Translation (NAT)

Il protocollo **NAT** viene utilizzato per associare una serie di indirizzi IP privati (locali) a uno o più indirizzi IP globali (pubblici). Nato per superare il limite numerico degli indirizzi IPv4 disponibili, viene eseguito dai router NAT attraverso la gestione di tabelle di corrispondenza interne.

4.4.1 Meccanismi di Traslazione

Il NAT può essere implementato in diverse forme, a seconda della complessità della rete e del coordinamento con altri servizi (come il DNS):

- **NAT Statico:** Un indirizzo interno viene mappato univocamente in un indirizzo esterno. Richiede lo stesso numero di indirizzi IP su entrambi i lati e viene usato tipicamente per esporre server interni.
- **NAT Dinamico:** Gli indirizzi interni vengono mappati in un pool di indirizzi esterni validi. La corrispondenza è temporanea e gestita in base alla disponibilità nel pool.
- **Coordinamento dei Nomi:** Per situazioni articolate, non basta la traslazione dei router, ma è necessario istituire i sistemi DNS affinché mantengano la consistenza tra i nomi di dominio e gli indirizzi traslati.

4.4.2 NAT a Porte (PAT - Port Address Translation)

Per massimizzare l'efficienza e minimizzare l'uso di indirizzi pubblici, si utilizza una versione più espressiva chiamata **NAPT** (Network Address Port Translation) o **NAT a porte**. Questo modello permette a un intero parco macchine interno di condividere un **unico indirizzo IP esterno**.

- **Funzionamento:** Il router distingue i molteplici host interni mappando le loro richieste su **porte di comunicazione** diverse dell'unico indirizzo esterno.
- **Esempio:** Due host interni (192.168.1.2 e 192.168.1.3) inviano dati verso l'esterno. Il router li riscrive entrambi con l'IP pubblico 203.0.113.1, ma assegna all'host A la porta 10001 e all'host B la porta 10002.
- **Vantaggi:** Limita drasticamente l'impegno di risorse esterne (tipico dei provider internet) e riduce i costi di assegnazione degli indirizzi.

4.4.3 Trattamento dei Datagrammi

Il router NAT deve processare ogni datagramma in transito, operando una manipolazione dell'header IP (e dell'header di trasporto in caso di PAT) nei due sensi:

1. **Outbound:** Sostituzione dell'indirizzo sorgente privato con quello pubblico.
2. **Inbound:** Consultazione della tabella per riportare l'indirizzo destinazione pubblico a quello privato originale dell'host interno.

4.5 Internet Control Message Protocol (ICMP)

L'ICMP è un protocollo di servizio e controllo che opera parallelamente a IP per mitigarne la natura *best-effort*. Esso permette di inviare messaggi di controllo o di errore al nodo sorgente di un datagramma, coordinando le entità di livello IP con azioni di gestione e diagnostica.

4.5.1 Natura del Protocollo

I messaggi ICMP sono incapsulati direttamente all'interno di comuni datagrammi IP. Questa scelta architetturale implica che:

- **Routing standard:** Sono soggetti alle medesime regole di instradamento di qualsiasi altro pacchetto.
- **Assenza di priorità:** Non godono di un canale preferenziale e possono essere scartati in caso di congestione.
- **Nessuna ricorsione:** Per evitare loop infiniti di messaggi di controllo, non vengono mai generati messaggi ICMP per segnalare errori su altri messaggi ICMP.
- **Basso costo:** Sono considerati messaggi di "meta-livello" con un overhead minimo.

4.5.2 Formato dell'Header

L'header ICMP segue immediatamente l'header IP (quando il campo *Protocol* di IP è impostato a 1) ed è così strutturato:

- **TYPE (8 bit):** Definisce la categoria del messaggio (es. 0, 3, 4, 5, 8, 11, ecc.).
- **CODE (8 bit):** Fornisce un ulteriore dettaglio specifico per quel tipo di errore.
- **Checksum (16 bit):** Copre l'intero messaggio ICMP.
- **Data (Variabile):** Contiene parametri specifici del messaggio (es. ID e Sequence Number per il ping).
- **Payload di errore:** In caso di segnalazione di errore, ICMP include l'**header del datagramma errato** più i primi 64 bit del payload originale, permettendo al mittente di identificare quale processo ha causato il problema.

4.5.3 Livelli di Errore

ICMP classifica le anomalie in tre categorie principali:

- **Destination Unreachable (Type 3):** Il datagramma non può essere consegnato (rete, host, porta o protocollo irraggiungibile).
- **Source Quench (Type 4):** Richiesta di rallentamento dell'invio (controllo di congestione elementare, oggi ampiamente deprecato).
- **Time Exceeded (Type 11):** Segnala la perdita di datagrammi dovuta a cicli di instradamento (TTL scaduto) o problemi nel riassemblaggio dei frammenti.

4.5.4 Coordinamento e Gestione

Oltre agli errori, ICMP supporta funzioni proattive di rete:

- **Echo Request/Reply (Type 8/0):** Base del comando `ping` per verificare la raggiungibilità e calcolare il **RTT (Round Trip Time)**.
- **Address Mask (Type 17/18):** Permette la sincronizzazione della maschera di sottorete tra host e gateway.
- **Timestamp (Type 13/14):** Utilizzato per misurare il tempo di latenza fisica tra i nodi in millisecondi.
- **Redirect (Type 5):** Consente a un gateway di istruire un host su un percorso locale più efficiente, aggiornando dinamicamente la tabella di routing del mittente.

Diagnostica Avanzata: Traceroute Il comando `traceroute` (o `tracert`) ricostruisce il cammino verso un nodo inviando pacchetti con **TTL crescente**. Ogni router intermedio che riceve un pacchetto con $TTL = 0$ invia un messaggio *Time Exceeded*, permettendo al mittente di identificare ogni hop del percorso. *Nota:* Il risultato è affidabile solo se le tabelle di routing rimangono consistenti durante l'intera esecuzione.

4.6 User Datagram Protocol (UDP)

L'UDP è un protocollo di trasporto **processo-a-processo**, operante secondo una filosofia *best-effort* a basso costo computazionale. Esso utilizza il protocollo IP per la consegna dei datagrammi, estendendone le funzionalità attraverso l'identificazione dei processi applicativi mediante il concetto di **porta**.

L'unione di un indirizzo IP e di un numero di porta definisce una **Socket**, l'endpoint logico della comunicazione.

4.6.1 Caratteristiche del Servizio

UDP fornisce un servizio **connectionless** e **unreliable**. Poiché non implementa meccanismi di riscontro o sequenziamento, i datagrammi possono essere:

- Persi o duplicati.
- Ritardati o consegnati fuori ordine.

Questa mancanza di controllo lo rende ideale per applicazioni *real-time* (streaming, VoIP) o servizi rapidi di richiesta-risposta (DNS, DHCP).

Struttura del Segmento L'header UDP è estremamente ridotto (soli 8 byte), composto da:

- **Porta Sorgente e Destinazione:** Per l'instradamento al processo corretto.
- **Lunghezza:** Dimensione totale del segmento (header + dati).
- **Checksum:** Campo opzionale in IPv4 (obbligatorio in IPv6) per il controllo dell'integrità dei dati.

4.6.2 Porte e Multiplexing

Il livello di trasporto permette a più applicazioni di condividere l'unica connessione di rete attraverso:

- **Multiplexing:** Raccolta di dati da diverse porte per l'invio tramite un unico protocollo IP.
- **Demultiplexing:** Consegna del datagramma in arrivo alla porta corretta basandosi sull'header UDP.

Classificazione delle Porte Le porte sono identificate da numeri a 16 bit (0 – 65535) e sono suddivise in categorie:

- **Well-known Ports (0-1023):** Porte statiche assegnate dall'IANA per servizi standard (es. DNS porta 53, HTTP porta 80).
- **Registered Ports (1024-49151):** Assegnate a specifiche applicazioni su richiesta.
- **Dynamic/Ephemeral Ports (49152-65535):** Assegnate dinamicamente dal sistema operativo per le connessioni client temporanee.

TCP e UDP utilizzano questo approccio **ibrido** per garantire sia l'accessibilità dei servizi noti sia la flessibilità per le applicazioni utente.

4.7 Transmission Control Protocol (TCP)

Il TCP è un protocollo di trasporto orientato alla connessione che fornisce una comunicazione affidabile **end-to-end** tra processi in esecuzione su host distinti. Operando sopra IP, TCP nasconde alle applicazioni la complessità della rete offrendo un canale logico affidabile e ordinato.

4.7.1 Modello di Servizio

TCP realizza un servizio di trasporto caratterizzato dai seguenti principi fondamentali:

- **Connessione orientata:** prima dello scambio dati viene stabilita una connessione logica tra i due endpoint.
- **Reliable Full Duplex Stream:** la comunicazione è bidirezionale e simultanea; ogni byte viene consegnato senza perdite, duplicazioni o riordinamenti errati.
- **Byte Stream non strutturato:** TCP vede i dati come un flusso continuo di byte senza confini di messaggio. L'interpretazione semantica è responsabilità dell'applicazione.
- **End-to-End Principle:** lo stato della connessione è mantenuto solo dagli host terminali. I router intermedi inoltrano semplicemente datagrammi IP.

Tipologie di dati TCP distingue tra:

- **Dati normali**
- **Dati urgenti**, segnalati tramite flag URG

4.7.2 Identificazione della Connessione

Un endpoint di comunicazione è definito dalla coppia (*indirizzo IP, numero di porta*), chiamata **socket**. Una connessione TCP è identificata univocamente da una **quadrupla**:

$$\{host_{sorg}, port_{sorg}, host_{dest}, port_{dest}\} \quad (1)$$

Esiste una sola connessione per ogni specifica combinazione di questi quattro parametri. Questo permette il **multiplexing**: un server può gestire più client sulla stessa porta (es. 80) perché le quadruple risulteranno diverse (cambia l'IP o la porta del client).

Porte standard Le associazioni comuni sono censite in `/etc/services`:

- **20/21:** FTP (Dati/Controllo) — **23:** Telnet — **25:** SMTP
- **80:** HTTP — **110:** POP3 — **443:** HTTPS

4.7.3 Struttura del Segmento TCP

L'header TCP ha una dimensione minima di **20 byte** (suddivisi in 5 word da 32 bit). La sua struttura è fondamentale per gestire la connessione, l'affidabilità e il controllo di flusso:

- **Source Port / Destination Port (16 bit ciascuno):** Identificano i processi applicativi sui due host (multiplexing).
- **Sequence Number (32 bit):** Indica il numero di sequenza del primo byte di dati contenuto nel segmento. Serve a riordinare i dati al ricevente.
- **Acknowledgment Number (32 bit):** Valido solo se il flag ACK è attivo. Indica il numero del prossimo byte atteso dal mittente (ACK cumulativo).
- **HLEN (Header Length - 4 bit):** Indica la lunghezza dell'header in word da 32 bit. È necessario perché il campo *Options* ha lunghezza variabile.
- **Reserved (RSRVD - 6 bit):** Bit riservati per usi futuri.
- **Code Bits (Flags - 6 bit):** Bit di controllo che definiscono il tipo di segmento:

- **URG:** Indica che il campo *Urgent Pointer* è valido.
 - **ACK:** Conferma la ricezione dei dati (il campo *Ack Number* è significativo).
 - **PSH (Push):** Chiede al ricevente di consegnare i dati all'applicazione immediatamente, senza bufferizzare.
 - **RST (Reset):** Forza la chiusura immediata della connessione in caso di errore grave.
 - **SYN (Synchronize):** Utilizzato durante il *Three-way handshake* per sincronizzare i numeri di sequenza iniziali.
 - **FIN (Finish):** Indica che il mittente ha terminato l'invio dei dati.
- **Window (16 bit):** Utilizzato per il controllo di flusso. Specifica il numero di byte che il ricevente è disposto ad accettare (*Window Advertisement*).
 - **Checksum (16 bit):** Controllo di integrità dell'intero segmento (header + dati) e di un "pseudo-header" IP.
 - **Urgent Pointer (16 bit):** Punta all'ultimo byte dei dati urgenti nel payload.
 - **Options & Padding:** Utilizzati per funzioni avanzate (es. dimensione massima del segmento - MSS). Il *Padding* assicura che l'header termini su un confine di 32 bit.

Questo viene poi seguito da un **payload**, il quale contiene i dati applicativi ed è limitato dalla **MSS** (Maximum Segment Size) negoziata durante l'apertura.

4.7.4 Affidabilità e Trasmissione dei Dati

Per garantire la QoS, TCP bilancia l'**Affidabilità** (conferma della ricezione) e l'**Asincronismo** (efficienza del throughput) attraverso meccanismi di ARQ.

ACK cumulativi Un ACK N conferma la ricezione di tutti i byte precedenti a N . Se un segmento è perso, il destinatario continua a confermare l'ultimo byte ricevuto in ordine, forzando il mittente a rispedire i dati mancanti.

Continuous Requests Il mittente può inviare più segmenti senza attendere ogni ACK, superando il modello *Stop-and-Wait*.

Piggybacking Le conferme vengono spesso incluse nei segmenti dati inviati nella direzione opposta.

4.7.5 Controllo di Flusso: Sliding Window

TCP implementa il **controllo di flusso** attraverso il meccanismo della **finestra scorrevole (Sliding Window)**, che permette di adattare dinamicamente la velocità di trasmissione alla capacità di ricezione del nodo destinatario.

Il principio fondamentale è evitare la saturazione del buffer del ricevente mantenendo sempre sotto controllo la quantità di dati non ancora confermati.

Window Advertisement Il ricevente comunica periodicamente lo spazio disponibile nel proprio buffer tramite il campo **Window** presente nell'header TCP. Questo valore rappresenta il numero massimo di byte che il mittente può ancora trasmettere senza ricevere ulteriori conferme.

- Se la finestra è positiva, il mittente può continuare a inviare dati.
- Una **Zero Window** indica buffer saturo e sospende temporaneamente la trasmissione fino a un nuovo aggiornamento.

La finestra effettiva di invio dipende quindi dalla capacità annunciata dal ricevente e costituisce il cuore del meccanismo di *flow control*.

Logica del Mittente (TX) Il mittente mantiene traccia di:

- *NumLastMess*: numero di sequenza dell'ultimo byte inviato;
- *NumLastAck*: numero di sequenza dell'ultimo byte confermato.

Affinché la trasmissione sia valida deve sempre valere:

$$\text{NumLastMess} - \text{NumLastAck} + 1 \leq \text{TXWinSize} \quad (2)$$

ovvero la quantità di dati inviati ma non ancora confermati non può superare la dimensione della finestra di trasmissione.

Logica del Ricevente (RX) Il ricevente monitora invece:

- *NumWaitedMess*: prossimo byte atteso;
- *NumAcceptableMess*: limite massimo accettabile in base al buffer disponibile.

Deve quindi valere:

$$\text{NumAcceptableMess} - \text{NumWaitedMess} + 1 \leq \text{RXWinSize} \quad (3)$$

Questa condizione garantisce che i dati ricevuti rientrino sempre nella capacità del buffer locale, prevenendo overflow e perdita di segmenti.

Significato Operativo La Sliding Window realizza un controllo di flusso puramente **end-to-end**: i nodi intermedi della rete non mantengono stato della connessione, mentre mittente e ricevente cooperano adattando continuamente il ritmo di trasmissione alle risorse disponibili.

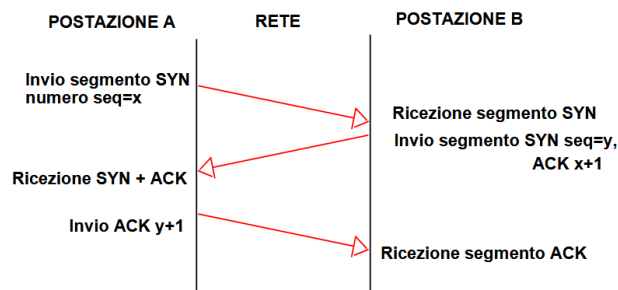
4.7.6 Fasi della Connessione

Il ciclo di vita di una connessione TCP si divide in tre fasi distinte:

1. **Fase Iniziale (Handshake)**: Stabilisce la connessione e negozia i parametri.
2. **Fase di Comunicazione**: Scambio dati *full-duplex* con controllo di flusso.
3. **Fase Finale (Chiusura)**: Può essere mono o bidirezionale (*Half-close*).

Three-way Handshake L'apertura avviene tramite tre passaggi necessari per sincronizzare i numeri di sequenza e misurare i tempi di rete:

1. **SYN**: Il Client invia un segmento con il flag SYN attivo e un proprio *Initial Sequence Number* (ISN_C).
2. **SYN-ACK**: Il Server risponde con un segmento che ha sia SYN che ACK attivi. Conferma l' ISN_C e propone il proprio ISN_S .
3. **ACK**: Il Client conferma la ricezione dell' ISN_S con un ACK.



Questa procedura permette a entrambi i peer di misurare il **RTT (Round Trip Time)**:

- Il Client misura il tempo tra l'invio del primo **SYN** e la ricezione del **SYN-ACK**.
- Il Server misura il tempo tra l'invio del **SYN-ACK** e la ricezione dell'ultimo **ACK**.

Durante questa fase avviene una forma di **Bidding** in cui ogni peer decide unilateralmente i parametri per il proprio verso di comunicazione, che l'altro deve accettare:

- **ISN (Initial Sequence Numbers)**: Scelti in modo casuale per evitare attacchi di *sequence guessing*.
- **MSS (Maximum Segment Size)**: La dimensione massima del payload per evitare frammentazione.
- **Window Scale Factor**: Per supportare finestre più ampie di 64KB su link ad alta velocità.
- **Timestamp**: Per il calcolo preciso del tempo di risposta e il coordinamento degli orologi.

In caso di perdita dei segmenti iniziali, si applicano *timeout* con intervalli esponenziali crescenti. È inoltre prevista l'**apertura simultanea**: se due host inviano un **SYN** l'uno all'altro nello stesso istante, TCP è in grado di convergere verso la creazione di un'unica connessione invece di due.

Four-way Handshake (Chiusura Graceful) TCP adotta una chiusura "gentile". Poiché una connessione TCP è composta da due flussi unidirezionali indipendenti, ogni verso deve essere chiuso separatamente. Se l'host A decide di chiudere la comunicazione con l'host B:

1. **FIN (A → B)**: A invia un segmento con il flag **FIN** attivo. Questo segnala che A non ha più dati da inviare.
2. **ACK (B → A)**: B riceve il **FIN** e risponde con un **ACK**.

A questo punto la connessione è in uno stato di **Half-Close**:

- L'output di A è chiuso, ma A può ancora ricevere dati da B e inviare i relativi **ACK**.
- B può continuare a trasmettere finché non ha esaurito i propri dati.

Per completare la chiusura totale:

3. **FIN (B → A)**: Quando anche B ha terminato, invia il proprio flag **FIN**.
4. **ACK (A → B)**: A risponde con l'ultimo **ACK**.

Dopo l'invio dell'ultimo **ACK**, l'host che ha iniziato la chiusura non chiude immediatamente la socket, ma entra in uno stato chiamato **TIME-WAIT**.

- **Scopo**: Garantire che l'ultimo **ACK** sia arrivato a destinazione. Se l'ultimo **ACK** andasse perso, B rinvierebbe il suo **FIN** e A deve essere ancora "vivo" per poter rispondere.
- **Sicurezza**: Impedisce che i segmenti ritardati di una vecchia connessione possano essere interpretati erroneamente come parte di una nuova connessione che utilizza la stessa quadrupla (IP/Porte).

4.7.7 Shutdown vs Close

A livello applicativo, esiste una distinzione netta tra la chiusura controllata del flusso (*Shutdown*) e la cessazione definitiva dell'endpoint (*Close*). L'operazione di **shutdown** permette di agire in modo granulare sulla connessione *full-duplex*, chiudendo i canali in modo indipendente:

- **SHUT_WR (Output)**: Il pari dichiara di non avere più dati da inviare (viene spedito il **FIN**). Tuttavia, resta in grado di leggere i dati che l'altro pari sta ancora trasmettendo.
- **SHUT_RD (Input)**: Il pari dichiara di non voler più leggere dati. Eventuali dati in arrivo vengono scartati dal buffer di ricezione a livello TCP.

Fase di Chiusura (lato che riceve la chiusura)

- **CLOSE-WAIT:** L'host ha ricevuto il FIN e ha inviato l'ACK. Attende che l'applicazione locale decida di chiudere il proprio lato della connessione.
- **LAST-ACK:** L'host ha inviato il proprio FIN e attende l'ultimo ACK per chiudere definitivamente.

Fasi Anomale e Chiusura Abortiva Vengono considerati anomali tutti i casi in cui il normale scambio di flag (SYN o FIN) fallisce ripetutamente o si verificano inconsistenze negli stati dei due peer.

- **Fallimento della Connessione:** Se un client tenta di connettersi (invio SYN) a una porta dove nessun processo è in ascolto (*LISTEN*), o se il server non è raggiungibile, il sistema operativo risponde con un segmento di **Reset**.
- **Segmento di Reset (Flag RST):** Il flag RST viene utilizzato per interrompere immediatamente una connessione. A differenza del FIN, il RST:
 - Non richiede un ACK di conferma.
 - Forza l'abbattimento della connessione in modo unilaterale.
 - Comporta la perdita immediata di tutti i dati ancora presenti nei buffer di trasmissione e ricezione.
- **Guasti di Rete:** In caso di guasti persistenti (es. timeout di ritrasmissione scaduti troppe volte), TCP procede alla chiusura in modo **abortivo**. Questo serve a liberare le risorse locali (socket, buffer) che altrimenti rimarrebbero impegnate indefinitamente in attesa di un peer non più esistente.
- **Inconsistenza di Stato:** Se un host riceve un segmento che non corrisponde allo stato attuale della connessione (ad esempio, dati che arrivano per una connessione già chiusa), invia un RST per sincronizzare la controparte sull'errore.

4.7.9 Timeout e Stima dell'RTT

A regime, TCP aggiorna costantemente i propri parametri operativi per riflettere lo stato corrente della rete. Il calcolo del **Retransmission Timeout (RTO)** è l'aspetto più critico per l'efficienza.

Granularità e Approssimazione Per ottimizzare le risorse computazionali del sistema operativo e gestire una granularità limitata dei timer, il timeout principale viene calcolato come un multiplo di valori discreti (100, 200 o 500 ms). Questa discretizzazione evita ricalcoli continui per variazioni millesimali irrilevanti e facilita la gestione delle code di timer nel kernel.

Il Problema delle Ritrasmissioni (Ambiguità degli ACK) In caso di perdita di un pacchetto e successiva ritrasmissione, sorge un problema fondamentale di interpretazione del segnale: se arriva un ACK, a quale invio si riferisce?

- **Ipotesi di associazione al primo invio:** Se l'ACK fosse per il primo messaggio (molto ritardato), ma noi lo considerassimo come risposta immediata al secondo, rischieremmo di calcolare un RTT falsamente basso, rendendo il protocollo **troppo reattivo** e instabile.
- **Ipotesi di associazione all'invio ritrasmesso:** Se l'ACK fosse in realtà per il primo invio ma noi lo associassimo al secondo, otterremmo un RTT falsamente alto, portando a un timeout troppo conservativo che **non si adegua** tempestivamente ai miglioramenti della rete.

Algoritmo di Karn Per risolvere l'ambiguità degli ACK, l'algoritmo di Karn impone due regole ferree:

1. **Esclusione delle ritrasmissioni:** Non si effettua il ricalcolo dell'RTT (e quindi del timeout) basandosi su segmenti che sono stati inviati più di una volta. Le stime vengono aggiornate solo per segmenti trasmessi con successo al primo tentativo.
2. **Back-off esponenziale:** In caso di timeout, il valore del timeout successivo viene raddoppiato (fino a un tetto massimo) per decongestionare rapidamente la rete, indipendentemente dal calcolo dell'RTT.

Il ricalcolo avviene solitamente tramite una media mobile esponenziale ponderata EWMA (*Exponentially Weighted Moving Average*):

$$\text{Timeout}_{new} = \alpha \cdot \text{Timeout}_{old} + (1 - \alpha) \cdot \text{RTT}_{current} \quad (4)$$

Dove α permette di bilanciare l'importanza della storia pregressa rispetto alla situazione istantanea.

4.7.10 Flow Control e Ottimizzazione

Il controllo di flusso in TCP non serve solo a non saturare il ricevente, ma anche a ottimizzare l'uso della banda disponibile attraverso la gestione della finestra e della dimensione dei segmenti.

Silly Window Syndrome Si verifica quando il ricevente annuncia finestre molto piccole o il mittente invia segmenti di pochi byte. Questo causa un'inefficienza estrema (overhead), poiché l'header (40 byte tra IP e TCP) risulterebbe molto più grande dei dati trasportati. Per evitarlo:

- Il ricevente non aggiorna la finestra (*Window Advertisement*) finché non ha liberato una porzione significativa del buffer.
- Il mittente attende di accumulare una quantità di dati sufficiente a riempire un segmento di dimensioni ragionevoli.

Algoritmo di Nagle L'algoritmo di Nagle è la soluzione lato mittente per evitare il sovraccarico di piccoli pacchetti (tipico di applicazioni come Telnet):

- Se i dati da inviare sono pochi, il mittente li invia solo se non ci sono ACK pendenti.
- Se c'è un messaggio "corto" in volo di cui non è ancora arrivata la conferma, il mittente bufferizza i nuovi dati in arrivo dall'applicazione finché non può comporre un segmento di dimensione massima (MSS) o finché non riceve l'ACK del messaggio precedente.

4.7.11 Eccezioni alla Trasparenza: PUSH e URG

Sebbene TCP tenda a raggruppare i dati per efficienza, esistono meccanismi per forzare l'invio immediato:

- **Flag PUSH (PSH):** Permette all'applicazione di superare l'algoritmo di Nagle. Quando impostato, il mittente invia immediatamente i dati nel buffer e il ricevente li consegna all'applicazione senza attendere che il proprio buffer sia pieno. È fondamentale per applicazioni interattive.
- **Dati Urgenti (Flag URG):** Utilizzato per dati fuori banda (o quasi). Sebbene il flusso rimanga unico, il flag URG e l'**Urgent Pointer** segnalano che una parte del segmento (spesso 1 solo byte di controllo, come un *interrupt*) ha priorità. Il ricevente viene notificato immediatamente della posizione dei dati urgenti nel flusso per poterli processare fuori ordine.

4.7.12 Congestione

La congestione si verifica quando la richiesta di risorse (banda, buffer dei router) supera la capacità di offerta, rendendo impossibile la consegna dei dati in tempi utili. Per TCP, la congestione è considerata un evento **distruttivo** che richiede un intervento immediato e unilaterale.

Identificazione e Reazione Immediata TCP identifica la congestione attraverso lo scatto del **timeout**. Per evitare il sovraccarico della rete, il mittente agisce autonomamente:

- **Dimezza la finestra** di invio (riduzione drastica del carico).
- **Raddoppia il timeout** (back-off esponenziale) per dare tempo alla rete di smaltire le code.
- **Riparte con Slow Start**: Terminata la fase critica, non si torna subito al regime precedente, ma si riparte da una finestra minima per un "passaggio dolce".

4.7.13 Meccanismi di Controllo: *cwnd* e *ssthresh*

Il controllo della congestione introduce variabili aggiuntive che lavorano insieme alla finestra del ricevente (*rwnd*):

- **Congestion Window (*cwnd*)**: La finestra variabile calcolata dal mittente in base allo stato della rete. La finestra reale di invio sarà sempre: $\min(cwnd, rwnd)$.
- **Slow Start Threshold (*ssthresh*)**: La soglia che divide la fase di recupero rapido da quella esplorativa.

Le Due Fasi di Operatività Il comportamento del protocollo varia a seconda della posizione rispetto alla soglia:

1. **Slow Start** ($cwnd < ssthresh$):

- Utilizzata nella fase iniziale ("situazione fredda") o dopo una congestione grave.
- **Crescita Esponenziale**: La *cwnd* aumenta rapidamente per ogni ACK ricevuto, cercando di occupare la banda disponibile nel minor tempo possibile.

2. **Congestion Avoidance** ($cwnd \geq ssthresh$):

- Utilizzata quando si è vicini al limite di saturazione ("situazione calda").
- **Crescita Lineare**: Si aggiunge un solo segmento per ogni RTT. Questa fase è esplorativa e cauta; si cerca il punto di rottura senza causare nuovi picchi di congestione.

4.7.14 Strategie principali riassunte

TCP adotta una serie di strategie operative per migliorare affidabilità, efficienza e stabilità della comunicazione in presenza di variabilità della rete. Tali meccanismi agiscono principalmente sulla gestione dei timeout, delle finestre di trasmissione e sull'adattamento dinamico alle condizioni di congestione.

- **Ricalcolo del Timeout**

Il valore del *Retransmission Timeout* (RTO) non è fisso ma viene ricalcolato dinamicamente in funzione del tempo di andata e ritorno (*RTT*). TCP utilizza una media mobile esponenziale (EWMA) per ottenere una stima stabile, evitando reazioni eccessive a variazioni temporanee della rete. Inoltre, i segmenti ritrasmessi non vengono considerati nel calcolo dell'RTT per evitare ambiguità sugli ACK ricevuti.

- **Exponential Backoff**

Quando si verifica un timeout, TCP assume una possibile congestione e aumenta progressivamente l'intervallo di ritrasmissione raddoppiando il timeout precedente. Questo meccanismo riduce rapidamente il carico sulla rete e previene il collasso dovuto a ritrasmissioni troppo aggressive.

- **Silly Window Syndrome**

La *Silly Window Syndrome* si verifica quando mittente o ricevente generano finestre molto piccole, causando l'invio di segmenti contenenti pochissimi byte rispetto all'overhead dell'header TCP/IP. Per evitarla:

- il ricevente aggiorna la finestra solo quando è disponibile spazio sufficiente nel buffer;
- il mittente tende ad aggregare i dati prima dell'invio (supportato anche dall'algoritmo di Nagle).

- **Limiti al TIME-WAIT**

Lo stato *TIME-WAIT* garantisce la corretta terminazione della connessione e previene la confusione con segmenti ritardati appartenenti a connessioni precedenti. Tuttavia, mantenere molte connessioni in questo stato può consumare risorse di sistema. Per questo motivo i sistemi operativi adottano strategie di ottimizzazione, come il riuso controllato delle socket o la riduzione del tempo massimo di attesa in condizioni sicure.

- **Long Fat Pipes**

Nei collegamenti ad alta banda e alta latenza (detti *Long Fat Networks*), la finestra TCP standard può risultare insufficiente a saturare il canale. TCP introduce quindi estensioni come:

- *Window Scaling*, per supportare finestre superiori a 64 KB;
- ottimizzazioni del controllo di congestione per mantenere elevato il throughput su link a lunga distanza.

5 Servizi applicativi standard Internet

La diffusione capillare di Internet è stata trainata dalla standardizzazione di servizi applicativi che hanno creato un'ecosistema omogeneo per gli utenti. Questi servizi permettono l'interoperabilità tra sistemi eterogenei grazie a protocolli comuni che astraggono le interfacce locali.

Le applicazioni si dividono principalmente in due modelli architetturici:

- **Modello Cliente-Servitore (Client-Server):** Un'architettura asimmetrica dove un servitore passivo attende le richieste da parte di clienti attivi. È il modello alla base di servizi come il Web (HTTP) e la posta elettronica (SMTP/POP3).
- **Infrastruttura Distribuita:** Sistemi in cui le responsabilità sono ripartite tra più nodi senza necessariamente un punto centrale di controllo, migliorando la scalabilità e la tolleranza ai guasti (es. CDN, sistemi P2P o reti di Name Server distribuiti).

L'adozione di interfacce standardizzate ha permesso lo sviluppo di applicazioni client indipendenti dal sistema operativo, favorendo una crescita esponenziale sia dell'offerta di contenuti che della base utenti.

5.1 Servizi UNIX (livello applicativo)

Il sistema operativo UNIX ha introdotto servizi fondamentali per l'interazione tra macchine remote, strutturandoli su tre pilastri operativi:

- **Terminale remoto:** Permette a un utente di loggarsi su una macchina distante e operare come se fosse seduto alla console locale.
- **File transfer:** Protocolli per lo spostamento di file tra file system eterogenei.
- **Comandi remoti:** Possibilità di invocare l'esecuzione di applicazioni su nodi remoti, sfruttando la potenza di calcolo distribuita.

5.1.1 Classificazione dei Servizi

I servizi si possono dividere in base alla loro portabilità:

- **Servizi specifici UNIX (r-commands):** Come `rlogin`, `rwho`, `rsh`, `rup`. Questi si basavano spesso su una fiducia implicita tra macchine ("trusted hosts") e sono stati precursori di sistemi più sicuri.
- **Servizi Generali (Standard Internet):** Protocolli come `ftp`, `telnet` e i protocolli di **mail** (SMTP), progettati per funzionare tra sistemi operativi diversi.

L'infrastruttura si regge su concetti cardine che garantiscono l'operatività:

- **Trasparenza alla allocazione:** L'utente può utilizzare la risorsa senza necessariamente conoscere l'esatta posizione fisica del server, purché il servizio sia raggiungibile.
- **Modello C/S (Client/Server):** Una netta separazione tra il processo che richiede il servizio (Client) e quello che lo eroga (Server o Daemon).
- **Standardizzazione:** L'uso di protocolli aperti (spesso definiti tramite RFC) che permettono a implementazioni diverse di comunicare senza ambiguità.

5.2 Terminale remoto

L'obiettivo del terminale remoto è consentire a un utente di operare su un sistema operativo distante come se fosse collegato localmente, superando l'eterogeneità dei sistemi coinvolti. Gli strumenti storici principali sono:

- **Telnet:** Protocollo generale e standard Internet per l'accesso remoto.
- **Rlogin:** Protocollo specifico del mondo BSD UNIX, più semplice ma meno portabile.

5.2.1 Il Problema dell'Eterogeneità

I terminali fisici e i relativi emulatori possono differire drasticamente per:

- **Set e codifica dei caratteri:** Differenze tra ASCII, EBCDIC o diverse estensioni.
- **Geometria della visualizzazione:** Numero variabile di righe e colonne supportate.
- **Tasti funzione:** Diverse sequenze di escape per gestire tasti come frecce, F1-F12 o il tasto Canc.

5.2.2 Network Virtual Terminal (NVT)

L'NVT non è solo un formato statico, ma un'astrazione flessibile. Una volta stabilito l'accordo iniziale, le caratteristiche del terminale possono variare in base alla negoziazione tra client e server.

Il protocollo utilizza un'unica connessione TCP dove i dati utente e i segnali di controllo sono multiplexati. Per distinguere i comandi dai dati, si utilizza il carattere speciale **IAC** (Interpret As Command, valore decimale 255).

I principali caratteri di controllo includono:

- **Segnali base:** NUL (null), BEL (bell), BS (backspace), HT (tab), LF (line feed), CR (carriage return).
- **Comandi di controllo:**
 - **IAC (255):** Precede ogni comando.
 - **IP (Interrupt Process):** Sospende o interrompe l'esecuzione sul server.
 - **AO (Abort Output):** Sospende l'invio di dati dal server al client.
 - **AYT (Are You There):** Verifica se il sistema remoto è ancora attivo.
 - **EC (Erase Character) / EL (Erase Line):** Gestione della cancellazione.
 - **SYNC:** Utilizzato per svuotare il buffer in caso di segnali urgenti.

Negoziazione delle Opzioni La negoziazione può avvenire sia all'avvio della sessione sia in qualsiasi momento successivo, permettendo al terminale di evolvere (ad esempio, passando da una modalità "carattere per carattere" a una modalità "linea per linea").

Il protocollo di negoziazione è **simmetrico** e si basa su quattro verbi fondamentali:

Comando	Significato
WILL	Il mittente desidera attivare una specifica opzione.
WONT	Il mittente rifiuta di attivare o desidera disattivare un'opzione.
DO	Il mittente richiede che il destinatario attivi un'opzione.
DONT	Il mittente richiede che il destinatario disattivi un'opzione.

Le regole di cortesia del protocollo impongono che le richieste **DONT** e **WONT** siano **sempre accettate**, poiché rappresentano un ritorno allo stato minimo garantito dell'NVT, garantendo che la connessione non cada mai in uno stato inconsistente o non supportato.

5.3 Telnet

Telnet opera come un protocollo simmetrico su connessioni TCP, dove client e server collaborano per mantenere l'illusione di un terminale locale. L'architettura software si basa su una chiara distinzione di ruoli:

- **Cliente:** Gestisce l'interfaccia con l'utente locale, cattura i caratteri digitati, li converte in formato NVT e li trasmette. Gestisce inoltre la visualizzazione dei dati di ritorno.
- **Servitore:** Un demone (ascoltatore) riceve la richiesta sulla porta 23 e genera un **processo figlio** dedicato per ogni sessione. Questo figlio interagisce con il sistema operativo tramite un terminale virtuale (pseudo-terminal).

5.3.1 Operatività e Modalità di Lavoro

La gestione del buffer di invio è critica per bilanciare reattività e overhead di rete. Si distinguono tre modalità principali:

- **One character at a time:** Ogni singolo carattere viene inviato in un pacchetto TCP. Il server effettua l'echo del carattere per confermare la ricezione. Sebbene sia la modalità più interattiva, genera un enorme **overhead** (40 byte di header per ogni byte di dati).
- **One line at a time:** I caratteri vengono bufferizzati localmente dal client e inviati solo alla pressione di "Invio". Riduce il traffico ma impedisce l'uso di applicazioni interattive "full-screen" (come editor di testo).
- **Linemode:** Una modalità avanzata e ottimizzata che permette di gestire l'editing della riga localmente pur mantenendo la capacità di passare alla modalità carattere quando necessario.

Uso di dati urgenti e Controllo Per gestire situazioni in cui lo stream di dati è congestionato (ad esempio, quando l'utente preme **Ctrl+C** durante un output massivo), Telnet utilizza il meccanismo **TCP Urgent Data**.

Alcuni segnali critici includono:

- **Flush:** Comanda al destinatario di scartare tutti i dati attualmente presenti nel buffer.
- **Flow Control:** Possibilità di abilitare o disabilitare il controllo di flusso lato client per evitare blocchi della visualizzazione.
- **Sliding Window:** Negoziazione dinamica della dimensione della finestra per ottimizzare il throughput in base alla latenza della rete.

5.4 Rlogin

Rlogin (Remote Login) è un servizio progettato specificamente per l'ecosistema UNIX/Linux. A differenza di Telnet, che punta alla massima interoperabilità, Rlogin ottimizza l'accesso tra macchine che condividono lo stesso sistema operativo.

Il sistema si basa sul concetto di **Trusted Host**:

- L'autenticazione può avvenire senza password tramite i file di configurazione **.rhosts** (nella home dell'utente) o **/etc/hosts.equiv** (a livello di sistema).
- Se una macchina è considerata "fidata", l'utente può passare da un nodo all'altro senza doversi riqualificare, facilitando l'operatività in cluster locali.

5.4.1 Caratteristiche Operative

Rlogin è significativamente più snello di Telnet (migliaia di righe di codice contro decine di migliaia), ma presenta limiti prestazionali dovuti alla sua semplicità:

- **Esportazione dell'ambiente:** Il client comunica automaticamente al server informazioni come il tipo di terminale (**TERM**) e la velocità della linea.
- **Gestione I/O:** Collega direttamente *stdin*, *stdout* e *stderr* del client allo shell remoto.
- **Modalità carattere:** Lavora esclusivamente un carattere alla volta. Questo può causare un overhead significativo e una percezione di latenza.
- **Controllo di flusso:** A differenza di Telnet, caratteri come **Ctrl+S** (stop) e **Ctrl+Q** (start) vengono gestiti localmente dal client per una risposta immediata.

5.4.2 Implementazione e Architettura

L'implementazione si basa su una struttura a più processi per gestire il flusso bidirezionale in modo asincrono:

Lato Client Il processo si sdoppia dopo aver stabilito la connessione TCP:

- **Padre:** Legge dallo standard input dell'utente e invia i dati al server.
- **Figlio:** Riceve i dati dal server e li scrive sullo standard output (video) dell'utente.

Lato Server (rlogind) Il demone `rlogind` riceve la connessione e:

1. Valida l'accesso tramite i file di fiducia.
2. Alloca uno **pseudo-terminale (PTY)** composto da una coppia master-slave.
3. Avvia una shell remota collegata alla parte slave del PTY.

Esecuzione remota (rsh) Strettamente correlato è il comando `rsh` (Remote Shell), che permette l'invocazione di un singolo comando senza aprire una sessione interattiva completa:

```
rsh nodo_remoto comando
```

In questo caso, il server esegue il comando e restituisce direttamente l'output, chiudendo la connessione al termine.

5.5 File Transfer Protocol (FTP)

Il protocollo FTP (e la sua variante semplificata TFTP, basata su UDP) è progettato per il trasferimento affidabile di file tra host. A differenza dei semplici comandi di copia remota, FTP crea un ambiente di sessione completo che gestisce:

- **Controllo dell'identità:** Richiede autenticazione (Username/Password), pur supportando l'accesso **Anonymous** (solitamente usando "anonymous" come utente e un'email come password) per server pubblici.
- **Accesso Interattivo:** Permette all'utente di navigare nel file system remoto, gestire directory e selezionare file.
- **Tipizzazione dei dati:** Consente di specificare esplicitamente se il trasferimento deve avvenire in modalità **ASCII** (per file di testo, con conversione dei caratteri di fine riga) o **Binary/Image** (per file eseguibili, immagini, etc.).

5.5.1 Operazioni Comuni

Il client FTP espone un set di comandi standard per l'interazione:

- **get / put:** Trasferimento di un singolo file (dal server al client e viceversa).
- **mget / mput:** Trasferimento multiplo di file (usando wildcard).
- **dir / ls:** Elenco dei file nella directory remota.
- **cd / lcd:** Cambio directory sul server (cd) o sul client locale (lcd).
- **ascii / binary:** Commutazione della modalità di traduzione dei dati.

5.5.2 Codifica e Risposte

Le risposte del server seguono lo standard NVT e sono strutturate gerarchicamente per facilitare il parsing automatico da parte del software client. La struttura a tre cifre permette una diagnosi rapida dello stato:

Codice	Significato (Prima Cifra - Macro-stato)
1xx	Risposta positiva preliminare (azione avviata).
2xx	Risposta positiva completa (azione conclusa con successo).
3xx	Risposta positiva intermedia (necessari altri dati, es. password).
4xx	Risposta negativa transitoria (errore temporaneo, riprovare).
5xx	Risposta negativa permanente (comando fallito).

La seconda cifra specifica la categoria funzionale dell'evento o dell'errore:

Codice	Significato (Seconda Cifra - Categoria)
x0x	Sintassi: Errori nei comandi o parametri non riconosciuti.
x1x	Informazioni: Risposte a richieste di aiuto o stato.
x2x	Connessione: Problemi relativi ai canali di controllo o dati.
x3x, x4x	Non specificati: Riservati per usi futuri o estensioni.
x5x	File system: Stato del file system remoto (file non trovato, permessi).

5.5.3 Implementazione

Il protocollo FTP sfrutta TCP per garantire un accesso concorrente e affidabile. La caratteristica distintiva è la persistenza di almeno due collegamenti logici per ogni sessione client-server:

- **Connessione di controllo:** Utilizzata per lo scambio di comandi e risposte (sempre attiva).
- **Connessioni dati:** Utilizzate per il trasferimento effettivo dei file o degli elenchi di directory (create al bisogno).

Questa architettura richiede l'impiego di processi multipli coordinati.

Struttura del Server Il server gestisce le richieste attraverso un'organizzazione gerarchica:

- **Processo Master (ftpd):** È il demone principale che rimane in ascolto sulla porta standard (solitamente 21). Una volta accettata una connessione, crea uno **slave** dedicato e torna immediatamente in ascolto per altre richieste.
- **Slave di controllo (Protocol Interpreter - PI):** Un processo che gestisce la sessione di controllo con il client per tutta la sua durata, interpretando i comandi NVT.
- **Slave di trasferimento (Data Transfer Process - DTP):** Un processo creato dinamicamente ogni volta che è richiesto un trasferimento dati. Possono essere creati molteplici processi DTP sequenziali all'interno della stessa sessione di controllo.

Struttura del Client Simmetricamente, anche il client non è un blocco monolitico ma separa le funzioni:

- Un'interfaccia utente (o processo di controllo) per inviare i comandi.
- Un'entità di trasferimento dati che si occupa di leggere/scrivere sul file system locale durante le operazioni di **get** o **put**.

5.5.4 Porte e Connessioni

Il protocollo FTP opera secondo un modello a due canali separati, ciascuno associato a una porta specifica sul lato server:

- **Collegamento di Controllo (Porta 21):** Utilizzata per l'invio dei comandi dal client e la ricezione dei codici di risposta dal server. Questa connessione utilizza il protocollo NVT (Network Virtual Terminal) e rimane aperta per l'intera durata della sessione.
- **Collegamento Dati (Porta 20):** In modalità attiva (*Active Mode*), il server utilizza questa porta come sorgente per stabilire la connessione verso il client per il trasferimento effettivo dei dati (file o listati di directory).

5.5.5 Impegno risorse e continuità

L'efficienza di FTP si manifesta nella gestione delle risorse durante trasferimenti critici o di grandi dimensioni.

Gestione delle Interruzioni In caso di trasferimento di grandi moli di dati, se la connessione si interrompe, il protocollo prevede meccanismi di **checkpointing**:

- Non è necessario ripartire dall'inizio del file.
- Grazie al comando **REST** (Restart), è possibile riprendere il trasferimento dall'ultima posizione correttamente ricevuta (offset), ottimizzando i tempi e la banda.

Coordinamento delle Porte Il client inizialmente stabilisce il collegamento di controllo verso la porta 21 del server partendo da una propria porta effimera (*xxx*).

- **Connessione Dati Iniziale:** La stessa porta *xxx* del cliente può essere riutilizzata per la prima connessione dati (nella modalità attiva classica).
- **Connessioni Successive:** Per trasferimenti multipli o paralleli, il cliente deve coordinarsi con il server tramite il comando **PORT**. Questo comando passa al server l'indirizzo IP e il numero di porta (*yyy*) su cui il client si metterà in ascolto per la nuova connessione dati.
- **Porte Multiple:** Il cliente può indicare porte addizionali, solitamente numerate in modo incrementale ($xxx + 1$, $xxx + 2$), per gestire flussi paralleli o sessioni distinte.

Senza questo scambio esplicito di numeri di porta (coordinamento), il servizio non potrebbe funzionare poiché il server non saprebbe a quale indirizzo di trasporto inviare i blocchi di dati richiesti.

5.5.6 Iniziativa delle connessioni (Modalità Attiva)

Sebbene nel modello client-server sia solitamente il client a iniziare ogni connessione, il protocollo FTP in modalità standard (Attiva) inverte questo comportamento per il canale dati.

Connessione di Controllo L'iniziativa è sempre del **Client**:

- Il Client esegue una `connect()` verso l'indirizzo IP del Server sulla **porta 21**.
- Il Client utilizza una propria porta effimera *xxx*.

Connessione Dati (Il ruolo "Attivo") In questa fase, i ruoli per l'apertura del socket si invertono:

1. **Il Client diventa "passivo"**: Apre una porta in ascolto (`accept()`) e comunica il numero di tale porta al server tramite il comando `PORT`.
2. **Il Server diventa "attivo"**: Ricevuto il comando (`get` o `put`), il server prende l'iniziativa di connessione. Esegue una `connect()` partendo dalla sua **porta fissa 20** verso la porta negoziata del client.

Canale	Iniziativa (Connect)	Porta Server	Porta Client
Controllo	Client	21	<i>xxx</i>
Dati (Active)	Server	20	<i>xxx (o yyy)</i>

5.5.7 Protocollo vs Politiche

Il successo e la longevità di FTP (e dei protocolli TCP/IP in generale) derivano dalla distinzione tra la specifica del protocollo e le strategie implementative:

- **Meccanismi (Protocollo)**: Rappresentano lo standard "puro". Definiscono il set di messaggi, la sintassi dei comandi e le sequenze di interazione che devono rimanere immutabili (*legacy*) per garantire l'interoperabilità nel tempo. Le API e i comandi di base non devono cambiare per non rompere la compatibilità con i sistemi esistenti.
- **Politiche (Strategie)**: Sono le decisioni locali prese dalle parti applicative. Riguardano la gestione delle risorse, le scelte di sicurezza, l'allocazione delle porte o le strategie di caching. Queste possono evolvere seguendo lo stato dell'arte tecnologico senza alterare il protocollo sottostante.

La definizione del set di comandi iniziale è un passaggio critico:

- Un set di meccanismi ben progettato e "fissato" funge da fondamenta stabili.
- Tale stabilità permette di implementare politiche estremamente flessibili e dinamiche, capaci di adattarsi a nuovi contesti (es. l'evoluzione dai mainframe ai moderni sistemi distribuiti) pur mantenendo lo stesso linguaggio di comunicazione.

5.5.8 API standard

Ad FTP è associato un insieme di comandi standardizzati che permettono di manipolare file e directory, gestire la sessione e ottimizzare i trasferimenti. Questi comandi riflettono le operazioni fondamentali di un file system remoto:

- **ls / dir**: Elenca i file e le sottodirectory presenti nella directory corrente del server.
- **chdir (cd)**: Cambia la directory di lavoro sul sistema remoto.
- **get (RETR)**: Recupera una copia di un file dal server al client.
- **put (STOR)**: Invia un file dal client al server.
- **port (PORT)**: Specifica l'indirizzo IP e la porta su cui il client attende la connessione dati (modalità attiva).
- **mget / mput**: Versioni multiple dei comandi `get/put` per trasferire gruppi di file utilizzando pattern (wildcard).
- **checkpoint**: Definisce punti di controllo durante il trasferimento per monitorarne l'avanzamento.
- **restart (REST)**: Consente di riprendere un trasferimento interrotto a partire da un marker specifico o un offset binario.

5.6 Posta elettronica

La posta elettronica (e-mail) è un sistema di comunicazione asincrona che permette lo scambio di messaggi tra utenti senza che questi debbano essere contemporaneamente attivi o collegati alla rete. L'infrastruttura si basa su standard che definiscono il supporto per l'invio, l'instradamento e la ricezione.

5.6.1 Architettura degli Agenti

Il sistema si articola attraverso l'interazione di due tipologie principali di entità software:

- **User Agent (UA):** È l'interfaccia utilizzata dall'utente finale (es. Outlook, Gmail, Thunderbird) per comporre, leggere e organizzare i messaggi.
- **Message Transfer Agent (MTA):** È il software di sistema (server) responsabile dell'instradamento e della consegna dei messaggi verso la destinazione corretta (es. Postfix, Sendmail).

5.6.2 Il Modello di Servizio

A differenza di servizi interattivi come FTP o Telnet, l'e-mail opera secondo un modello **asincrono reciproco**:

- **Disaccoppiamento (Spooling):** Il mittente consegna il messaggio al proprio server (MTA locale) e può terminare la sessione. Il messaggio viene depositato in un'area di memoria temporanea chiamata *buffer* o *spool*.
- **Comunicazione tra Agenti:** Gli agenti intermedi lavorano a coppie comunicando in modo **batch sincrono bloccante**. Questo significa che, durante una specifica transazione tra due server, entrambi devono essere attivi per garantire il trasferimento del messaggio da uno spool all'altro.
- **Affidabilità:** Se il server di destinazione non è raggiungibile, l'MTA mittente conserva il messaggio e riprova l'invio a intervalli regolari, garantendo la consegna differita.

L'infrastruttura complessiva garantisce che il servizio risulti asincrono per l'utente finale, pur basandosi su una catena di trasferimenti sincroni tra i server lungo il percorso.

5.6.3 Dati e protocollo

Il formato standard delle e-mail è storicamente basato su testo semplice (ASCII a 7 bit), una scelta che ne garantisce la massima compatibilità ma che ne limita il contenuto. Un messaggio è diviso in due parti fondamentali:

- **Header (Intestazione):** Contiene i metadati necessari per l'instradamento e la visualizzazione.
- **Corpo (Body):** Il contenuto informativo del messaggio, separato dall'header da una riga vuota.

MIME (Multipurpose Internet Mail Extensions) Per superare i limiti dell'ASCII puro e permettere l'invio di file binari, immagini, audio o testo in diverse codifiche, è stato introdotto lo standard **MIME**. Esso non sostituisce il protocollo di trasporto, ma arricchisce la struttura del messaggio.

5.6.4 Formato dati

L'header è composto da una serie di campi standardizzati:

- **From / To:** Mittente e destinatario principale.
- **Date:** Data e ora di composizione.
- **Subject:** Oggetto del messaggio.

- **CC / BCC:** Copia Conoscenza e Copia Conoscenza Nascosta (Blind CC).
- **Reply-to:** Indirizzo a cui inviare eventuali risposte.
- **Message-ID:** Identificativo univoco del messaggio generato dal sistema.

Campi specifici MIME MIME introduce intestazioni aggiuntive per descrivere la natura del contenuto:

- **MIME-Version:** Indica la versione dello standard utilizzata (tipicamente 1.0).
- **Content-Type:** Definisce il tipo di dati (es. `text/html`, `image/jpeg`, `multipart/mixed`).
- **Content-Transfer-Encoding:** Specifica come i dati binari sono stati convertiti in testo (es. `base64` o `quoted-printable`).
- **Content-ID / Content-Description:** Identificativi e descrizioni opzionali per le singole parti del messaggio.

5.6.5 Sistema di nomi e Indirizzamento

L'identità di un utente nel sistema e-mail è definita dalla coppia *mailbox@nodo*. Il sistema deve mappare questi identificatori logici in indirizzi IP fisici, gestendo al contempo strutture complesse:

- **Alias e Forwarding:** Un utente può avere più nomi (pseudonimi) che puntano alla stessa mailbox, o inoltrare automaticamente i messaggi verso altri indirizzi.
- **Nomi Multipli e Liste:** È possibile definire un unico identificatore che corrisponde a un gruppo di destinatari (Mailing List).
- **Gestione dei Cicli:** Le mailing list possono includere destinatari non locali; una configurazione errata (es. A invia a B, che reinvia ad A) può generare **cicli infiniti**, che gli MTA devono rilevare e interrompere.

5.6.6 Componenti di servizio e MTA

L'attività di trasferimento è a carico del **Message Transfer Agent (MTA)**, che agisce come un cliente attivo per conto dell'UA:

1. Effettua il mapping del dominio di destinazione in un indirizzo IP (diretto o di un intermediario).
2. Tenta di stabilire una connessione TCP con il server successivo.
3. Se la connessione ha successo, trasferisce il messaggio e libera le risorse locali (spool).

5.6.7 DNS e Instradamento

Il routing della posta elettronica è logicamente separato dal routing IP standard. Mentre IP cerca il percorso più breve per un pacchetto, il sistema mail cerca il server responsabile per un dominio.

Record MX (Mail Exchanger) Il DNS gioca un ruolo fondamentale attraverso i record di tipo **MX**:

- Un record MX specifica quale server è autorizzato a ricevere posta per un determinato dominio.
- Permette di definire priorità diverse per server multipli, garantendo ridondanza.
- Consente l'uso di **Mail Gateway**: macchine intermedie che accettano la posta per conto di domini interni non direttamente accessibili.

Protocolli e Porte L'infrastruttura si divide in invio e recupero:

Fase	Protocollo	Porta Standard
Trasferimento (Invio/Relay)	SMTP	25
Accesso Mailbox (Ricezione)	POP3	110
Accesso Mailbox (Sincroniz.)	IMAP	143

L'accesso finale avviene tramite vari client (UA) come Outlook o interfacce Web-based, che utilizzano POP3 o IMAP per recuperare i messaggi depositati dai server SMTP.

5.7 Simple Mail Transfer Protocol (SMTP)

SMTP è il protocollo deputato al trasferimento dei messaggi tra agenti di posta (MTA). Opera sulla porta **25** e si basa su uno scambio di messaggi in chiaro (ASCII).

Durante una sessione SMTP, i ruoli di *sender* (client) e *receiver* (server) sono legati alla transazione specifica. Una volta terminato l'invio di un messaggio, i ruoli possono essere invertiti per permettere la trasmissione di posta nel verso opposto senza dover ristabilire la connessione TCP.

5.7.1 Struttura della Comunicazione

La conversazione avviene tramite:

- **Comandi:** Parole ASCII inviate dal mittente (es. `HELO`, `MAIL FROM`, `RCPT TO`, `DATA`, `QUIT`).
- **Risposte:** Codici numerici a 3 cifre seguiti da un testo descrittivo.

5.7.2 Codifica standard risposte

Le tabelle seguenti dettagliano la semantica dei codici di risposta, fondamentale per il controllo di flusso del protocollo:

Codice	Significato (Prima Cifra - Successo/Errore)
1xx	Risposta positiva preliminare.
2xx	Risposta positiva completa (azione conclusa con successo).
3xx	Risposta positiva intermedia (attesa di dati, es. corpo mail).
4xx	Risposta negativa transitoria (errore di rete o server occupato).
5xx	Risposta negativa permanente (indirizzo inesistente, spam).

Codice	Significato (Seconda Cifra - Categoria)
x0x	Sintassi: Errori di digitazione dei comandi.
x1x	Informazioni: Risposte a comandi <code>HELP</code> o di sistema.
x2x	Connessione: Stato del canale di trasmissione.
x3x, x4x	Non specificati: Riservati per usi futuri.
x5x	Mail system: Stato del sistema di recapito locale.

5.7.3 Mail Transaction

Il trasferimento di una singola mail è considerato una **transazione atomica**. Se la sequenza dei comandi viene interrotta o riceve un errore di tipo 4xx o 5xx, la transazione fallisce. In caso di problemi durante la transazione, il mittente può ricevere messaggi di errore disordinati o tentare ripetutamente l'invio (*retry policy*), aumentando il traffico di rete ma garantendo l'asincronicità del servizio.

5.8 Usenet News

USENET è un sistema di bacheche elettroniche (bulletin board) organizzato in **gruppi di discussione** (newsgroup). Ogni gruppo è dedicato a un tema specifico (*topic*) e permette un'interazione pubblica asincrona basata sullo scambio di articoli.

5.8.1 Gerarchie Principali

I gruppi sono organizzati in una struttura gerarchica ad albero per facilitare la navigazione e la distribuzione:

- **comp:** Informatica e tecnologia.
- **sci:** Scienze e ricerca.
- **rec:** Svago, hobby e attività ricreative.
- **soc:** Temi sociali e culture.
- **talk:** Dibattiti e opinioni.
- **alt:** Gerarchia alternativa (più libera e meno regolamentata).
- **biz:** Affari e attività commerciali.

5.8.2 Architettura e Coordinamento

L'architettura di USENET si differenzia dai sistemi centralizzati per l'uso di database distribuiti:

- **Modello di Database:** I server di news mantengono database locali di articoli. Questi database sono **coordinati ma non consistenti** (non vi è garanzia che tutti i server abbiano lo stesso articolo nello stesso istante; la propagazione avviene per "inondazione" o *flooding*).
- **Nodo Client:** L'utente utilizza un *newsreader* (UA) che si connette a un server locale per scaricare le intestazioni o i corpi dei messaggi e per inviare (*post*) nuovi articoli.

5.9 Network News Transfer Protocol (NNTP)

Il protocollo standard per lo scambio di news è l'**NNTP** (Network News Transfer Protocol), che opera sulla porta TCP **119**.

5.9.1 Meccanismo di Comunicazione

Seguendo il modello già visto in SMTP e FTP, la comunicazione è basata su testo ASCII:

- **Comandi Client:** Parole ASCII (es. **GROUP** per selezionare un gruppo, **ARTICLE** per leggere un messaggio, **POST** per inviare).
- **Risposte Server:** Codici numerici a 3 cifre seguiti da testo esplicativo (es. 200 per successo, 411 per gruppo non trovato).

5.9.2 Codifica standard risposte

Il protocollo NNTP utilizza un sistema di codifica a tre cifre per gestire il flusso di controllo tra client e server. Mentre la prima cifra indica lo stato globale dell'operazione, la seconda definisce il contesto funzionale specifico del servizio news.

Codice	Significato (Prima Cifra - Stato)
1xx	Messaggio informativo.
2xx	Risposta positiva completa (azione conclusa con successo).
3xx	Risposta positiva intermedia (il server attende ulteriori dati).
4xx	Comando corretto ma non eseguibile (errore temporaneo).
5xx	Comando non implementato o errore permanente.

La seconda cifra categorizza l'area funzionale dell'interazione:

Codice	Significato (Seconda Cifra - Contesto)
x0x	Messaggi di connessione, setup e comunicazioni varie.
x1x	Selezione e gestione dei newsgroup .
x2x	Selezione e recupero degli articoli .
x3x	Funzioni di distribuzione e instradamento tra server.
x4x	Operazioni di posting (invio nuovi articoli).
x8x	Estensioni non standard o specifiche di implementazione.
x9x	Output di debugging e diagnostica.

Esempi pratici di interazione:

- **200:** (2=successo, 0=setup) "Service available, posting allowed". Indica che il server è pronto e accetta nuovi post.
- **411:** (4=errore eseguibile, 1=newsgroup) "No such newsgroup". Il comando è corretto sintatticamente, ma il gruppo richiesto non esiste sul server.
- **500:** (5=errore perm, 0=sintassi) "Command not recognized". Il server non comprende il comando inviato.

5.10 Servizi Sincroni vs Asincroni

L'analisi dei protocolli storici permette di distinguere due modalità operative fondamentali, ciascuna con compromessi specifici tra prestazioni, coerenza e costi.

5.10.1 Il modello USENET: Asincronicità e Distribuzione

USENET rappresenta l'esempio estremo di servizio asincrono e distribuito:

- **Dimensione Globale:** Gestisce moli enormi di dati distribuendoli su scala mondiale.
- **Meccanismo di Flooding:** La diffusione avviene per "inondazione" tra server, senza un controllo centralizzato.
- **Consistenza Debole:** Si accetta una minore consistenza dei dati (un messaggio può apparire su un server prima che su un altro) in cambio di un costo infrastrutturale più basso e una maggiore resilienza.

5.10.2 Evoluzione verso la Sicurezza Infrastrutturale

I servizi di "prima generazione" (nati in ambiti accademici e militari non ostili) non prevedevano nativamente meccanismi di sicurezza. L'evoluzione ha portato alla creazione di versioni protette che agiscono a livello infrastrutturale:

Servizio Originale (In chiaro)	Versione Sicura (Cifrata)	Miglioramento
Telnet	SSH (Secure Shell)	Cifratura di sessione e auth forte.
FTP	SFTP / FTPS	Protezione del canale dati e comandi.
SMTP / NNTP	TLS/SSL (Smtps, Nntps)	Incapsulamento in tunnel sicuri.

Cifratura Trasparente La tendenza moderna è lo spostamento della sicurezza dall'utente all'infrastruttura. La cifratura dei messaggi avviene "sotto il cofano" (senza l'intervento diretto degli utenti), garantendo che la riservatezza e l'integrità siano proprietà intrinseche del canale di comunicazione utilizzato dagli agenti Client/Server.

6 Sistemi di Chiamate Remote

Internet risponde alla filosofia **MAY-BE**. Per limitare i costi, il sistema si basa su un solo invio asincrono di ogni datagramma o informazione: il messaggio può arrivare o meno e non è previsto alcun meccanismo nativo di *recovery*.

- **Best-effort**: L'intero progetto di Internet segue questa filosofia. Ogni nodo della rete fa il massimo per consegnare il pacchetto, ma senza garanzie.
- **IP (Internet Protocol)**: Rappresenta il cuore di questa visione; ogni azione di instradamento è fatta una volta sola (*stateless*).
- **UDP (User Datagram Protocol)**: Estende questo concetto al trasporto end-to-end, fornendo un servizio minimo senza controllo di flusso o ritrasmissioni.

Lo standard sacrifica deliberatamente la **Qualità del Servizio (QoS)** in favore di una **applicabilità globale**. Questa visione permette di avere codice visibile, provabile e universalmente adottabile.

6.1 Semantiche

6.1.1 At-least-once

Internet deve anche fornire risposte di maggiore qualità, tenendo conto delle informazioni ricevute e delle ritrasmissioni successive allo scadere dei *timeout*. La semantica **At-least-once** prevede che il mittente effettui ritrasmissioni a intervalli regolari finché non riceve una conferma. Questa garanzia assicura che l'azione venga eseguita *almeno una volta*, ma presenta il limite intrinseco della duplicazione: in caso di ACK perso, il server potrebbe eseguire la stessa operazione più volte.

6.1.2 At-most-once

La semantica **At-most-once** richiede un intervento coordinato tra Cliente e Servitore per ottenere garanzie di correttezza, assicurando che il messaggio, se arriva, venga considerato **al più una volta**. Il controllo è delegato al protocollo e richiede che il server mantenga uno stato per un tempo predefinito tramite identificativi univoci, così da riconoscere i messaggi già ricevuti ed evitare di rieseguire azioni già completate. In questo modello, il cliente si occupa delle ritrasmissioni (gestite ad esempio da TCP) mentre il servitore garantisce l'univocità dell'esecuzione.

6.1.3 Exactly-once

A livello applicativo si richiede spesso la massima garanzia definita come **Exactly-once** o **Atomicità**. Mentre nelle semantiche precedenti un guasto può lasciare i peer in uno stato di incertezza dovuto a decisioni unilaterali, qui si ricerca il coordinamento totale secondo la logica del **Tutto o Niente**: il messaggio arriva esattamente una volta per entrambi, oppure l'operazione non viene considerata da nessuno dei due. Questa semantica richiede il massimo sforzo in termini di affidabilità. Al termine dell'operazione entrambi i pari sanno con certezza se l'azione è stata eseguita, richiedendo protocolli di accordo complessi (come il *Two-Phase Commit*) affinché la visione dello stato sia coerente e condivisa tra i nodi.

6.2 Semantica protocolli TCP/IP

La semantica rispettata dai protocolli dello stack TCP/IP varia in base al livello di affidabilità richiesto dal trasporto:

- **May-be** per IP e UDP: seguono la filosofia del *best-effort*, dove il pacchetto viene inviato una sola volta senza alcuna garanzia di ricezione o notifica di successo.
- **At-most-once** per il TCP: grazie all'uso dei numeri di sequenza, dei buffer e degli ACK, il protocollo garantisce che ogni segmento di dati venga consegnato e processato al massimo una volta, eliminando i duplicati.

In caso di insuccesso critico (ad esempio l'abbattimento della connessione), non è comunque presente una garanzia di accordo definitivo sullo stato della comunicazione tra i due peer. La scelta di queste semantiche permette di mantenere un equilibrio ottimale tra la durata delle operazioni, il carico computazionale e il costo complessivo dei protocolli, garantendo al contempo l'accessibilità e la scalabilità globale della rete.

6.3 Remote Procedure Call (RPC)

Le RPC rappresentano un'estensione del normale meccanismo di chiamata a procedura locale, portando il concetto di astrazione funzionale in un contesto distribuito. Si tratta di un approccio di livello 7 (Applicazione) nella pila ISO/OSI: il cliente invia la richiesta ed attende in maniera sincrona la risposta dal server, emulando il comportamento di una funzione tradizionale.

Tuttavia, rispetto ad una chiamata a procedura locale, le RPC introducono differenze sostanziali:

- **Processi distinti:** L'esecuzione avviene su entità computazionali diverse.
- **Ciclo di vita separato:** Cliente e server hanno vita propria e possono fallire indipendentemente.
- **Assenza di memoria condivisa:** I processi non condividono lo spazio di indirizzamento; non è possibile passare puntatori o riferimenti diretti alla memoria.
- **Affidabilità del nodo:** Possono verificarsi malfunzionamenti parziali sui singoli nodi che non compromettono l'intera rete, ma interrompono la specifica chiamata.

Le realizzazioni delle chiamate remote rendono il server e i client specializzati per il contesto distribuito. Di conseguenza, queste funzioni non possono più essere utilizzate nel normale meccanismo di chiamata locale, perdendo la compatibilità con il codice non remotizzato. Viene quindi superato definitivamente il vincolo di località: le funzioni progettate per essere remotizzate sono vincolate a questa architettura e non possono essere richiamate in modo locale all'interno dello stesso processo.

6.3.1 Proprietà

Le RPC permettono il controllo dinamico del tipo dei parametri e del risultato, garantendo l'integrità dei dati scambiati tra macchine potenzialmente diverse. Fondamentale è il trattamento dei dati in ingresso e in uscita attraverso il **marshalling** (o serializzazione), ovvero il processo di trasformazione dei dati in un formato adatto alla trasmissione su rete.

Le proprietà principali includono:

- **Trasparenza:** L'obiettivo è rendere la chiamata remota il più simile possibile a una chiamata locale per il programmatore.
- **Type checking e parametrizzazione:** Verifica della coerenza dei tipi tra client e server durante la chiamata.
- **Controllo concorrenza:** Gestione di molteplici richieste simultanee da parte del server.
- **Binding distribuito:** Il meccanismo con cui un client individua e si connette al server che offre il servizio desiderato.
- **Trattamento orfani:** Gestione delle computazioni sul server che continuano a girare nonostante il client sia crashato o la connessione sia caduta.

6.4 Storia

La prima visione sistemica del modello RPC è dovuta a **Birrel e Nelson nel 1984**, con i lavori pionieristici presso Xerox PARC. Successivamente, lo sviluppo è proseguito attraverso standard di aziende come Sun (ONC RPC), HP (DCE RPC) e Apollo.

Un momento chiave è stato il progetto **Athena** (MIT, ITC, CMU), dove l'utilizzo delle RPC è diventato fondamentale per scambiare messaggi coordinati in una rete di workstation distribuite, gettando le basi per i moderni sistemi operativi di rete.

6.5 Primitive

Ogni sistema RPC implementa primitive differenti senza seguire uno standard universale rigoroso. A livello di API, le operazioni fondamentali si articolano attorno alla struttura della **call** di servizio, che specifica il servitore, gli argomenti e il risultato atteso.

Dal punto di vista del servitore, le operazioni si basano tipicamente su due primitive principali:

- **GetRequest:** Il servitore si mette in attesa di una richiesta in arrivo.
- **SendReply:** Il servitore invia il risultato dell'elaborazione al cliente.

L'architettura del server può adottare due diverse politiche di accettazione e risposta, che determinano il grado di concorrenza del sistema:

1. **Approccio Esplicito e Sequenziale:** Il servizio è svolto da un unico processo che gestisce una coda di richieste. È il processo stesso a decidere quando e se eseguire i metodi, processandoli uno alla volta in ordine.
2. **Approccio Implicito e Parallelo:** Il sistema genera automaticamente un processo (o un thread) indipendente per ogni richiesta in arrivo. Questo permette l'esecuzione contemporanea di più servizi, migliorando le prestazioni in sistemi multicore ma aumentando la complessità della gestione delle risorse.

6.6 Tolleranza ai guasti

L'obiettivo applicativo primario è mascherare i malfunzionamenti che possono colpire la comunicazione o i nodi, come la perdita di messaggi (richiesta o risposta) e il crash del cliente o del servitore. In caso di crash del servitore prima della risposta, o di perdita del messaggio, il cliente può adottare diverse politiche:

- **Attesa indefinita:** Il cliente resta bloccato in attesa di un evento che potrebbe non verificarsi mai.
- **Timeout e ritrasmissione:** Si tenta nuovamente l'invio utilizzando identificatori unici per permettere al server di distinguere i duplicati.
- **Timeout ed eccezione:** Il fallimento viene riportato all'applicazione cliente per una gestione di alto livello.

Molte strategie si basano sull'assunzione che le operazioni siano **idempotenti**, ovvero che possano essere eseguite un numero qualunque di volte producendo lo stesso esito finale per il sistema.

Strategie e Semantiche Le RPC adottano semantiche specifiche a seconda del grado di affidabilità richiesto:

- **May-be:** Gestione tramite semplice timeout lato cliente.
- **At-least-once:** Timeout e ritrasmissioni (richiede idempotenza).
- **At-most-once:** Utilizzo di tabelle delle azioni effettuate per evitare riesecuzioni.
- **Exactly-once:** L'azione viene portata a termine garantendo l'accordo sullo stato finale.

Trattamento degli Orfani In caso di crash del cliente, sul nodo servitore possono rimanere dei processi "orfani" in attesa di consegnare un risultato a un destinatario non più esistente. Le politiche tipiche per la loro gestione sono:

- **Sterminio:** Ogni processo orfano derivante da un crash rilevato viene immediatamente distrutto.
- **Terminazione a tempo:** Ogni calcolo ha una scadenza predefinita oltre la quale viene abortito automaticamente.
- **Reincarnazione (ad epoche):** Il tempo è diviso in "epoche" numerate. Al riavvio di un nodo, inizia una nuova epoca e tutti i calcoli relativi a epoche precedenti sono considerati obsoleti e terminati.

6.7 Open Network Computing (ONC)

Proposto da Sun Microsystems, questo modello introduce la primitiva `callrpc`. A differenza di una chiamata locale, richiede parametri aggiuntivi oltre a quelli logici: il nome del nodo remoto, l'identificatore della procedura e le specifiche di trasformazione degli argomenti tramite **XDR** (External Data Representation). A causa della necessità di gestire esplicitamente questi parametri, uno schema basato esclusivamente su modello ONC **non è trasparente** per lo sviluppatore.

6.8 Network Computing Architecture (NCA)

Per ovviare alla mancanza di trasparenza, l'architettura NCA introduce ai due endpoint di comunicazione delle routine chiamate **stub**. Grazie a queste, la chiamata diventa del tutto locale dal punto di vista del programmatore, garantendo la **trasparenza** sintattica.

Il ruolo degli Stub Gli stub sono componenti generati automaticamente dall'implementazione del sistema RPC. Il loro compito è nascondere la complessità della rete:

- Le parti logiche del programma rimangono inalterate.
- Il flusso dell'applicazione si dirige verso lo stub locale invece che verso la rete.
- Lo stub nasconde tutte le operazioni di marshalling e comunicazione.

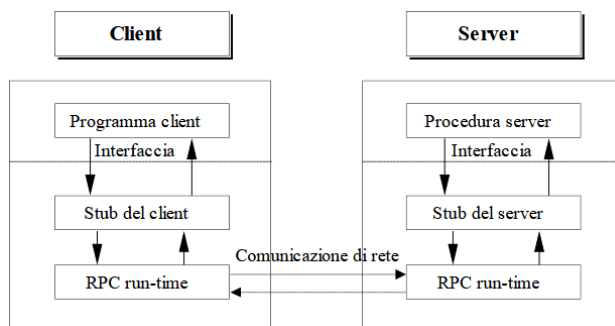
Costi e Ottimizzazioni L'astrazione ha un costo prestazionale, poiché ogni RPC richiede effettivamente **due chiamate** (una alla funzione stub e una remota). Sistemi come **Mercury** cercano di mitigare questo overhead ottimizzando lo scambio di messaggi attraverso l'uso di stream di chiamate.

6.9 Modello con Stub

In un modello cliente/servitore asimmetrico basato su stub, l'interazione viene mediata da componenti specializzati che agiscono per conto delle parti logiche del programma. Il **client** invoca localmente uno stub (client-stub) che si incarica di gestire l'intera transazione di rete, rendendo la chiamata indistinguibile da una procedura locale.

Il **servitore** non riceve i dati grezzi direttamente dalla rete, ma interagisce con uno stub lato server (spesso chiamato *skeleton*), il quale si incarica del trattamento dei parametri, effettuando l'unmarshalling per riportarli nel formato nativo del servitore.

Al completamento del servizio. Per garantire la coerenza e ridurre gli errori di sviluppo, lo stub dovrebbe essere il prodotto di un **processo automatico** basato sulla definizione dell'interfaccia (IDL), assicurando che la firma della funzione sia identica su entrambi i fronti.



6.10 Passaggio di parametri

Nelle chiamate a procedura locale, i parametri possono essere passati tipicamente per valore o per riferimento. Tuttavia, data l'assenza di memoria condivisa nei sistemi distribuiti, le chiamate RPC si basano principalmente sul passaggio **per valore** tramite operazioni di marshalling e unmarshalling.

Il trattamento dei parametri tra cliente e server segue logiche differenti a seconda della tipologia:

- **Passaggio per valore:** Avviene tramite trasferimento e visita del dato. Il valore copiato sul server è indipendente dall'originale; eventuali modifiche rimangono locali al server e il valore viene perso al termine del servizio se non esplicitamente restituito.
- **Passaggio per riferimento:** Non potendo trasferire l'indirizzo di memoria fisica, si rende l'oggetto **remoto**. L'oggetto rimane nel nodo di partenza e viene identificato in modo unico nell'intero sistema.

Se un server deve riferire un'entità residente sul cliente, riceve un **riferimento remoto**. Ogni operazione successiva su tale entità da parte dei nodi remoti dovrà essere effettuata attraverso ulteriori chiamate RPC verso il nodo proprietario.

6.11 Eccezioni

Le RPC integrano meccanismi per la trattazione delle eccezioni, permettendo di gestire eventi anomali intrinseci alla distribuzione (come guasti di rete o crash dei nodi). In genere, il programmatore specifica un'azione dedicata per il trattamento dell'anomalia.

Nei sistemi più evoluti, è possibile inserire la gestione delle eccezioni direttamente nello **scope del linguaggio** di programmazione, permettendo di catturare un errore di rete (es. *RemoteException*) con la stessa sintassi utilizzata per gli errori logici locali (es. *try-catch*).

6.12 Interface Definition Language (IDL)

Le IDL sono linguaggi specializzati per la descrizione delle operazioni remote, utili a definire la firma dei metodi e permettere la generazione automatica degli stub. Poiché cliente e server possono essere scritti in linguaggi diversi e girare su architetture differenti, l'IDL funge da contratto neutrale.

In particolare, le IDL si occupano di:

- **Identificazione unica del servizio:** Ogni interfaccia e ogni procedura devono essere associate a un identificativo univoco per evitare conflitti nel binding.
- **Definizione astratta dei dati:** Forniscono una sintassi per descrivere i tipi di dati (interi, stringhe, strutture) in modo indipendente dal linguaggio di programmazione, permettendo al compilatore IDL di generare il corretto codice di marshalling per ogni piattaforma.

6.12.1 Linguaggi più comuni

Esistono diversi linguaggi di definizione delle interfacce, sviluppati nel tempo dai vari vendor. Tra i più comuni si annoverano **XDR** (Sun/ONC), **DCE IDL** (OSF), **ANSAware**, **NCS IDL** e il generico **IDL** di CORBA. Nonostante i numerosi tentativi di unificazione, **non è stato definito uno standard** unico universale, portando a una frammentazione basata sulle diverse architetture distribuite.

6.12.2 eXternal Data Representation (XDR)

XDR è lo standard utilizzato in ambito ONC RPC per descrivere i dati e le procedure in modo indipendente dall'architettura. Di seguito un esempio di definizione:

```
program MESSAGEPROG {
    version MESSAGEVERS {
        int PRINTMESSAGE(string) = 1;
    } = 1;
} = 0x20000013;
```

In questo frammento, il programma è identificato dal numero esadecimale **0x20000013**, definisce una versione specifica (1) e una procedura (**PRINTMESSAGE**) che accetta una stringa e restituisce un intero.

6.12.3 Da IDL a Stub

Le IDL hanno lo scopo fondamentale di definire la struttura degli stub. Per facilitare lo sviluppo, i sistemi RPC mettono a disposizione dei compilatori specifici che automatizzano la generazione del codice.

Uno strumento emblematico è **RPCGEN**: partendo da un file scritto in XDR, esso produce automaticamente gli stub necessari sia per il client che per il server, oltre ai file header e alle routine di serializzazione dei dati. Questo approccio garantisce che le due estremità della comunicazione siano sempre perfettamente allineate sulla firma delle procedure e sul formato dei messaggi.

6.13 Sviluppo e supporto RPC

Lo sviluppo di un'applicazione basata su RPC richiede il coordinamento di diverse fasi tecniche e il supporto di componenti infrastrutturali. Le fasi tipiche sono:

- **Compilazione di sorgenti e stub:** Il processo inizia con la compilazione del file IDL per generare gli stub e lo skeleton, che vengono poi linkati rispettivamente al codice sorgente del client e del server.
- **Binding entità:** Fase cruciale in cui il client individua la posizione del server (indirizzo IP e porta). Spesso avviene tramite un *Name Server* o un *Portmapper* che mette in contatto i due peer.
- **Trasporto dati:** Scelta e gestione del protocollo di trasporto sottostante (solitamente TCP per l'affidabilità o UDP per la velocità), delegata allo stub.
- **Controllo concorrenza:** Gestione delle richieste multiple sul server. Il supporto RPC deve decidere se serializzare le chiamate o istanziare nuovi thread/processi per ogni richiesta (modello implicito vs esplicito).
- **Supporto alla rappresentazione dei dati:** Utilizzo di standard come XDR o NDR per garantire che la semantica dei dati (formato dei numeri, ordine dei byte) sia preservata tra architetture eterogenee.

6.13.1 Binding

Il binding delle entità, ovvero il processo di associazione tra il chiamante e l'esecutore del servizio, può essere di due tipi:

- **Pessimistico e statico:** L'indirizzo e la porta del server sono cablati nel codice del client o configurati rigidamente al momento della compilazione. È poco flessibile e vulnerabile ai cambiamenti della rete.
- **Ottimistico e dinamico:** Il collegamento avviene a tempo di esecuzione, permettendo al sistema di adattarsi a server che cambiano indirizzo o ripartono su porte diverse. Si articola in:
 - **Servizio (fase statica):** Il client conosce a priori il nome del servizio o l'interfaccia che desidera invocare.
 - **Indirizzamento (fase dinamica):** L'individuazione del nodo e della porta avviene a runtime. Può essere **esplicita** (il client interroga un Name Server) o **implicita** (il supporto RPC risolve l'indirizzo automaticamente al primo invio).

In caso di binding dinamico, la prima chiamata richiede un passaggio aggiuntivo di collegamento. Per ottimizzare le prestazioni, spesso dopo aver stabilito il primo legame, il client memorizza le informazioni ottenute (caching) e utilizza lo stesso binding per le chiamate successive, comportandosi come se fosse di tipo statico per tutta la durata della sessione.

6.14 Sistema di nomi

Le RPC hanno introdotto la necessità di sistemi di nomi specializzati (come **Binder**, **Broker** o **Name Server**) per supportare il binding dinamico. Un binder funge da intermediario e deve fornire operazioni fondamentali per permettere agganci flessibili tra le parti:

- **Register:** Il servitore comunica la propria presenza e i servizi offerti al binder.
- **Unregister:** Il servitore notifica la propria disattivazione per essere rimosso dall'elenco.
- **Lookup:** Il client interroga il binder per ottenere l'indirizzo del servitore che espone una determinata interfaccia.

Il nome del servitore può essere dipendente dal nodo di residenza; in tal caso, il sistema di nomi deve essere in grado di tracciare e gestire ogni variazione di indirizzamento.

Evoluzione del servizio di Binding Per evitare colli di bottiglia e punti di fallimento singoli, il binding viene spesso attuato come un servizio coordinato tra più entità:

- **Binder multipli:** Vengono distribuiti sulla rete per limitare l'overhead di comunicazione.
- **Caching:** I singoli client o i nodi memorizzano i risultati delle lookup per velocizzare le chiamate successive.
- **Localizzazione iniziale:** I client possono utilizzare messaggi in **broadcast** per individuare il binder più vicino o conveniente all'interno del proprio segmento di rete.

6.15 Controllo e asincronicità

La necessità di ottenere un maggiore parallelismo e di gestire processi leggeri sui server ha portato alla nascita di modelli RPC non bloccanti. In questo scenario, il cliente non attende necessariamente il completamento dell'operazione per proseguire, ottimizzando l'uso delle risorse. Si distinguono due obiettivi principali:

- **RPC a bassa latenza:** Mira a minimizzare il tempo di risposta per singole operazioni critiche.
- **RPC a throughput elevato:** Mira a massimizzare il numero di operazioni completate nell'unità di tempo attraverso il parallelismo.

6.15.1 RPC asincrone

Dal punto di vista implementativo, queste varianti si appoggiano sia su protocolli UDP che TCP. Di seguito le implementazioni storiche più significative:

- **Athena (XWindows):** Basata su UDP, focalizzata sulla bassa latenza. Utilizza una semantica **may-be** e spesso non prevede un risultato di ritorno (fire-and-forget).
- **SUN:** Utilizza TCP per garantire un elevato throughput con semantica **at-most-once**. Permette l'invio di molteplici RPC asincrone seguite da una chiamata di **batching** per sincronizzare i flussi. Anche in questo caso, le singole chiamate asincrone sono tipicamente senza risultato immediato.
- **Chronus:** Progettata per bassa latenza, ma a differenza di Athena, supporta la restituzione di un risultato.
- **Mercury:** Ottimizzata per l'alto throughput, supporta la gestione dei risultati, spesso tramite meccanismi di *promise* o *callback*.

6.16 RPC di SUN

L'implementazione RPC di SUN si basa su processi server tipicamente **sequenziali** e non offre una trasparenza completa, richiedendo al programmatore di gestire esplicitamente alcuni parametri di rete.

Le caratteristiche principali includono:

- **Entità richiedibili:** Singole operazioni o funzioni procedurali.
- **Semantica:** Supporta *at-most-once* o *at-least-once* a seconda della configurazione.

- **Comunicazione:** Gestisce sia modalità sincrone che asincrone.
- **Localizzazione:** La ricerca del servitore avviene tramite il **port mapper** residente sul nodo server.
- **Dati e Parametri:** La presentazione dei dati è affidata a XDR (IDL) con generazione degli stub tramite **rpcgen**. Il passaggio dei parametri avviene esclusivamente **per valore**.

6.17 RMI in JAVA

Remote Method Invocation (RMI) rappresenta l'evoluzione del concetto RPC per l'ecosistema Java. È progettato per oggetti passivi con un'alta trasparenza alla locazione, pur rinunciando all'eterogeneità (richiede Java su entrambi i fronti) e privilegiando il parallelismo rispetto all'efficienza pura.

- **Entità richiedibili:** Metodi di oggetti definiti tramite interfacce remote.
- **Semantica e Trasporto:** Utilizza TCP con semantica **at-most-once**.
- **Sincronia:** Le chiamate sono esclusivamente **sincrone**.
- **Localizzazione:** Utilizza il **RMRegistry** come broker. Sebbene di base sia un registro unico, è possibile implementare organizzazioni più complesse.
- **Server Parallelo:** Il server gestisce le richieste in modo concorrente tramite l'uso di **thread**.

A differenza di Sun RPC, poi, Java RMI offre una gestione avanzata dei dati:

- **Default per valore:** Gli oggetti comuni vengono serializzati e copiati.
- **Per riferimento:** Gli oggetti che implementano interfacce *remotizzabili* vengono passati per riferimento remoto.
- **Integrazione di sistema:** RMI è strettamente integrato con il sistema di sicurezza di Java e con il **Distributed Garbage Collector** (DGC) per la gestione automatica della memoria degli oggetti remoti non più utilizzati.

7 JAVA RMI

L'architettura RMI permette l'invocazione di metodi remoti in Java, adattando il concetto di RPC (*Remote Procedure Call*) al paradigma Object-Oriented (OO).

RMI fornisce l'infrastruttura per consentire a un'applicazione Java di invocare metodi su oggetti residenti in una **JVM (Java Virtual Machine) differente**, potenzialmente su un host remoto.

- **Riferimento Remoto:** Localmente viene creato solo un riferimento all'oggetto remoto. L'oggetto reale è istanziato e attivo sul nodo server.
- **Trasparenza:** Il programma client invoca i metodi attraverso questo riferimento locale (una variabile di tipo interfaccia), come se l'oggetto fosse locale.
- **Eterogeneità e Portabilità:** Grazie al **Bytecode** e alla JVM, RMI garantisce l'interoperabilità tra sistemi operativi diversi (eterogeneità dei sistemi), mantenendo un ambiente di sviluppo omogeneo (solo Java).

7.1 Accesso ad oggetti remoti

Poiché Java non gestisce nativamente i puntatori a memoria remota, RMI introduce una struttura a livelli basata sul **Pattern Proxy** per simulare la presenza locale dell'oggetto.

7.1.1 Stub e Skeleton

L'accesso è mediato da due entità fondamentali che nascondono la complessità della rete:

- **Stub (lato Cliente):** Agisce come rappresentante locale dell'oggetto remoto. Riceve l'invocazione del metodo, effettua il *marshalling* dei parametri (impacchettamento) e li invia sulla rete.
- **Skeleton (lato Server):** Riceve la richiesta dalla rete, effettua l'unmarshalling dei parametri e invoca effettivamente il metodo sull'oggetto reale (server).

7.1.2 Differenze rispetto all'invocazione locale

Nonostante la trasparenza sintattica, un'invocazione RMI differisce da una locale per tre aspetti critici:

1. **Affidabilità:** La rete può fallire o essere lenta; è necessario gestire eccezioni specifiche (**RemoteException**).
2. **Semantica:** Il passaggio dei parametri può avvenire per valore (copia) per gli oggetti non remoti, o per riferimento per gli oggetti remoti.
3. **Durata:** La vita dell'oggetto remoto non è legata direttamente allo scope del client, richiedendo meccanismi di *Distributed Garbage Collection*.

7.1.3 Variabili Interfaccia e Disaccoppiamento

Il sistema RMI si fonda sull'uso intensivo delle **variabili interfaccia** per garantire che il client interagisca solo con la definizione dei metodi e mai con l'implementazione concreta dell'oggetto remoto.

Definizione e Polimorfismo Una variabile interfaccia è un riferimento che può puntare dinamicamente a qualsiasi istanza di una classe che implementi tale interfaccia. In RMI, questo meccanismo viene sfruttato per gestire il proxy:

- **Contratto Formale:** L'interfaccia remota (che deve estendere `java.rmi.Remote`) elenca i metodi che il server mette a disposizione.
- **Contenitore del Proxy:** La variabile interfaccia nel codice client contiene, a runtime, un riferimento allo **Stub**.

- **Trasparenza:** Poiché lo Stub implementa la stessa interfaccia dell'oggetto remoto, il programmatore scrive codice identico a quello per un oggetto locale:

```

1      NomeInterfaccia obj = (NomeInterfaccia) Naming.lookup(...);
2      obj.metodoRemoto();

```

Il passaggio tra ambienti L'interfaccia funge da "ponte" logico. La variabile interfaccia permette di:

1. **Controllare l'invocazione:** Verificare a tempo di compilazione che i metodi chiamati esistano effettivamente.
2. **Preparare l'ambiente:** Innescare le routine dello Stub che si occupano di gestire la rete, la serializzazione e la comunicazione con il servitore, nascondendo completamente il passaggio da un indirizzamento locale a uno remoto.

7.2 Architettura

L'architettura RMI è organizzata in livelli sovrapposti, tutti operanti all'interno della JVM, che collaborano per rendere l'invocazione remota trasparente.

- **Stub (Client-side):** Il proxy che espone l'interfaccia dell'oggetto.
- **Skeleton (Server-side):** Il dispatcher che riceve le chiamate e le inoltra all'oggetto reale.

7.2.1 Remote Reference Layer (RRL)

Il livello **RRL** funge da intermediario tra il livello dei proxy e il trasporto. È responsabile della gestione della semantica dell'invocazione:

- Gestisce la comunicazione tra lo Stub e lo Skeleton.
- Implementa diverse strategie di riferimento (es. unicast punto-punto o multicast per oggetti replicati).
- Gestisce il ciclo di vita dei riferimenti remoti.

7.2.2 Transport Layer (TL)

Il livello di **Trasporto** è il punto di contatto con la rete fisica ed è responsabile della gestione delle connessioni tra diversi spazi di indirizzamento (JVM diverse). Si occupa di:

- **Connettività:** Gestisce l'apertura e la chiusura dei socket tra host.
- **Protocolli:** Sebbene utilizzi **TCP** come protocollo di trasporto affidabile (*connection-oriented*), a livello applicativo sfrutta un protocollo proprietario denominato **JRMP** (Java Remote Method Protocol).
- **Integrazione JVM:** È strettamente integrato con i meccanismi di attivazione della JVM per "risvegliare" oggetti remoti non ancora attivi.

7.2.3 Il Registro RMI (Registry)

Fondamentale per il bootstrapping del sistema, il **Registry** è un servizio di nomi (directory) che opera sulla porta standard **1099**:

- **Server side (bind):** Il server pubblica un riferimento al proprio oggetto remoto associandolo a un nome mnemonico (es. "MyService").
- **Client side (lookup):** Il client contatta il registro chiedendo l'oggetto per nome e riceve in risposta lo **Stub** necessario per iniziare la comunicazione diretta con il server.

7.3 Caratteristiche Fondamentali

RMI si distingue per la sua capacità di proiettare il modello ad oggetti di Java su un'architettura distribuita, mantenendo la coerenza del linguaggio ma adattandosi ai vincoli della rete.

- **Modello ad Oggetti Distribuito:** Un oggetto distribuito è un'entità i cui metodi possono essere invocati da una JVM differente da quella in cui risiede l'oggetto stesso.
 - L'oggetto remoto è definito e descritto obbligatoriamente tramite **interfacce remote** (che estendono `java.rmi.Remote`).
 - Solo i metodi dichiarati nell'interfaccia sono accessibili remotamente.
- **Chiamata Locale vs. Chiamata Remota:** Sebbene il client utilizzi un riferimento remoto (tramite variabile interfaccia) con una **sintassi identica** a quella locale (garantendo trasparenza sintattica), la natura dell'interazione è profondamente diversa:
 - **Trasparenza per il Client:** Il codice non cambia, ma ogni chiamata può lanciare una `RemoteException`.
 - **Affidabilità:** A differenza delle chiamate locali (sempre affidabili), quelle remote possono fallire parzialmente.
 - **Semantica "At-most-once":** RMI utilizza TCP per garantire che, in caso di successo, l'operazione sia stata eseguita esattamente una volta; in caso di errore, il sistema garantisce che sia stata eseguita *al massimo* una volta, evitando duplicazioni.
 - **Server Parallelo:** Mentre una chiamata locale è sequenziale nel thread chiamante, il server RMI gestisce le richieste in modo parallelo (multi-threaded), richiedendo attenzione alla sincronizzazione dei dati.

7.4 Interfacce e Implementazioni

Il design di un'applicazione RMI si basa sul principio della separazione tra contratto e implementazione. Questo garantisce che il client conosca solo "cosa" l'oggetto può fare, senza dipendere da "come" lo faccia.

7.4.1 Interfaccia

L'interfaccia remota funge da specifica del servizio. Per essere valida nel framework RMI, deve rispettare due vincoli sintattici fondamentali:

- **Estensione di Remote:** L'interfaccia deve estendere `java.rmi.Remote`. Questo è un "marker interface" che segnala alla JVM che l'oggetto è accessibile da remoto.
- **Gestione Eccezioni:** Ogni metodo dichiarato nell'interfaccia deve includere nella propria firma la clausola `throws java.rmi.RemoteException`. Questo obbliga il client a gestire i potenziali fallimenti di rete.

7.4.2 Classe Server

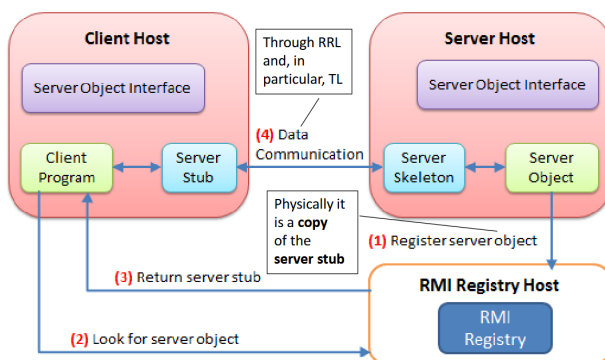
La classe che realizza logicamente il servizio deve integrare le funzionalità di rete fornite dal framework:

- **Implementazione dell'interfaccia:** Deve fornire il corpo di tutti i metodi dichiarati nell'interfaccia remota definita al punto precedente.
- **Estensione di `UnicastRemoteObject`:** Solitamente la classe estende `java.rmi.server.UnicastRemoteObject`. Questo garantisce che:
 - L'oggetto venga "esportato" automaticamente (ovvero reso in ascolto su una porta TCP) al momento della creazione dell'istanza.
 - Vengano ereditati i metodi corretti per la gestione dell'identità remota (`equals`, `hashCode`, `toString`).

7.5 Utilizzo

Il ciclo di vita di un'applicazione RMI prevede una sequenza di passi coordinati tra ambiente di sviluppo e ambiente di esecuzione:

1. **Definizione dell'Interfaccia e dell'Implementazione:** Si definisce l'interfaccia remota (estendendo `Remote`) e la classe che ne fornisce l'implementazione concreta (estendendo `UnicastRemoteObject`).
2. **Compilazione e Generazione degli Artefatti:**
 - Si compila il codice sorgente con `javac`.
 - *Nota storica:* In passato era necessario il comando `rmic` per generare esplicitamente **stub** e **skeleton**. Nelle versioni moderne di Java (dalla 5.0+), questi componenti vengono generati dinamicamente a runtime.
3. **Pubblicazione sul Registry (Naming Service):**
 - Si avvia il processo `rmiregistry` sul server (solitamente sulla porta 1099).
 - Il server istanzia l'oggetto remoto e lo registra tramite il metodo `Naming.rebind("nomeServizio", oggetto)`, associando un nome logico al riferimento fisico.
4. **Accesso e Compilazione del Client:**
 - Il client viene compilato avendo a disposizione l'interfaccia remota (il contratto).
 - A runtime, il client interroga il registry tramite `Naming.lookup("rmi://host/nomeServizio")`.
 - Il Registry restituisce lo **stub**, che il client casta nella variabile interfaccia per iniziare a invocare i metodi.



7.6 RMI Registry

Perché un'interazione possa avvenire, il client deve innanzitutto localizzare il server. Esistono diverse strategie di localizzazione:

- **Hardcoding:** Il client conosce l'indirizzo staticamente (poco flessibile).
- **Configurazione utente:** L'indirizzo viene passato come parametro all'avvio.
- **Indirizione tramite Naming Service:** Un servizio terzo e noto fornisce la posizione del server.

Java RMI adotta la terza soluzione tramite il **RMI Registry**, una directory che mappa nomi mnemonici a riferimenti remoti (Stub). Il registro mantiene coppie `{name, reference}` e per nome utilizzato (`//host:porta/servizio`) la locazione è esplicitata. Pertanto, RMI offre trasparenza nell'invocazione ma non trasparenza alla locazione nel momento del reperimento iniziale.

7.6.1 Classe Naming e Attivazione Registry

Il Registry deve essere avviato sull'host del server come processo separato.

- **Comando:** `rmiregistry [porta]` (default 1099).
- **Isolamento:** Esegue in una istanza della JVM distinta da quella dell'applicazione server.

L'interazione con il registro avviene tramite la classe `java.rmi.Naming`, che espone i seguenti metodi statici:

```
1 public static void bind(String name, Remote obj)
2     throws AlreadyBoundException, MalformedURLException, RemoteException;
3
4 public static void rebind(String name, Remote obj)
5     throws RemoteException, MalformedURLException;
6
7 public static void unbind(String name)
8     throws RemoteException, NotBoundException, MalformedURLException;
9
10 public static String[] list(String name)
11     throws RemoteException, MalformedURLException;
12
13 public static Remote lookup(String name)
14     throws NotBoundException, MalformedURLException, RemoteException;
```

Il parametro `name` (`//registryHost:port/service_name`) combina la posizione del registro e il nome del servizio:

- **registryHost:** Host dove risiede il registro e l'oggetto remoto.
- **port:** Default 1099.
- **service_name:** Identificativo logico scelto dal programmatore.

7.7 Stub e Skeleton

Il processo di invocazione remota trasforma una chiamata a un metodo locale in un messaggio di rete e viceversa. Questo flusso garantisce che la complessità della comunicazione rimanga nascosta all'applicazione. L'iterazione segue i passi descritti di seguito:

1. **Ottenimento del Riferimento:** Il client ottiene il riferimento remoto (lo **Stub**) tramite il servizio di naming (Registry).
2. **Fase Client (Stub):** Il client chiama il metodo sullo stub come se fosse l'oggetto reale. Lo stub esegue:
 - **Serializzazione (Marshalling):** Converte i parametri della chiamata in un formato adatto alla trasmissione binaria.
 - **Inoltro:** Invia le informazioni allo skeleton del server utilizzando i servizi del livello **RRL** (*Remote Reference Layer*).
3. **Fase Server (Skeleton):** La chiamata viene ricevuta dallo skeleton che esegue:
 - **Deserializzazione (Unmarshalling):** Ricostruisce i parametri dai dati binari ricevuti.
 - **Invocazione:** Chiama il metodo effettivo sull'oggetto servitore reale.
 - **Serializzazione del Ritorno:** Ricevuto l'esito (o l'eccezione), serializza il valore di ritorno per rispedirlo al client.
4. **Fase Finale (Stub):** Lo stub riceve la risposta dal server:
 - **De-serializzazione:** Ricostruisce il valore di ritorno e lo consegna al thread del client che era rimasto in attesa.

7.8 Deployment

In un'applicazione RMI è essenziale che i file `.class` siano disponibili a runtime nelle locazioni che lo richiedono per l'esecuzione o per i processi di serializzazione e deserializzazione.

Il **Server** deve poter accedere a:

- **Interfacce** che definiscono il servizio → a tempo di compilazione.
- **Implementazione** del servizio → a tempo di compilazione.
- **Stub e skeleton** delle classi di implementazione → a tempo di esecuzione.
- **Altre eventuali classi** utilizzate dal server → a tempo di compilazione o esecuzione.

Il **Client**, invece, deve poter accedere a:

- **Interfacce** che definiscono il servizio → a tempo di compilazione.
- **Stub** delle classi di implementazione del servizio → a tempo di esecuzione.
- **Classi del server** usate dal client (es. valori di ritorno) → a tempo di compilazione o esecuzione.
- **Altre classi** utilizzate dal client → a tempo di compilazione o esecuzione.

7.9 Serializzazione

Grazie all'uniformità del **bytecode**, Java non necessita di un livello di traduzione esterna (come l'XDR per RPC). Il sistema si affida alla **Serializzazione**, il processo di trasformazione di un oggetto (o di un intero grafo di oggetti collegati) in una sequenza di byte, preservandone lo stato interno.

- **Serializzazione:** L'oggetto viene trasformato in un flusso lineare. In Java, questo avviene tramite il metodo `writeObject()` della classe `ObjectOutputStream`.
- **Deserializzazione:** Il processo inverso, che ricostruisce una copia esatta dell'oggetto originale a partire dal flusso di byte. Si utilizza il metodo `readObject()` della classe `ObjectInputStream`.

Tale meccanismo viene utilizzato, per esempio, da **Stub** e **Skeleton** per trasportare i parametri. Viene poi anche utilizzato per il trasporto di informazioni sul disco, permettendo di salvare lo stato degli oggetti e ripristinarlo in momenti successivi.

7.9.1 Interazione con stream

Un qualunque oggetto **Serializable** può essere scritto su uno stream di output, ma è importante notare che non è l'istanza fisica originaria a viaggiare, bensì una sua rappresentazione sotto forma di dati.

- **Cosa viene scritto:** Non viene trasferito l'intero grafo dell'oggetto inteso come locazione di memoria, ma solamente le **informazioni e lo stato** contenuti nei suoi campi non statici e non transienti.
- **Cosa viene ricostruito:** Al momento della deserializzazione, il ricevente non ottiene l'oggetto originale, ma viene ricreata una **nuova copia dell'istanza** "trasmessa".
- **Requisito della Classe Locale:** Il processo di ricostruzione avviene utilizzando la **classe locale ricevente**. Se la JVM che riceve lo stream non possiede il file `.class` corrispondente all'oggetto serializzato, la deserializzazione fallirà sollevando una `ClassNotFoundException`.

Questo meccanismo evidenzia come la serializzazione in RMI sia un passaggio **per valore** (copia): modifiche apportate all'oggetto dal ricevente non avranno alcun effetto sull'oggetto originale rimasto presso il mittente.

7.9.2 Trasferimento remoto

Il trasferimento dei parametri durante un'invocazione RMI segue regole differenti in base alla natura del dato, per bilanciare efficienza e coerenza:

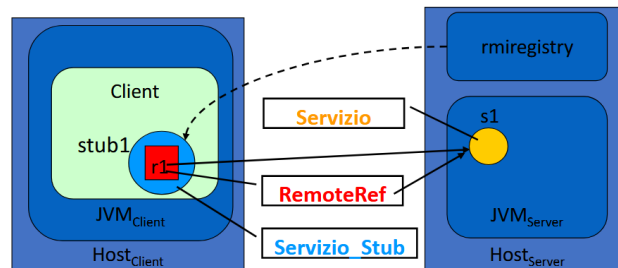
- **Tipi Primitivi:** Vengono passati per **valore** (copia dei bit).

- **Oggetti standard:** Vengono passati per **valore**. L'oggetto deve implementare `Serializable` o viene effettuata una **deep copy**.
- **Oggetti Remoti:** Vengono passati per **riferimento remoto**. Se un oggetto implementa l'interfaccia `Remote`, non viene inviato l'oggetto stesso, ma il suo **stub**.

7.10 Stub e riferimenti remoti

Affinché la comunicazione avvenga correttamente, lo Stub deve possedere le informazioni necessarie per indirizzare la chiamata all'istanza specifica sul server.

- **Identificativo Univoco (ObjID):** Ogni istanza di stub contiene un **ObjID**, un identificativo univoco all'interno della JVM del server che permette di distinguere l'oggetto remoto target tra tutti quelli attivi.
- **RemoteRef:** Lo stub contiene al suo interno un oggetto di tipo **RemoteRef** (riferimento remoto). Questo componente funge da interfaccia verso il livello **RRL** (Remote Reference Layer) e racchiude i dettagli del protocollo di trasporto e l'indirizzo dell'host.



7.11 Implementazione del Registry

Il Registry non è un'entità magica, ma è esso stesso un server **RMI** (nello specifico, un'istanza della classe `sun.rmi.registry.RegistryImpl`) che espone un'interfaccia remota nota per permettere la registrazione e la ricerca degli oggetti.

Esistono due modalità principali per attivare questo servizio:

7.11.1 Registry Esterno

Come visto in precedenza, si utilizza l'eseguibile `rmiregistry`. In questo caso, il servizio gira in una **JVM separata** rispetto a quella del server applicativo. Questo garantisce che il registro rimanga attivo anche se il server crasha, ma richiede una gestione separata del classpath.

7.11.2 Registry Integrato (LocateRegistry)

È possibile creare e avviare un registro direttamente all'interno del codice Java del server. Questo approccio è spesso preferito per semplicità di deployment, poiché il registro condivide lo stesso ciclo di vita e la stessa istanza della **JVM del server**.

Per farlo, si utilizza la classe `java.rmi.registry.LocateRegistry`:

```

1 import java.rmi.registry.LocateRegistry;
2 import java.rmi.registry.Registry;
3
4 public class MyServer {
5     public static void main(String[] args) {
6         try {
7             // Crea il registry sulla porta specificata (es. 1099)
8             Registry registry = LocateRegistry.createRegistry(1099);

```

```

9
10         // Procede con il bind dell'oggetto
11         // registry.rebind("NomeServizio", oggettoRemoto);
12     } catch (Exception e) {
13         e.printStackTrace();
14     }
15 }
16

```

Listing 1: Creazione di un Registry integrato

Vantaggi del Registry Integrato

- **Semplicità:** Non è necessario avviare manualmente un processo esterno.
- **Sicurezza:** Il registro è accessibile più facilmente per operazioni di configurazione programmatica.
- **Classpath:** Non ci sono problemi di allineamento delle classi tra server e registro, essendo nella stessa JVM.

7.12 Stub

Lo **Stub** è l'entità che concretamente effettua l'invocazione remota lato client. Il suo compito è mascherare la complessità della rete, gestendo interamente il ciclo di vita della chiamata.

7.12.1 Responsabilità Operative

Lo stub si occupa di:

- **Gestione Dati:** Esegue la **serializzazione** degli argomenti e la **deserializzazione** del valore di ritorno o delle eccezioni.
- **Comunicazione:** Spedisce e riceve i pacchetti di dati appoggiandosi al **Remote Reference Layer (RRL)**, che gestisce i dettagli del protocollo di trasporto.

7.12.2 Composizione Tecnica

Uno stub non è una semplice classe, ma un oggetto con una struttura specifica definita dal framework:

- **Gerarchia delle Classi:** Estende `java.rmi.server.RemoteStub` e implementa sia l'interfaccia `java.rmi.Remote` che l'interfaccia remota specifica definita dal programmatore (ad esempio `EchoInterface`). Questo permette al client di trattarlo come se fosse l'oggetto reale.
- **Riferimento Remoto (RemoteRef):** Contiene al suo interno un'istanza di `java.rmi.server.RemoteRef` (accessibile tramite `super.ref`). Questo campo è vitale perché racchiude le informazioni sulla locazione del server e le modalità di connessione.
- **Stato della Chiamata:** Mantiene un riferimento allo stato corrente dell'invocazione (**remotecall**), necessario per coordinare l'invio della richiesta e l'attesa della risposta.

7.12.3 Esempio

Il seguente frammento illustra come lo **stub** traduca una chiamata a un metodo in operazioni di rete. Questo codice, tipicamente generato da **rmic**, evidenzia l'interazione con il **Remote Reference Layer (RRL)**.

```

1 // 1. FASE DI SETUP: CREAZIONE DELLA CHIAMATA
2 // Il metodo newCall inizializza l'oggetto remotecall tramite il riferimento
3 // remoto (super.ref). L'ID numerico finale e' un hash che garantisce la
4 // compatibilit tra le versioni dell'interfaccia di Client e Server.
5 java.rmi.server.RemoteCall remotecall =

```

```

6         super.ref.newCall(this, operations, 0, 6658547101130801417L);
7
8 // 2. FASE DI MARSHALLING: SERIALIZZAZIONE DEI PARAMETRI
9 // Viene ottenuto uno stream di output specifico per la chiamata.
10 // L'oggetto 'message' viene serializzato in una sequenza di byte.
11 try {
12     ObjectOutputStream objectoutput = remotecall.getOutputStream();
13     objectoutput.writeObject(message);
14     // In questo istante i dati vengono preparati per il trasporto
15 } catch (java.io.IOException e) {
16     throw new java.rmi.MarshalException("Errore durante la serializzazione", e);
17 }
18
19 // 3. FASE DI INVOCAZIONE: INVIO SULLA RETE
20 // Lo stub delega al livello RRL (super.ref) l'invio effettivo dei dati.
21 // Il thread del client si blocca qui in attesa della risposta dal server.
22 super.ref.invoke(remotecall);
23
24 // 4. FASE DI UNMARSHALLING: RICEZIONE DEL RISULTATO
25 // Una volta che invoke() ritorna, lo stub accede allo stream di input
26 // per leggere il valore di ritorno o l'eventuale eccezione remota.
27 Serializable risultato;
28 try {
29     ObjectInput objectinput = remotecall.getInputStream();
30
31     // Il metodo readObject() restituisce un tipo Object.
32     // Viene effettuato il cast a Serializable (tipo generico per gli oggetti).
33     // Durante questa operazione, la JVM del client deve caricare
34     // localmente la classe specifica dell'oggetto ricevuto.
35     risultato = (Serializable) objectinput.readObject();
36
37 } catch (java.io.IOException | ClassNotFoundException e) {
38     // ClassNotFoundException viene sollevata se il client
39     // non ha il file .class dell'oggetto ricevuto nel suo classpath.
40     throw new java.rmi.UnmarshalException("Errore nella deserializzazione", e);
41 }
42
43 // 5. FASE DI CLEANUP: SEGNALAZIONE AL LIVELLO RRL
44 finally {
45     // Comunica al Remote Reference Layer che la chiamata e' terminata.
46     // Questo permette al framework di rilasciare le risorse (es. socket),
47     // chiudere gli stream e pulire lo stato della sessione di chiamata.
48     super.ref.done(remotecall);
49 }
50
51 // 6. RITORNO AL LIVELLO APPLICATIVO
52 // Il risultato viene finalmente consegnato al codice chiamante del client.
53 return risultato;

```

7.13 Skeleton

Lo skeleton gestisce la de/serializzazione e la comunicazione con il **Remote Reference Layer (RRL)** lato server. Il suo compito principale è l'invocazione del metodo richiesto sull'oggetto reale.

- **Metodo dispatch:** È il cuore dello skeleton. Viene invocato dal RRL quando arriva una richiesta. Riceve il riferimento al server, l'oggetto della chiamata, l'indice del metodo (**opnum**) e l'hash di controllo.

- **Thread Management:** È importante notare che **non c'è creazione di thread** all'interno del metodo dispatch; la gestione della concorrenza è delegata ai livelli inferiori del framework RMI.

7.13.1 Esempio

Il frammento seguente mostra come lo skeleton smista le chiamate in arrivo:

```

1 public void dispatch(Remote remote, RemoteCall remotecall, int opnum, long hash)
   throws Exception {
2     // 1. CAST ALL'INTERFACCIA REALE
3     // Lo skeleton lavora su un riferimento generico 'Remote'
4     EchoRMIServer echormiserver = (EchoRMIServer)remote;
5
6     // 2. DISPATCHING (Selezione del metodo)
7     switch(opnum) {
8         case 0: // Ad esempio il metodo getEcho
9             String message;
10            try {
11                // 3. UNMARSHALLING DEI PARAMETRI
12                ObjectInput objectinput = remotecall.getInputStream();
13                message = (String)objectinput.readObject();
14            } finally {
15                // Libera il canale di input appena i dati sono letti
16                remotecall.releaseInputStream();
17            }
18
19            // 4. ESECUZIONE LOCALE
20            // Viene invocato il metodo vero e proprio sull'istanza del server
21            String message1 = echormiserver.getEcho(message);
22
23            try {
24                // 5. MARSHALLING DEL RISULTATO
25                // Prepara lo stream per inviare il valore di ritorno (true =
                // success)
26                ObjectOutput objectoutput = remotecall.getResultStream(true);
27                objectoutput.writeObject(message1);
28            } catch(IOException e) {
29                throw new MarshalException("Errore invio risultato", e);
30            }
31            break;
32
33            default:
34                throw new UnmarshalException("Metodo non valido o non trovato");
35        }
36    }

```

7.14 Livello trasporto

Il Livello di Trasporto rappresenta lo strato più basso dell'architettura RMI ed è responsabile della gestione fisica dei dati sulla rete. Il suo compito è stabilire le connessioni tra le JVM, gestire il flusso di byte e orchestrare le risorse di sistema per supportare le richieste in arrivo.

7.14.1 Concorrenza

La gestione della concorrenza e la generazione dei thread non avvengono nello Stub o nello Skeleton, ma vengono delegate ai livelli inferiori della JVM, precisamente al **Livello di Trasporto**.

Poiché la specifica RMI è aperta, diverse implementazioni possono adottare strategie differenti. Tuttavia, l'implementazione standard di Java segue il modello del **Server Parallelo**:

- **Generazione dei Thread:** RMI utilizza i thread nativi di Java. Tipicamente, viene generato (o prelevato da un pool) un **nuovo thread per ogni call** (invocazione remota) che arriva all'oggetto server.
- **Parallelismo:** Questo implica che più invocazioni sullo stesso oggetto remoto possono essere in esecuzione contemporaneamente nella stessa JVM su thread diversi.
- **Responsabilità del Programmatore:** Il server deve essere rigorosamente **thread-safe**. Se l'oggetto remoto gestisce risorse condivise o variabili di istanza, è necessario gestire le problematiche di sincronizzazione e mutua esclusione utilizzando i meccanismi di lock di Java (parola chiave **synchronized**).

Il **Livello Trasporto** è il vero "regista" delle attività concorrenti:

1. Ascolta le connessioni in ingresso su una porta specifica.
2. All'arrivo di una richiesta, assegna un thread per gestire il flusso di dati.
3. Passa la gestione al RRL e allo Skeleton all'interno di quel thread specifico.

Questo approccio basato sui thread è molto più efficiente rispetto all'uso di processi separati (tipico di alcuni sistemi RPC vecchio stile), poiché i thread condividono lo spazio di memoria della JVM, rendendo la comunicazione interna estremamente rapida.

7.14.2 Comunicazione

Anche per quanto riguarda la comunicazione fisica, la specifica RMI rimane flessibile, stabilendo come principio guida il buon utilizzo delle risorse. L'obiettivo primario è evitare l'overhead della creazione di nuove connessioni se ne esiste già una attiva tra due JVM.

Vengono ipotizzate tre strategie principali per la gestione del canale di trasporto:

1. **Connessione Esclusiva Sequenziale:** Si apre una sola connessione e la si utilizza per servire una richiesta alla volta. Questo approccio è efficiente nelle risorse ma causa forti effetti di **sequenzializzazione**, bloccando le chiamate successive finché la precedente non è terminata.
2. **Connessione Condivisa con Fallback (Strategia Comune):** Si utilizza una connessione aperta se libera; in caso di occupazione da parte di un'altra invocazione, se ne apre una nuova. Questo mitiga la sequenzializzazione a fronte di un **maggior impiego di socket**.
3. **Multiplexing su singola connessione:** Si utilizza un'unica connessione al livello di trasporto per servire diverse richieste contemporaneamente, effettuando il **demultiplexing** dei messaggi in arrivo e in partenza. In questo scenario, la connessione permane fino alla chiusura di una delle due JVM.

Indipendentemente dalla strategia, RMI tende a mantenere le connessioni aperte tra Client e Server come "canali persistenti". Questo evita il tipico costo di *handshake* del protocollo TCP per ogni singola chiamata, ottimizzando i tempi di risposta nelle invocazioni successive.

7.15 Caricamento classi e applet

Il linguaggio Java permette una gestione estremamente flessibile del codice grazie alla capacità di caricare le classi in modo dinamico e da remoto solo al momento dell'effettivo bisogno (Lazy Loading) durante l'esecuzione.

7.15.1 Le Applet

Le **Applet** sono componenti "class" associati a pagine Web che vengono trasferiti al client e messi in esecuzione automaticamente. Questo processo è reso possibile dalla presenza di una JVM integrata nel browser, dotata di un apposito **ClassLoader**. Quest'ultimo è il componente responsabile di:

- Localizzare il bytecode remoto tramite URL.
- Scaricare la classe nella memoria locale.

- Verificare l'integrità del codice prima dell'esecuzione.

Quando il codice viene scaricato dalla rete, però, la sua esecuzione viene costantemente monitorata da un componente critico: il **Security Manager**. Quest'ultimo:

- **Controlla gli Accessi:** Il Security Manager funge da controllore per ogni operazione sensibile (accesso al file system, apertura di socket, ecc.), bloccando quelle non esplicitamente consentite.
- **Applica la politica Sandbox:** Le prime versioni delle Applet adottavano una politica di sicurezza estremamente restrittiva denominata **sandbox**. Questa modalità permetteva un'esecuzione sicura impedendo ogni azione distruttiva sul computer dell'utente, limitando l'applet a operare esclusivamente all'interno di un'area isolata della memoria.

7.16 Class Loading

Il sistema di caricamento delle classi è il pilastro che rende Java un linguaggio dinamico ed estensibile. Invece di collegare tutte le librerie in un unico eseguibile monolitico, Java risolve i riferimenti ai tipi durante l'esecuzione stessa.

Java definisce un **ClassLoader**, cioè un'entità capace di risolvere i problemi di caricamento delle classi dinamicamente. Questo componente ha il compito di riferire e trovare le classi ogni volta che ce ne sia necessità, oltre che di caricarle e gestirne la rilocalizzazione in memoria.

È possibile anche definire una **gerarchia di ClassLoader** diversi, ciascuno responsabile del caricamento di classi provenienti da sorgenti differenti (locali, di sistema o remote). Questa struttura permette anche all'utente di definire ClassLoader personalizzati.

Il concetto di classloader comunque rispetta:

- **Namespace e Località:** I ClassLoader costituiscono ciascuno una "località" diversa l'uno dall'altro. Questo significa che classi caricate da diversi ClassLoader non interferiscono tra loro, garantendo un isolamento logico.
- **Visioni non consistenti:** Grazie a questo isolamento, due ClassLoader possono caricare classi con lo stesso nome ma provenienti da sorgenti diverse (ad esempio due versioni differenti della stessa libreria), mantenendo visioni non consistenti senza generare conflitti.
- **Meccanismo di Delegazione:** Tipicamente, un ClassLoader prima di caricare una classe interroga il proprio "padre" (parent), garantendo che le classi di sistema (come `java.lang.Object`) siano sempre caricate dal ClassLoader più autorevole.

7.16.1 Codebase

Il meccanismo del codebase permette di rendere le classi raggiungibili tramite un qualsiasi URL standard (HTTP, FTP, file, ecc.), consentendo il caricamento dinamico anche da locazioni remote. Questo è l'elemento che abilita la vera "mobilità del codice" in RMI.

L'**RMIClassLoader** non è un ClassLoader vero e proprio nel senso stretto della gerarchia Java, ma un componente di supporto specializzato che orchestra il processo eseguendo due operazioni fondamentali:

- **Estrazione del Codebase:** Quando un oggetto viene ricevuto via rete, l'**RMIClassLoader** estrae le informazioni sul "campo codebase" annotate nel flusso di serializzazione dell'oggetto remoto. Questo campo contiene l'URL dove risiede il bytecode (.class).
- **Gestione dei Codebase ClassLoaders:** Una volta ottenuto l'URL, il componente utilizza (o istanzia) dei **codebase classloaders** specifici per scaricare le classi necessarie dalla locazione remota e caricarle nella JVM locale.

7.17 Localizzazione del codice

Nel caso sia necessario ottenere dinamicamente del codice (come lo stub o classi di oggetti scambiati), il sistema RMI deve seguire un protocollo rigoroso:

1. **Localizzare il codice:** Individuare la sorgente, sia essa locale o remota.
2. **Download:** Effettuare il trasferimento del bytecode se la sorgente è remota.
3. **Esecuzione Sicura:** Verificare e caricare il codice sotto il controllo di un Security Manager.

Le informazioni su dove reperire il codice sono memorizzate sul server e passate al client **by need** (al momento del bisogno). Il Server RMI viene avviato configurando la proprietà `java.rmi.server.codebase`, la quale definisce l'URL da cui prelevare le classi necessarie. Tale URL può puntare a un server HTTP (`http://`), FTP (`ftp://`) o a una directory locale (`file://`).

Il codebase agisce come una proprietà del server che viene **annotata** nel **RemoteRef** pubblicato sul Registry; di conseguenza, ogni istanza dello stub scaricata dal client porta intrinsecamente con sé l'informazione sulla posizione delle proprie classi. Tuttavia, per ottimizzare le prestazioni e la sicurezza, la JVM segue una precisa gerarchia: le classi vengono cercate **sempre prima nel CLASSPATH locale** e solo in caso di insuccesso il sistema interroga il codebase remoto.

7.17.1 Uso del codebase

Il **codebase** agisce come un puntatore universale che permette la cooperazione tra macchine che non condividono lo stesso file system. Dal lato client, esso viene estratto dal **RemoteRef** per scaricare tutte le classi necessarie all'interazione con il server, incluse l'interfaccia remota, lo stub specifico e le definizioni degli oggetti restituiti come valore di ritorno. In questo processo è fondamentale distinguere tra l'**istanza** dello stub, che rappresenta i dati serializzati del riferimento remoto, e la **classe** dello stub, ovvero il bytecode necessario per interpretare ed eseguire correttamente tali dati sulla JVM locale.

Questo meccanismo di risoluzione dinamica non è limitato alla sola direzione server-client; nel caso di un passaggio per valore dal client verso il server di un oggetto la cui classe risulti ignota a quest'ultimo, il processo si inverte. Il **server** utilizzerà il codebase fornito dal client per reperire e scaricare le classi necessarie a completare la deserializzazione dei parametri ricevuti, garantendo che l'invocazione del metodo remoto possa procedere correttamente anche con tipi di dati definiti dinamicamente dal chiamante.

7.18 Security Manager

Per ogni ClassLoader può essere attivato un **Security Manager**, ovvero un garante e controllore che verifica la correttezza e la sicurezza di ogni operazione sensibile all'interno di quella località prima che venga effettivamente eseguita. La specifica di quali operazioni siano autorizzate è definita esternamente in un file di policy di autorizzazione. Affinché il controllo dinamico della sicurezza sia attivo, sia il client che il server devono essere lanciati specificando il percorso di tale file, che verrà poi consultato dal Security Manager durante tutto il ciclo di vita dell'applicazione.

7.18.1 RMISecurityManager

Per garantire un'esecuzione sicura del codice scaricato dinamicamente, RMI richiede l'utilizzo del **RMISecurityManager**. Questo componente specializzato effettua il controllo degli accessi alle risorse di sistema in base alle regole definite nel file di policy, bloccando tempestivamente ogni tentativo di accesso non autorizzato. All'interno di un'applicazione RMI, sia lato client che lato server, il Security Manager deve essere creato e installato programmaticamente se non ne è già presente uno istanziato nel sistema.

```
1  if (System.getSecurityManager() == null) {
2      System.setSecurityManager(new RMISecurityManager());
3  }
```

7.18.2 File di policy

Il file di policy presenta una struttura basata su una sequenza di operazioni consentite all'interno di un blocco `grant`. Un esempio tipico di configurazione permette di definire i permessi di rete e di file system necessari per l'operatività di RMI:

```
1 grant {
2     // Permette connessioni sulle porte utente (superiori a 1024)
3     permission java.net.SocketPermission ".*:1024-65535", "connect, accept";
4
5     // Permette di scaricare il codice tramite HTTP sulla porta 80
6     permission java.net.SocketPermission ".*:80", "connect";
7
8     // Permette la lettura dei file in una directory specifica
9     permission java.io.FilePermission "c:\\home\\RMIdir\\-", "read";
10 };
```

7.18.3 Problema e Soluzione di Sicurezza del Registry

Un potenziale problema critico riguarda la vulnerabilità del **Registry** rispetto ad attacchi di manipolazione. Poiché il registry è un servizio pubblico individuabile interrogando le porte di un host, un utente malintenzionato potrebbe teoricamente accedere all'elenco dei servizi registrati tramite il metodo `list()` e successivamente utilizzare `rebind()` per ridirigere le chiamate dei client verso un server malevolo sotto il proprio controllo.

Per contrastare questo rischio, RMI implementa una soluzione basata su restrizioni di accesso a livello di rete:

- **Restrizione Locale:** I metodi che modificano lo stato del registry, ovvero `bind()`, `rebind()` e `unbind()`, sono invocabili esclusivamente dall'host su cui è effettivamente in esecuzione il registry stesso.
- **Protezione da Nodi Esterni:** Grazie a questo vincolo, il sistema non accetta modifiche della struttura client/server provenienti da nodi esterni. Un tentativo di invocare un `rebind()` da una macchina remota verrebbe immediatamente rigettato con una `AccessException`, garantendo che solo il processo server legittimo (o un amministratore locale) possa pubblicare o aggiornare i riferimenti agli oggetti.
- **Accesso in Lettura:** Rimane invece libera e consentita l'invocazione dei metodi di sola lettura, come `lookup()` e `list()`, da parte di qualsiasi nodo della rete, permettendo così ai client legittimi di trovare i servizi di cui hanno bisogno.

7.19 Problema del Bootstrap

Il problema del bootstrap riguarda le modalità di avvio del sistema e la possibilità di recuperare il primo riferimento remoto necessario a iniziare la comunicazione. Se ogni interazione richiede uno stub, sorge il problema di come ottenere lo stub del servizio di naming senza avere già un meccanismo di risoluzione attivo.

Per risolvere questa criticità, Java mette a disposizione la classe **Naming**, la quale realizza dei metodi statici per effettuare le operazioni di registrazione, de-registrazione e reperimento dei riferimenti dei server. Tuttavia, poiché anche il Registry è un oggetto remoto, per invocare i suoi metodi il client necessiterebbe del suo stub.

La soluzione adottata da RMI prevede la **costruzione locale dell'istanza dello stub** del Registry, evitando così di doverlo consultare preventivamente. Tale costruzione avviene combinando alcune informazioni chiave:

- **Dati di Rete:** L'indirizzo del server e la porta (standard 1099) contenuti nell'URL dell'oggetto remoto (es. `rmi://host:port/name`).
- **Identificatore Fisso:** Un identificatore univoco dell'oggetto Registry sulla macchina server, il quale è fissato a priori dalla specifica RMI di SUN come una **costante fissa**.

Grazie a questo approccio, la classe **Naming** può sintetizzare "al volo" lo stub del Registry locale o remoto basandosi solo su parametri noti o convenzionali, permettendo così l'invocazione del metodo `lookup()` e l'avvio dell'intera catena di risoluzione dei nomi.

7.20 Visione interna Memoria Java

La gestione della memoria in Java segue un principio dinamico: le istanze e le classi non vengono caricate in modo statico preventivo, ma caricate **by need** solo quando sono effettivamente necessarie all'esecuzione del programma. Questo approccio ottimizza l'uso delle risorse, evitando di occupare memoria per componenti mai utilizzati.

La struttura della memoria si articola principalmente in due aree:

- **Heap:** Un'area di memoria condivisa dove risiedono tutti gli oggetti (sia le istanze che i descrittori delle classi) allocati dinamicamente.
- **Stack:** Ogni thread possiede un proprio stack privato. Qui vengono memorizzate le variabili locali e i riferimenti che puntano agli oggetti situati nell'heap. Le variabili di istanza e di interfaccia all'interno degli stack fungono quindi da "ancore" verso i dati reali contenuti nell'area comune.

7.20.1 Heap

All'interno dell'heap, la gestione del ciclo di vita degli oggetti richiede strategie precise per l'allocazione e, soprattutto, per la deallocazione delle risorse non più utilizzate. Storicamente si distinguono due approcci:

1. **Reference Counting:** Ogni oggetto mantiene un contatore interno che tiene traccia di quanti riferimenti attivi puntano ad esso. Se il contatore scende a zero, l'oggetto può essere rimosso. Questa strategia solleva dubbi sulla correttezza del valore in scenari complessi (come i cicli di riferimenti).
2. **Garbage Collection (GC):** Un'attività in background identifica e dealloca gli oggetti che non sono più raggiungibili da alcun riferimento attivo. Java adotta un modello di GC ispirato all'algoritmo **mark-and-sweep** di Dijkstra, eseguendo la pulizia "on the fly" mentre l'applicazione è in esecuzione.

In ambito RMI, la gestione locale del Garbage Collector non è sufficiente poiché non può conoscere lo stato dei riferimenti residenti su JVM remote. Se si utilizzasse una strategia di **Reference Counting distribuito puro**, in cui il server interroga costantemente tutti i potenziali client per contare i riferimenti attivi o i client notificano ogni singola variazione, il costo in termini di traffico di rete e latenza diventerebbe un **costo insostenibile**.

Per ovviare a questo problema, RMI adotta una strategia basata sul **Leasing** (noleggio):

- **Concessione del Lease:** Quando un client ottiene un riferimento a un oggetto remoto, "noleggia" l'oggetto per un periodo di tempo determinato.
- **Rinnovo (Dirty Call):** Il client ha la responsabilità di rinnovare periodicamente il contratto di leasing inviando una notifica al server prima della scadenza.
- **Scadenza e Cleanup:** Se il server non riceve il rinnovo entro il termine stabilito (ad esempio a causa di un crash del client o di un guasto di rete), il lease scade. Il server presume che il client non sia più interessato o raggiungibile, permettendo al Garbage Collector locale di deallocare l'oggetto.

7.21 Remote references

Il sistema di nomi **RMI-registry** rappresenta lo strumento di default per la propagazione della visibilità globale degli oggetti remoti. In un'architettura Client/Server, il registry viene utilizzato principalmente per ottenere il **primo riferimento remoto** (lo stub) del servitore; una volta stabilito questo contatto iniziale, i riferimenti remoti possono essere propagati in modo più fluido attraverso le normali chiamate ai metodi remoti, passandoli come parametri o ricevendoli come valori di ritorno.

Nonostante la sua potenza, RMI **NON** si propone come uno strumento adatto a sistemi *in-the-large* (su scala geografica o globale) per una serie di ragioni tecniche e architetturali:

- **Località del Registry:** Il registry è progettato per registrare solo servizi locali alla macchina su cui è in esecuzione; non costituisce un sistema di nomi globale o federato.
- **Complessità del GC:** Come analizzato precedentemente, la Garbage Collection distribuita (DGC) basata su leasing e conteggio dei riferimenti è intrinsecamente complessa e comporta un costo computazionale e di rete rilevante.
- **Efficienza e Flessibilità:** Il costo prestazionale delle chiamate RMI è elevato rispetto ad altre forme di comunicazione, e lo strumento offre una flessibilità limitata nel gestire scenari eterogenei.
- **Rigidità delle Strategie:** RMI presenta meccanismi non facilmente estendibili e strategie (di comunicazione, di sicurezza o di threading) che non sempre si adattano a requisiti applicativi specifici o non standard.

8 Implementazione RPC di SUN

La chiamata di procedura remota (**RPC**) permette di invocare una procedura non locale come se fosse disponibile nello spazio di indirizzamento corrente. A differenza di RMI, che è orientato agli oggetti e specifico per Java, SUN RPC è orientato alle procedure e progettato per l'interoperabilità tra linguaggi (principalmente C) e sistemi diversi.

8.1 Requisiti implementativi

L'infrastruttura SUN RPC viene inglobata nei processi coinvolti e si occupa di gestire la complessità della comunicazione:

- **Identificazione:** Ogni messaggio di chiamata e risposta deve essere chiaramente distinguibile; ogni procedura remota è identificata in modo unico da una terna: (**Program Number, Version Number, Procedure Number**).
- **Gestione Eterogeneità:** Il supporto gestisce le differenze di rappresentazione dei dati (es. *endian-ness*) tramite:
 - **Marshalling/Unmarshalling:** Impacchettamento dei parametri.
 - **Serializzazione:** Trasformazione in un flusso di byte standard.
- **Gestione Errori:** Il supporto deve gestire fallimenti come implementazioni errate del server, errori dell'utente o necessità di **roll-over** (ripristino dello stato) in caso di guasto.

8.2 Semantica di Invocazione

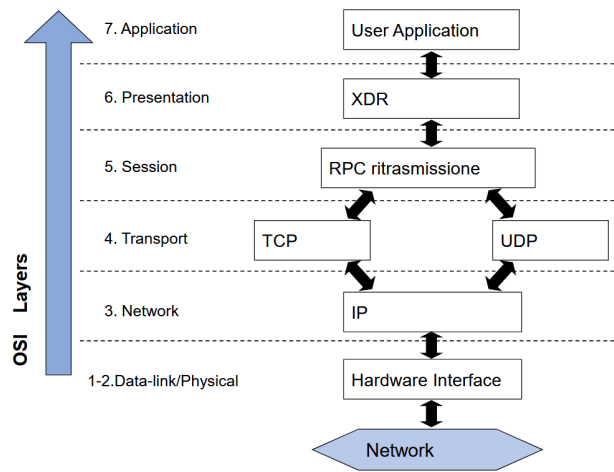
In un sistema distribuito, l'affidabilità dipende dalla semantica scelta per gestire i guasti:

- **Maybe:** Nessuna garanzia; la procedura può essere eseguita zero o una volta.
- **At-least-once:** (Default in SUN RPC) La procedura viene eseguita almeno una volta. In caso di mancata risposta, il client riprova (utile per operazioni *idempotenti*).
- **At-most-once:** La procedura viene eseguita al massimo una volta. Richiede che il server mantenga una cronologia delle risposte per evitare riesecuzioni.
- **Exactly-once:** L'ideale teorico (difficile da realizzare senza costi elevati di coordinamento).

8.3 ONC (Open Network Computing)

ONC è la suite di Sun che include gli strumenti fondamentali per lo sviluppo RPC:

Componente	Funzione
XDR (eXternal Data Repr.)	Standard per la rappresentazione dei dati tra macchine diverse.
RPCGEN	Compilatore che genera automaticamente stub del client e del server.
Portmapper	Servizio (porta 111) che risolve il numero del programma in un numero di porta TCP/UDP.
NFS (Network File System)	L'applicazione principale basata su RPC per la condivisione di file.



8.4 Contratto

Le RPC si basano su un **contratto esplicito** sulle informazioni scambiate che permetta un accordo preciso tra entità potenzialmente eterogenee. Questo contratto si articola in due momenti:

- **Contratto tra le opzioni:** Si definisce quali servizi il server dichiara di esportare e quali il client può richiedere. In particolare, l'implementazione Sun utilizza una struttura rigida basata su una tripla composta da `<#programma, #versione, #procedura>`.
- **Specifica della procedura:** Si definisce l'interfaccia della procedura da invocare usando argomenti di tipo specificato, garantendo che entrambe le parti interpretino i dati nello stesso modo.

Poiché ci si trova in architetture diverse (che possono differire per sistema operativo o rappresentazione dei dati), i processi devono essere **autocontenuti**. Ciò significa che ogni chiamata deve trasportare tutte le informazioni necessarie alla propria esecuzione senza fare assunzioni sullo stato della memoria della macchina remota.

8.5 Programma

Le RPC sono basate su un contratto esplicito sulle informazioni scambiate, che consiste di due parti descrittive scritte in un apposito linguaggio di interfaccia:

1. **Definizioni di programmi RPC:** Specificano il protocollo per i servizi offerti, includendo l'identificazione univoca dei servizi e il tipo dei parametri.
2. **Definizioni XDR:** Descrivono i tipi di dati dei parametri, necessari qualora il tipo di dato non rientri tra quelli predefiniti in RPC.

Tutte queste specifiche vengono raggruppate in un unico file con estensione `.x` (formato XDR sorgente), che deve limitarsi a descrivere esclusivamente il contratto di interazione senza dettagli implementativi.

8.5.1 Definizione del servizio remoto

Prendendo come esempio il file `stampa.x`, la definizione segue una struttura gerarchica che identifica programma, versione e procedura:

```

1 program STAMPAPROG {
2     version STAMPAVERS {
3         int PRINTMESSAGE(string) = 1;
4     } = 1;
5 } = 0x200000013;
```

Questo codice definisce la procedura `PRINTMESSAGE`, che accetta un argomento `string` e restituisce un `int`. È fondamentale osservare i vincoli tipici di Sun RPC:

- **Arity limitata:** Ogni procedura può avere **un solo parametro di ingresso** e **un solo parametro di uscita**. Se serve passare più valori, occorre definirli come un'unica `struct XDR`.
- **Convenzioni di Naming:** Per convenzione, gli identificatori usano lettere maiuscole.
- **Identificatori Numerici:** Ogni procedura, versione e programma deve essere associato a un numero unico. In particolare, i numeri di programma (come `0x20000013`) sono divisi in intervalli per evitare conflitti tra servizi standard e definiti dall'utente.

8.5.2 Implementazione del programma

Nello sviluppo di un'applicazione RPC, il programmatore ha la responsabilità di scrivere solo la logica applicativa:

1. **Il programma cliente:** Contiene il `main()` e la logica necessaria per il reperimento e il **binding** dei servizi remoti (solitamente nel file `esempio.c`).
2. **Il programma server:** Contiene l'effettiva implementazione delle procedure dichiarate nel file `.x` (solitamente nel file `esempio_svc_proc.c`).

Tutta la complessa infrastruttura di rete (gli stub del cliente e del servitore) viene generata automaticamente dal compilatore `rpcgen` a partire dal file `.x`. Un punto cruciale è che il **main del server** non viene definito dal programmatore: esso viene generato automaticamente per gestire l'ascolto sulle porte, la registrazione presso il portmapper e il dispacciamento delle richieste ai thread o ai processi corretti.

8.6 Procedura di servizio

La trasformazione di una funzione locale in una procedura remota implica modifiche strutturali per gestire l'eterogeneità e la persistenza dei dati durante la comunicazione. Di seguito il confronto tra l'implementazione locale e quella remota:

```
1  /* VERSIONE LOCALE */
2  int printmessage (char *msg) {
3      FILE *f = fopen("/dev/console", "w");
4      if (f == NULL) return(0);
5      fprintf(f, "%s\n", msg); fclose(f);
6      return(1);
7  }
8
9  /* VERSIONE REMOTA (Lato Server) */
10 int * printmessage_1_svc (char **msg, struct svc_req *rp) {
11     static int result; /* Variabile statica per la persistenza */
12     FILE *f = fopen("/dev/console", "w");
13     if (f == NULL) { result = 0; return(&result); }
14     fprintf(f, "%s\n", *msg); fclose(f);
15     result = 1;
16     return(&result);
17 }
```

Listing 2: Confronto tra funzione locale e remota

Dall'analisi del codice si osservano i vincoli operativi di Sun RPC:

- **Passaggio per riferimento:** Sia l'argomento di ingresso che quello di uscita vengono passati tramite puntatore (`char **` e `int *`). Questo è necessario perché gli stub devono poter manipolare i dati per la serializzazione XDR.

- **Argomento di supporto:** Viene aggiunto il parametro `struct svc_req *rp`, che contiene i dettagli sulla richiesta (credenziali, protocollo, ecc.).
- **Variabile statica per il risultato:** Il risultato punta a una variabile **static**. Poiché la funzione restituisce un puntatore, se la variabile fosse automatica (sullo stack), verrebbe deallocata al termine della funzione, rendendo il puntatore non valido per lo stub che deve inviare la risposta. L'uso di **static** garantisce che il dato sopravviva nell'area dati globale oltre la fine della chiamata.
- **Naming Convention:** Il nome della procedura viene modificato dal compilatore `rpcgen` aggiungendo il suffisso `_1_svc` (numero di versione e identificatore service).

8.6.1 Implementazione del Client

Il client deve gestire esplicitamente il ciclo di vita della connessione (binding) e l'invocazione tramite lo stub:

```

1 int main(int argc, char *argv[]) {
2     int *result;
3     CLIENT *cl; /* Handle del client per il trasporto */
4
5     if(argc < 3) { exit(1); }
6
7     /* Creazione dell'handle client (Binding) */
8     cl = clnt_create(argv[1], STAMPAProg, STAMPAVERS, "udp");
9     if (cl == NULL) { clnt_pcreateerror(argv[1]); exit(1); }
10
11    /* Invocazione dello stub del client */
12    result = printmessage_1(&argv[2], cl);
13
14    if (result == NULL) { clnt_perror(cl, argv[1]); exit(1); }
15    printf("Messaggio consegnato\n");
16 }

```

Nel client, la funzione `clnt_create` contatta il **Portmapper** sull'host remoto per scoprire su quale porta il servizio è in ascolto, stabilendo il canale di comunicazione (in questo caso via UDP).

8.7 Passi di Sviluppo

Lo sviluppo di un'applicazione basata su Sun RPC segue un workflow strutturato, centrato sull'automazione offerta dal compilatore `rpcgen`. I passi fondamentali sono:

- **Definizione di servizi e tipi di dati:** Si redige il file di interfaccia (estensione `.x`) definendo la tripla `<programma, versione, procedura>` e le eventuali strutture dati XDR.
- **Generazione degli stub e funzioni XDR:** Si utilizza il comando `rpcgen -C esempio.x`. Questo genera automaticamente:
 - Il file header (`esempio.h`).
 - Lo stub del client (`esempio_clnt.c`).
 - Lo skeleton del server (`esempio_svc.c`).
 - Le routine di serializzazione (`esempio_xdr.c`).
- **Realizzazione, compilazione e linking:** Il programmatore scrive la logica del client (`main`) e le procedure del server. Tutti i moduli (stub generati e codice utente) vengono compilati e linkati insieme.
- **Pubblicazione dei servizi:** Si attiva il **Portmapper** (o `rpcbind`) sull'host server. All'avvio, il server si registra presso il Portmapper, associando la propria tripla identificativa alla porta di ascolto dinamica.

- **Reperimento dell'endpoint:** Il client, tramite la funzione `clnt_create`, interroga il Portmapper remoto per ottenere l'indirizzo (endpoint) e creare il gestore di trasporto (`CLIENT *`) necessario allo scambio dei messaggi.

8.8 RPC in UNIX

Le Sun RPC sono progettate per essere indipendenti dal protocollo di trasporto, ma nel mondo UNIX si appoggiano direttamente ai servizi delle **socket**. Il programmatore può scegliere il protocollo sottostante in fase di creazione del gestore di trasporto (`clnt_create`):

- **UDP (User Datagram Protocol):** È il protocollo di **default** per Sun RPC. Offre prestazioni migliori grazie alla mancanza di gestione della connessione, ma essendo un protocollo *unreliable*, non garantisce la consegna dei pacchetti. È adatto per servizi leggeri e veloci (come NFS in reti locali affidabili).
- **TCP (Transmission Control Protocol):** Viene utilizzato quando è richiesta una connessione affidabile per il trasferimento di grandi quantità di dati o quando la rete non è sicura. TCP gestisce la ritrasmissione e l'ordinamento dei pacchetti, a fronte di un overhead maggiore.

L'integrazione con UNIX prevede che il **Portmapper** (o `rpcbind`) rimanga in ascolto sulla porta standard **111** (sia TCP che UDP). Quando un server RPC viene avviato, esso ottiene dal sistema operativo una porta libera (effimera) e la comunica al Portmapper, associandola alla propria tripla `<programma, versione, protocollo>`.

8.9 RPC di SUN

L'architettura delle Sun RPC impone alcuni vincoli fondamentali che ne definiscono il comportamento e la semplicità di gestione:

- **Gerarchia del servizio:** Un singolo programma RPC può contenere al suo interno più procedure logicamente correlate.
- **Versionamento:** Il sistema prevede esplicitamente la gestione di versioni multiple per lo stesso programma, permettendo la coesistenza di protocolli vecchi e nuovi sullo stesso server.
- **Arity fissa:** Ogni procedura può accettare **un unico argomento in ingresso** e restituire **un unico argomento in uscita**. Per superare questo limite, è necessario incapsulare più dati all'interno di una struttura dati complessa (`XDR struct`).

8.9.1 Gestione della Concorrenza

Nelle Sun RPC standard, la **mutua esclusione** è garantita dal design del server stesso. Di default, non è prevista alcuna concorrenza nell'ambito dello stesso programma server: il **Server è sequenziale**. Ciò significa che viene eseguita una sola invocazione per volta e le richieste concorrenti vengono accodate, semplificando drasticamente la scrittura del codice (non serve gestire semafori o lock) ma limitando le prestazioni in scenari ad alto carico.

8.9.2 Sincronicità

Il modello di interazione è strettamente **sincrono e bloccante**. Dal momento in cui viene effettuata la chiamata allo stub, il processo cliente rimane sospeso. Esso non riprenderà l'esecuzione fino a quando non riceverà il risultato di ritorno dal server o non scatterà un timeout di rete. Questo garantisce una semantica simile a quella di una chiamata a funzione locale, ma rende il client vulnerabile a eventuali rallentamenti o crash del server.

8.10 Rischi e problemi

L'adozione di un modello basato su server sequenziali introduce rischi strutturali significativi, in particolare per quanto riguarda la gestione dei flussi di controllo tra nodi diversi:

- **Possibilità di deadlock:** Poiché un server sequenziale è bloccato fino al completamento della procedura corrente, si verifica un deadlock se tale server richiede, a sua volta, un servizio RPC al programma chiamante (chiamata circolare). Il chiamante è in attesa del server, e il server è in attesa del chiamante, bloccando l'intero sistema.
- **Gestione delle richieste multiple:** Sebbene un nodo server possa ricevere più chiamate RPC contemporaneamente, l'architettura di default le serve **una alla volta**, accodando le altre. Questo può causare colli di bottiglia e aumentare i tempi di risposta (latenza) per i client in coda.

8.10.1 Semantica e affidabilità

L'affidabilità delle Sun RPC è strettamente legata alla scelta del protocollo di trasporto e alla gestione dei fallimenti di rete:

- **Uso di protocollo UDP:** Essendo il default in ambito Sun, il sistema deve gestire internamente la mancanza di affidabilità tipica dei datagrammi (smarrimento o duplicazione dei pacchetti).
- **Semantica *at-least-once*:** A causa dell'assenza di connessione, il sistema adotta una strategia di persistenza. Se il client non riceve risposta entro un intervallo di **time-out** (tipicamente 4 secondi), effettua automaticamente un certo numero di **ritrasmissioni**.

Questa semantica garantisce che l'operazione venga eseguita *almeno una volta*, ma introduce il rischio di esecuzioni multiple sul server se la risposta è andata persa o è arrivata in ritardo. Per questo motivo, le procedure chiamate tramite UDP dovrebbero idealmente essere **idempotenti** (ovvero la loro esecuzione multipla produce lo stesso effetto di una singola esecuzione).

8.11 Identificazione di RPC

Per garantire un'identificazione globale e univoca, ogni messaggio RPC deve contenere tre identificatori fondamentali: il **numero di programma**, il **numero di versione** e il **numero di procedura**.

I numeri di programma sono espressi in notazione esadecimale e sono suddivisi in intervalli gestiti centralmente per scopi diversi:

- **0 - 0x1FFFFFFF (Predefinito Sun):** Questo intervallo è riservato ai servizi di sistema standard definiti da Sun Microsystems (ad esempio, NFS, NIS o il **Portmapper**).
- **0x20000000 - 0x3FFFFFFF (Definibile dall'utente):** Spazio destinato agli utenti e agli sviluppatori per le proprie applicazioni distribuite personalizzate.
- **0x40000000 - 0x5FFFFFFF (Riservato):** Intervallo utilizzato dalle applicazioni per generare dinamicamente numeri di programma temporanei.
- **0x60000000 - 0xFFFFFFFF:** Intervalli riservati per usi futuri o estensioni del protocollo.

Questa categorizzazione permette al **Portmapper** di gestire le registrazioni senza sovrapposizioni tra i servizi vitali del sistema operativo e quelli sviluppati dai programmatori.

8.12 Supporto standard RPC

Sun ha formalizzato il protocollo RPC tramite una serie di strutture **XDR** (definite negli RFC relativi) che standardizzano lo scambio di messaggi. Ogni pacchetto che viaggia sulla rete è rappresentato dalla struttura `rpc_msg`, che funge da contenitore universale per chiamate e risposte.

```

1 struct rpc_msg {
2     unsigned int xid; /* Transaction ID: identificativo unico della chiamata */
3     union switch (msg_type mtype) {
4         case CALL: call_body cbody;
5         case REPLY: reply_body rbody;
6     } body;
7 };

```

Il campo **XID** (Transaction ID) è fondamentale per la semantica del protocollo:

- Viene generato un ID unico per ogni nuova invocazione.
- In caso di **ritrasmissione** (tipica di UDP dopo un timeout), lo XID rimane identico. Questo permette al server di riconoscere messaggi duplicati e, se necessario, di non rieseguire procedure non idempotenti.

8.12.1 Anatomia dei messaggi: CALL e REPLY

Il corpo della chiamata (`call_body`) implementa il sistema di nomi completo necessario per l'instradamento corretto del servizio:

```

1 struct call_body {
2     unsigned int rpcvers; /* Versione del protocollo RPC (attualmente 2) */
3     unsigned int prog;    /* Numero del programma */
4     unsigned int vers;    /* Numero della versione */
5     unsigned int proc;    /* Numero della procedura */
6     opaque_auth cred;     /* Credenziali di autenticazione */
7     opaque_auth verf;     /* Verificatore di autenticazione */
8 };

```

La risposta (`reply_body`) è invece un'unione che riflette l'esito della richiesta a due livelli: **accettazione** (il messaggio è arrivato correttamente al processo server) o **rifiuto** (problemi di protocollo o autorizzazione).

8.12.2 Gestione degli Errori e Stati

Sun RPC definisce un set rigoroso di enumerazioni per mappare ogni possibile fallimento, sia lato rete che lato applicativo:

- **accept_stat:** Gestisce gli errori dopo che il server ha ricevuto il pacchetto. Esempi critici sono `PROG_MISMATCH` (versione non supportata) o `GARBAGE_ARGS` (impossibile decodificare i parametri con XDR).
- **reject_stat:** Gestisce il rifiuto immediato del messaggio, solitamente per `RPC_MISMATCH` (versione del protocollo RPC diversa dalla 2) o `AUTH_ERROR`.
- **clnt_stat:** Rappresenta lo stato visto dal programmatore client. Include errori locali (come `RPC_TIMEDOUT` o `RPC_CANTENCODEARGS`) e riflette gli errori remoti riportati dal server (come `RPC_PROGNOTREGISTERED`).

8.13 Il Sistema di Nomi

Il coordinamento tra client e server in Sun RPC non avviene in modo diretto, ma richiede un intermediario noto come **Portmapper** (o `rpcbind`). Questo componente funge da "pagine gialle" del sistema, permettendo di mappare gli identificatori logici delle procedure sulle porte fisiche della rete. La gestione della tabella `port_map` è affidata a un processo demone unico per ogni nodo, che abilita due gestori di trasporto (UDP e TCP) legati alla porta ben nota **111**. Essendo esso stesso un servizio RPC, il Portmapper risponde al numero di programma **100000**, versione **2**.

8.13.1 Meccanismo di Registrazione e Struttura Dati

Il sistema si basa su una tabella dinamica che mette in relazione l'identità logica di un servizio con il suo indirizzo fisico. La registrazione avviene tramite una tripla composta da numero di programma, numero di versione e protocollo di trasporto, associata a un numero di porta. È fondamentale notare che nella registrazione **manca il numero di procedura**: questa scelta deriva dal fatto che tutte le procedure di uno stesso programma condividono lo stesso endpoint fisico; la selezione della procedura specifica avviene dinamicamente solo al momento dell'invocazione.

La struttura dati utilizzata per gestire questa corrispondenza è una lista dinamica di mappature:

```
1 struct mapping {
2     unsigned long prog;
3     unsigned long vers;
4     unsigned int prot; // Costanti IPPROTO_UDP o IPPROTO_TCP
5     unsigned int port; // Porta assegnata dal sistema operativo
6 };
7
8 struct pmaplist {
9     mapping map;
10    struct pmaplist *next;
11};
```

8.13.2 Interazione Client-Server

L'interazione operativa si articola in fasi distinte che garantiscono l'efficienza del sistema:

- **Lato Server:** Il processo utilizza funzioni come `register_rpc()` per popolare la tabella del Portmapper. La registrazione trasforma una funzione locale in un servizio globale. Se il server termina senza de-registrarsi, il Portmapper manterrà un'informazione obsoleta, causando errori di tipo `RPC_PROGNOTREGISTERED` ai client successivi.
- **Lato Client:** Prima della chiamata (*naming phase*), il client interroga il Portmapper remoto per ottenere la porta del servizio. Questa informazione viene poi memorizzata nel gestore di trasporto del client (`CLIENT handle`) per evitare interrogazioni ridondanti del Portmapper durante le invocazioni successive.

In sintesi, registrare il programma anziché le singole procedure garantisce **economicità** di gestione, riduce la dimensione della tabella `port_map` e disaccoppia il reperimento dell'indirizzo fisico dai dettagli logici dell'applicazione, ottimizzando il contratto di comunicazione tra le entità distribuite.

8.13.3 Servizi Portmapper

Il Portmapper espone un'interfaccia di servizio standard (basata essa stessa su RPC) per consentire la manipolazione e la consultazione della tabella di mappatura. I servizi principali includono:

- **Inserimento (`PMAPPROC_SET`):** Utilizzato dai server all'avvio per registrare una nuova associazione. Il server comunica la tripla `{prog, vers, prot}` e la porta su cui è in ascolto. Se l'operazione ha successo, il servizio diventa visibile ai client.
- **Eliminazione (`PMAPPROC_UNSET`):** Utilizzato dai server in fase di terminazione per rimuovere le proprie voci dalla tabella. È una fase critica per evitare che la tabella contenga riferimenti a porte "morte".
- **Corrispondenza (`PMAPPROC_GETPORT`):** È la funzione di lookup principale utilizzata dai client. Dato un identificatore astratto (`programma, versione, protocollo`), il Portmapper restituisce il numero di porta specifico per quel servizio. Se restituisce zero, il programma non è registrato.

- **Intera lista (PMAPPROC_DUMP):** Permette di ottenere l'elenco completo di tutte le registrazioni correnti nella `port_map`. Questa operazione è quella invocata dal comando di sistema `rpcinfo -p`.
- **Supporto (PMAPPROC_CALLIT):** Consente a un client di richiedere al Portmapper di invocare una procedura per suo conto sul nodo locale, senza conoscere preventivamente la porta del server finale (utile per il broadcasting).

8.14 Primitiva `svc_run`

Una volta completata la registrazione delle procedure, il processo server deve entrare in uno stato di ascolto attivo. Per fare ciò, si utilizza la primitiva `svc_run()`, che trasforma il processo in un demone incaricato di gestire il ciclo di vita delle richieste.

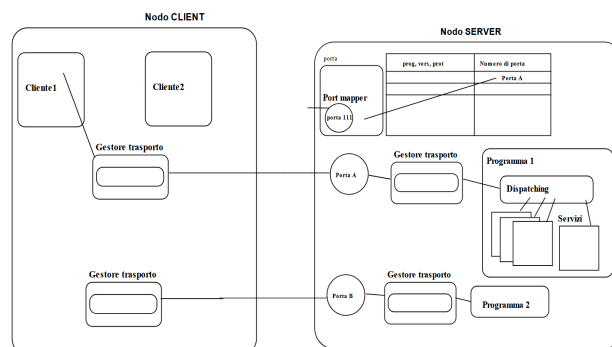
- **Attesa Infinita:** La funzione `svc_run()` implementa un ciclo infinito al cui interno viene eseguita una `select()`. Questo permette al server di rimanere sospeso in modo efficiente, venendo "risvegliato" dal kernel solo quando arrivano dati sulle socket associate ai programmi registrati.
- **Natura Bloccante:** Essendo implementata come un loop senza condizione di uscita, la `svc_run()` non termina mai il controllo al chiamante. Tutto il codice successivo a questa chiamata nel `main` del server risulterebbe dunque non raggiungibile.

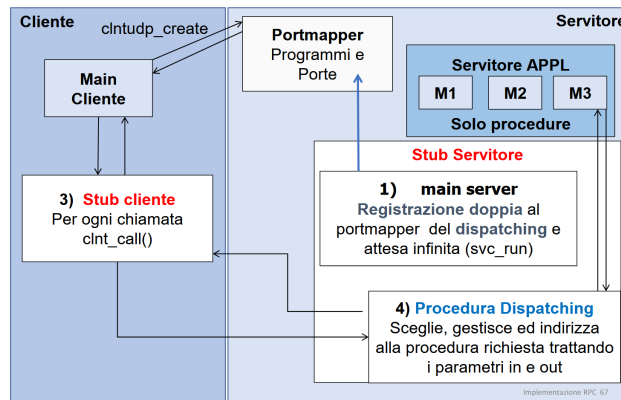
8.14.1 Vincoli di Trasporto e Compatibilità

La gestione dei processi servitori presenta vincoli precisi che dipendono strettamente dalle primitive di registrazione utilizzate durante lo sviluppo. Le procedure registrate tramite la funzione semplificata `register_rpc()` sono progettate per essere compatibili esclusivamente con il protocollo **UDP**, operando quindi in modalità senza connessione e sfruttando la velocità dei datagrammi per lo scambio dei messaggi. TCP a stream risulta quindi incompatibile.

8.15 Architettura RPC

È possibile quindi definire l'architettura RPC, divisa tra nodo client e nodo server, come segue:





8.16 Interface Definition Language: XDR

Per definire il contratto tra client e server in modo rigoroso e indipendente dall'architettura, Sun ha introdotto l'**eXternal Data Representation (XDR)**. XDR è un linguaggio di definizione delle interfacce (IDL) che permette di determinare in modo non ambiguo sia le procedure remote che la struttura dei dati scambiati sulla rete.

8.16.1 Vincoli dell'Interfaccia

L'interfaccia definita in XDR impone una struttura rigida per ogni procedura remota, che deve rispettare i seguenti vincoli:

- Ogni procedura accetta **esattamente un argomento** in ingresso.
- Ogni procedura restituisce **esattamente un risultato**.

Per superare il limite del singolo parametro, XDR permette di incapsulare più informazioni all'interno di tipi composti (strutture), che vengono poi trattati come un unico oggetto logico durante il trasporto.

8.16.2 Tipologie di Dati in XDR

Il linguaggio XDR supporta una vasta gamma di tipi di dati per coprire ogni esigenza applicativa:

- **Tipi atomici predefiniti:** Includono tipi base come `int`, `unsigned int`, `bool`, `float` e `double`. XDR garantisce che un `int` sia sempre rappresentato a 32 bit con codifica Big-Endian, indipendentemente dall'architettura del nodo.
- **Tipi standard (Costruttori):** Supporto per tipi di dati comuni come stringhe (`string`), enumerazioni (`enum`) e dati opachi (`opaque`), usati per sequenze di byte non interpretate.
- **Tipi costruiti:** Permettono di definire strutture complesse tramite costruttori come `struct`, `union` (unioni discriminate) e array (sia a dimensione fissa che variabile).

8.16.3 Funzioni

Il compito principale del protocollo XDR è il **Marshalling** (o serializzazione). A tal fine, XDR fornisce una libreria di routine predefinite (le funzioni `xdr_*`) che automatizzano la conversione per le diverse categorie di dati:

- **Tipi atomici predefiniti:** Funzioni per la gestione di tipi base come `xdr_int()`, `xdr_float()`, `xdr_bool()` e `xdr_u_long()`. Queste routine si occupano di gestire l'ordine dei byte (*Endianness*) e la dimensione dei tipi.
- **Tipi standard (Costruttori riconosciuti):** Routine per tipi più evoluti ma standardizzati, come `xdr_string()` per le stringhe di caratteri e `xdr_enum()` per le enumerazioni.

Queste procedure sono **bidirezionali**: la stessa funzione viene utilizzata sia dal mittente per codificare (*encoding*) sia dal destinatario per decodificare (*decoding*) i dati, semplicemente cambiando lo stato dell'oggetto XDR passato come parametro.

8.16.4 Posizionamento nel modello OSI

All'interno dell'architettura ISO/OSI, XDR si colloca esattamente al **Livello di Presentazione** (Livello 6). Il suo scopo non è gestire la connessione (compito dei livelli inferiori), ma garantire che l'informazione scambiata sia sintatticamente corretta e comprensibile per sistemi con architetture hardware differenti.

8.16.5 Stream XDR

In uno stream XDR i dati viaggiano in un formato universale. Per ogni informazione trasmessa, il protocollo prevede due trasformazioni: una effettuata dal mittente e una dal destinatario. Sulla rete, l'unico formato ammesso è quello standard XDR.

Ogni nodo ha la responsabilità di implementare esclusivamente le proprie funzionalità di trasformazione locale: il mittente si occupa della conversione dal proprio formato nativo a quello standard (*encoding*), mentre il destinatario provvede alla conversione dal formato standard a quello locale (*decoding*). Questo approccio "a due fasi" semplifica l'interconnessione, poiché un nodo non deve conoscere le specifiche hardware di tutti i suoi possibili interlocutori, ma deve semplicemente saper "parlare" il linguaggio comune XDR.

8.17 API di basso livello

Per ottenere la massima capacità espressiva e il controllo totale sui parametri di comunicazione, Sun RPC mette a disposizione un set di API di basso livello. Queste funzioni permettono di gestire esplicitamente i descrittori di trasporto, i tempi di attesa e le risorse di memoria.

8.17.1 Cliente

Il controllo lato client si basa sulla creazione di un *client handle*, una struttura che mantiene lo stato della connessione:

- **clntudp_create / clnttcp_create**: Inizializzano il client handle specificando il protocollo di trasporto. Permettono di definire l'indirizzo del server e la dimensione dei buffer di invio/ricezione.
- **clnt_call**: Sostituisce la **callrpc**. Consente di specificare timeout personalizzati e di gestire i risultati tramite puntatori XDR, offrendo maggiore flessibilità rispetto alla versione semplificata.
- **clnt_perror**: Funzione di diagnostica che stampa un messaggio di errore leggibile basandosi sullo stato dell'ultimo fallimento contenuto nel client handle.
- **clnt_destroy**: Dealloca le risorse associate al client handle e chiude le eventuali socket aperte.

8.17.2 Server

Lato server, le API di basso livello permettono di gestire servizi multiprotocollo e registrazioni granulari:

- **svcudp_create / svctcp_create**: Creano un gestore di servizio (*service handle*) per un particolare trasporto. A differenza delle API semplificate, qui il programmatore può scegliere una porta specifica o lasciare che il sistema ne assegni una dinamica.
- **pmap_unset**: Utilizzata per ripulire la tabella del Portmapper da registrazioni precedenti o obsolete riguardanti il programma e la versione correnti.
- **svc_register**: Associa il numero di programma e versione a una specifica procedura di *dispatch* (il gestore delle chiamate). Questa funzione mappa il servizio sia internamente al processo che esternamente presso il Portmapper.
- **svc_run**: Entra nel ciclo di attesa infinito. In questo contesto, gestisce tutti i trasporti registrati precedentemente, reagendo alle richieste in arrivo tramite una **select()**.

8.18 Server

8.18.1 Gestore di trasporto

Il gestore di trasporto è la struttura dati fondamentale che permette l'astrazione tra il protocollo RPC e il livello di trasporto sottostante (UDP o TCP). È definito dal tipo `SVCXPRT` e funge da interfaccia polimorfica per tutte le operazioni di I/O del server.

```
1 typedef struct {
2 #ifdef KERNEL
3     struct socket *xp_sock;
4 #else
5     int xp_sock;           /* Socket associata al servizio */
6 #endif
7     u_short xp_port;       /* Numero di porta locale associato */
8     struct xp_ops {
9         bool_t (*xp_rcv)(); /* Ricezione delle richieste */
10        enum xpstat (*xp_stat)(); /* Verifica dello stato del trasporto */
11        bool_t (*xp_getargs)(); /* Decodifica XDR degli argomenti */
12        bool_t (*xp_reply)(); /* Invio della risposta al client */
13        bool_t (*xp_freeargs)(); /* Deallocazione memoria argomenti */
14        void (*xp_destroy)(); /* Distruzione del gestore */
15    } *xp_ops;
16    int xp_addrlen;         /* Lunghezza dell'indirizzo remoto */
17    struct sockaddr_in xp_raddr; /* Indirizzo del client remoto */
18    struct opaque_auth xp_verf; /* Verificatore per l'autenticazione */
19    caddr_t xp_p1; caddr_t xp_p2; /* Dati privati per l'implementazione */
20 } SVCXPRT;
```

La struttura `SVCXPRT` è un'entità astratta che assolve a compiti critici per il funzionamento del server:

- **Astrazione Funzionale:** Contiene una tabella di puntatori a funzione (`xp_ops`) che implementano le operazioni sui dati. Questo permette a `svc_run()` di invocare le stesse funzioni logiche indipendentemente dal protocollo fisico.
- **Gestione degli Endpoint:** Riferisce la socket locale (`xp_sock`) per l'ascolto delle richieste e memorizza l'indirizzo del chiamante (`xp_raddr`) per indirizzare correttamente i risultati dell'esecuzione remota.

Creazione La creazione del gestore avviene tramite funzioni specifiche a seconda del protocollo desiderato:

- **svcudp_create(int sock)** Se il parametro `sock` viene impostato alla costante `RPC_ANYSOCK`, il sistema crea automaticamente una nuova datagram socket. In caso contrario, utilizza il descrittore fornito. Se la socket non è ancora legata a una porta, il sistema ne assegna una automaticamente.
- **svctcp_create(int sock, u_int send_size, u_int rcv_size)** Funziona in modo analogo alla versione UDP ma richiede parametri aggiuntivi per definire le dimensioni dei buffer di invio e ricezione. Questo è necessario per gestire correttamente la natura "a stream" del protocollo TCP, garantendo che i messaggi RPC siano correttamente delimitati all'interno del flusso continuo di dati.

In entrambi i casi, se il sistema non riesce ad allocare le risorse o a creare le socket necessarie, la funzione restituisce `NULL`, indicando il fallimento nella creazione del gestore.

8.18.2 Dispatching

Il **dispatching** è il processo con cui il server identifica e inoltra la richiesta RPC alla specifica implementazione della procedura richiesta. Per essere attiva, una funzione di dispatching deve essere preventivamente registrata nel sistema.

L'associazione tra l'identità logica del servizio (#programma, #versione) e la funzione di dispatch avviene tramite la primitiva `svc_register()`. Questa operazione lega la tripla identificativa al gestore di trasporto e al protocollo scelto.

```

1 bool_t svc_register (xpirt, prognum, versnum, dispatch, protocol);
2
3     SVCXPRT *xpirt;           /* Gestore di trasporto precedentemente creato */
4     u_long prognum, versnum; /* Identificativi di programma e versione */
5     void (*dispatch)();      /* Puntatore alla funzione di selezione */
6     u_long protocol;         /* Protocollo (es. IPPROTO_UDP o IPPROTO_TCP) */

```

Quando `svc_run()` riceve un messaggio, invoca la funzione di dispatch passando un puntatore alla struttura `svc_req`. Questa struttura contiene i metadati necessari per distinguere la richiesta:

```

1 struct svc_req {
2     u_long rq_prog;           /* Numero di programma */
3     u_long rq_vers;           /* Versione del programma */
4     u_long rq_proc;           /* Numero della procedura specifica richiesta */
5     struct opaque_auth rq_cred; /* Credenziali di autenticazione del client */
6     caddr_t rq_clntcred;      /* Credenziali in sola lettura */
7     SVCXPRT *rq_xprt;         /* Riferimento al gestore di trasporto (SVCXPRT) */
8 };

```

La procedura di dispatching (solitamente implementata come uno `switch` basato sul campo `rq_proc`) funge da punto unico di ingresso per tutte le procedure di un programma che condividono lo stesso trasporto. Le sue responsabilità principali sono:

1. **Identificazione:** Interpretare `rq_proc` per determinare quale logica eseguire.
2. **Estrazione Parametri:** Utilizzare la funzione `svc_getargs(xpirt, xdr_filter, arg_ptr)` per decodificare i dati in arrivo dallo stream XDR.
3. **Esecuzione e Risposta:** Dopo aver eseguito la logica locale, utilizzare `svc_sendreply(xpirt, xdr_filter, res_ptr)` per codificare e spedire il risultato al client attraverso il gestore di trasporto `rq_xprt`.

8.18.3 Procedure di conversione e gestione errori

Oltre alla registrazione dei servizi nel meccanismo di dispatching, il sistema RPC prevede una serie di funzioni built-in per garantire la robustezza dell'interazione. Tali procedure permettono di ottenere informazioni dettagliate sullo stato del gestore di trasporto e di gestire le eccezioni durante l'invocazione.

Al momento della registrazione e durante l'operatività, vengono attivati i seguenti meccanismi di controllo:

- **NULLPROC (Procedura 0):** Per ogni programma registrato, la procedura numero 0 è riservata per scopi di diagnostica. Essa non accetta argomenti, non restituisce risultati e non richiede l'intervento della logica applicativa. Viene utilizzata dai client (e dal Portmapper) per verificare tramite un "ping" se il server è attivo e raggiungibile.
- **Controllo del numero di procedura:** Il dispatcher verifica che il valore `rq_proc` ricevuto nella richiesta corrisponda a una procedura effettivamente implementata per quella specifica versione del programma.
- **Gestione errori tramite `svcerr_noproc()`:** Qualora venga richiesta una procedura inesistente, il sistema non interrompe l'esecuzione ma invoca la funzione `svcerr_noproc()`. Questa routine genera e invia automaticamente un messaggio di errore standard al client, notificando il fallimento della chiamata a causa di una procedura non valida.

Questi meccanismi di protezione assicurano che il server possa rispondere in modo coerente anche a fronte di richieste malformate o versioni del client non allineate, mantenendo l'integrità del processo servitore.

8.19 Client

8.19.1 Gestore di trasporto

Il client RPC necessita di una struttura dati di supporto per gestire la comunicazione, definita dal tipo `CLIENT`. Questo "handle" astrae i dettagli del trasporto, consentendo all'applicazione di effettuare chiamate remote indipendentemente dal protocollo sottostante. Mentre la funzione semplificata `callrpc()` crea internamente un gestore UDP, le API di basso livello permettono una configurazione granulare.

```
1 typedef struct {
2     AUTH *cl_auth;           /* Struttura per l'autenticazione */
3     struct clnt_ops {
4         enum clnt_stat (*cl_call)(); /* Esecuzione RPC */
5         void (*cl_abort)();          /* Annullamento chiamata */
6         void (*cl_geterr)();         /* Recupero codice errore */
7         bool_t (*cl_freeres)();      /* Liberazione memoria risultati */
8         void (*cl_destroy)();        /* Distruzione del gestore */
9         bool_t (*cl_control)();      /* Configurazione parametri (timeout,
10                                     porta) */
11     } *cl_ops;
12     caddr_t cl_private;        /* Dati privati dell'implementazione */
13 } CLIENT;
```

Creazione La creazione dell'handle avviene tramite primitive specifiche che differiscono per gestione della connessione e parametri temporali:

- `clntudp_create(addr, prognum, versnum, wait, sockp)`: Crea un gestore basato su datagrammi. Il parametro `wait` definisce l'intervallo di tempo tra le eventuali ritrasmissioni automatiche. Se il campo della porta in `addr` è 0, la funzione interroga automaticamente il Portmapper remoto per risolverlo.
- `clnttcp_create(addr, prognum, versnum, sockp, sendsz, rcvsvsz)`: Crea un gestore orientato alla connessione. Non prevede un parametro di timeout per le ritrasmissioni (essendo TCP un protocollo affidabile) e richiede la definizione dei buffer di I/O. La creazione del gestore comporta l'instaurazione immediata della connessione TCP.

Un aspetto fondamentale di entrambi i costruttori è la risoluzione dei nomi: se la socket passata tramite `sockp` non è aperta, il sistema ne crea una nuova e, in caso di successo, restituisce il puntatore alla struttura `CLIENT` configurata.

8.19.2 Chiamata procedura remota: `clnt_call()`

Una volta ottenuto il gestore di trasporto (*client handle*), il client è pronto per invocare le procedure remote. Poiché il gestore è già legato a un'entità specifica e a una porta, la funzione `clnt_call()` deve specificare solo i dettagli della singola operazione.

```
1 enum clnt_stat clnt_call (clnt, procnum, inproc, in, outproc, out, tout)
2 CLIENT *clnt;           /* Gestore di trasporto */
3 u_long procnum;         /* Numero della procedura da invocare */
4 xdrproc_t inproc;       /* Routine XDR per la codifica degli argomenti */
5 char *in;               /* Puntatore ai dati di ingresso */
6 xdrproc_t outproc;      /* Routine XDR per la decodifica del risultato */
7 char *out;              /* Puntatore dove memorizzare il risultato */
8 struct timeval tout;     /* Timeout totale della chiamata */
```

Il parametro `tout` (timeout totale) viene gestito diversamente in base al protocollo:

- **In UDP**: Il sistema calcola il numero di ritrasmissioni dividendo il timeout totale per l'intervallo di attesa (`wait`) definito durante la creazione del gestore con `clntudp_create()`.

- **In TCP:** Non essendoci ritrasmissioni applicative (gestite nativamente dallo stack TCP), `tout` indica semplicemente il tempo massimo di attesa prima di considerare il server irraggiungibile o la connessione interrotta.

8.19.3 Analisi errori e Distruzione

Il risultato della `clnt_call()` è un'enumerazione di tipo `clnt_stat`. Per interpretare eventuali fallimenti, si utilizza la funzione di diagnostica:

- `clnt_perror(clnt, s)`: Analizza lo stato interno del gestore e stampa sullo *standard error* un messaggio descrittivo preceduto dalla stringa `s`. È essenziale per distinguere errori di rete da errori di codifica XDR o indisponibilità del server.

Al termine delle operazioni, è fondamentale liberare le risorse:

- `clnt_destroy(clnt)`: Dealloca la struttura dati `CLIENT` e chiude le risorse associate. È importante notare che, se più gestori condividono la stessa socket, la distruzione di uno di essi potrebbe non chiudere immediatamente la socket se altri handle vi fanno ancora riferimento.

8.20 Generazione Automatica delle RPC

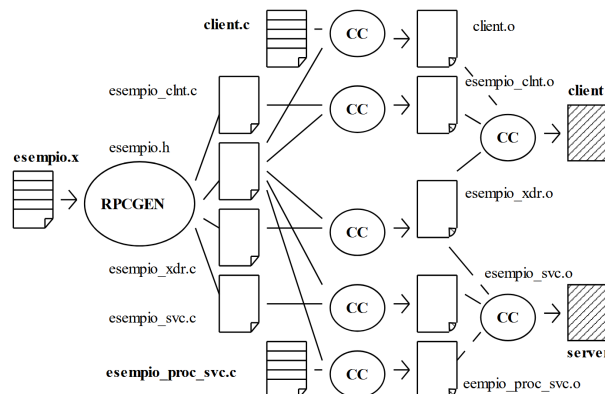
Per innalzare il livello di astrazione e ridurre la complessità legata alla programmazione dei socket e del marshalling manuale, Sun RPC fornisce il **Remote Procedure Call Generator (rpcgen)**. Si tratta di un compilatore di protocollo che automatizza la creazione delle procedure **stub**, ovvero dei pezzi di codice che fungono da interfaccia locale per le chiamate remote.

`rpcgen` elabora un file di ingresso con estensione `.x`, scritto in linguaggio **XDR**, contenente la descrizione formale del contratto tra client e server. Nello specifico, il compilatore processa:

- **Definizioni dei tipi di dati:** Strutture, unioni, enumerazioni e array che devono essere scambiati.
- **Definizioni delle procedure remote:** Numeri di programma, versione e procedura, inclusi i tipi dei parametri di ingresso e dei valori di ritorno.

A partire da queste specifiche, `rpcgen` genera automaticamente diversi file sorgente in linguaggio C:

1. **Client Stub (`_clnt.c`):** Implementa le funzioni che il client chiamerà localmente; queste si occupano di preparare i dati e invocare la `clnt_call()`.
2. **Server Skeleton (`_svc.c`):** Contiene il `main` del server, la creazione del gestore di trasporto e la funzione di *dispatching*.
3. **XDR Filter (`_xdr.c`):** Include le routine di conversione XDR per i tipi di dati personalizzati definiti dall'utente.
4. **Header File (`.h`):** Contiene le definizioni delle costanti e i prototipi delle funzioni per garantire la coerenza tra le parti.



8.21 Esempio

8.21.1 File .x (Definizione dell'Interfaccia)

Il file .x definisce il contratto formale tra client e server utilizzando la sintassi XDR. In questo esempio, viene modellato un servizio di lettura directory remota (*RLS - Remote List Service*).

```
1  const MAXNAMELEN = 255;
2
3  /* Definizione di tipi di dati tramite alias e strutture ricorsive */
4  typedef string nametype <MAXNAMELEN>;
5  typedef struct namenode *namelist;
6
7  struct namenode {
8      nametype name;
9      namelist next;
10 };
11
12 /* Unione discriminata per il risultato: ritorna la lista se errno e 0,
13    altrimenti non restituisce nulla (void) */
14 union readdir_res switch (int errno) {
15     case 0:
16         namelist list;
17     default:
18         void;
19 };
20
21 /* Definizione del programma RPC */
22 program RLSPROG {
23     version RLSVERS {
24         readdir_res READDIR (nametype) = 1; /* Procedura 1 */
25     } = 1; /* Versione 1 */
26 } = 0x20000013; /* Identificativo univoco del Programma */
```

8.21.2 Header file (Generato da rpcgen)

L'header file (.h) trasforma le astrazioni XDR in strutture C concrete e definisce i prototipi necessari per l'integrazione con le librerie di sistema.

```
1  #include <rpc/rpc.h>
2
3  #define MAXNAMELEN 255
4  typedef char *nametype;
5  typedef struct namenode *namelist;
6
7  struct namenode {
8      nametype name;
9      namelist next;
10 };
11 typedef struct namenode namenode;
12
13 /* La union XDR viene tradotta in una struct C con un campo discriminante
14    e una union interna anonima */
15 struct readdir_res {
16     int errno;
17     union {
18         namelist list;
19     } readdir_res_u;
20 };
```

```

21 typedef struct readdir_res readdir_res;
22
23 #define RLSPROG 0x20000013
24 #define RLSVERS 1
25 #define READDIR 1
26
27 /* Prototipi per il lato Client: aggiungono il parametro CLIENT* */
28 extern readdir_res * readdir_1(nametype *, CLIENT *);
29
30 /* Prototipi per il lato Server (Skeleton): aggiungono il puntatore svc_req */
31 extern readdir_res * readdir_1_svc(nametype *, struct svc_req *);
32
33 /* Dichiarazione delle funzioni di filtro XDR per il marshalling */
34 extern bool_t xdr_nametype (XDR *, nametype*);
35 extern bool_t xdr_namelist (XDR *, namelist*);
36 extern bool_t xdr_readdir_res (XDR *, readdir_res*);

```

8.21.3 _xdr.c (Filtri di conversione)

Il file `_xdr.c` contiene le routine di marshalling per i tipi di dati complessi definiti nel file `.x`. Si noti l'uso della ricorsione per gestire la lista concatenata (`namelist`).

```

1  #include "rls.h"
2
3  bool_t xdr_nametype (XDR *xdrs, nametype *objp) {
4      if (!xdr_string (xdrs, objp, MAXNAMELEN)) return FALSE;
5      return TRUE;
6  }
7
8  bool_t xdr_namelist (XDR *xdrs, namelist *objp) {
9      /* xdr_pointer gestisce i riferimenti e la ricorsione della lista */
10     if (!xdr_pointer (xdrs, (char **)objp, sizeof(struct namenode), (xdrproc_t)
11         xdr_namenode))
12         return FALSE;
13     return TRUE;
14 }
15
16 bool_t xdr_namenode (XDR *xdrs, namenode *objp) {
17     if (!xdr_nametype (xdrs, &objp->name)) return FALSE;
18     if (!xdr_namelist (xdrs, &objp->next)) return FALSE;
19     return TRUE;
20 }
21
22 bool_t xdr_readdir_res (XDR *xdrs, readdir_res *objp) {
23     if (!xdr_int (xdrs, &objp->errno)) return FALSE;
24     switch (objp->errno) {
25         case 0:
26             if (!xdr_namelist (xdrs, &objp->readdir_res_u.list)) return FALSE;
27             break;
28         default:
29             break;
30     }
31     return TRUE;
32 }

```

8.21.4 Stub Client

Lo stub del client fornisce l'astrazione della chiamata remota. Notare che `clnt_res` è dichiarata `static` poiché la funzione restituisce un puntatore; il contenuto verrà sovrascritto a ogni chiamata.

```
1 #include "rls.h"
2 #include <memory.h>
3
4 static struct timeval TIMEOUT = { 25, 0 };
5
6 readdir_res * readdir_1(nametype *argp, CLIENT *clnt) {
7     static readdir_res clnt_res;
8     memset((char *)&clnt_res, 0, sizeof(clnt_res));
9
10    if (clnt_call(clnt, READDIR,
11                 (xdrproc_t)xdr_nametype, (caddr_t)argp,
12                 (xdrproc_t)xdr_readdir_res, (caddr_t)&clnt_res,
13                 TIMEOUT) != RPC_SUCCESS) {
14        return (NULL);
15    }
16    return (&clnt_res);
17 }
```

8.21.5 Stub Server (Skeleton)

Il server stub implementa il `main` (inizializzazione dei trasporti e registrazione) e il `dispatching` (instradamento delle richieste).

```
1 #include "rls.h"
2 #include <stdio.h>
3
4 static void rlsprog_1 (struct svc_req *rqstp, register SVCXPRT *transp);
5
6 int main (int argc, char **argv) {
7     register SVCXPRT *transp;
8     pmap_unset (RLSPROG, RLSVERS);
9
10    /* Registrazione UDP */
11    transp = svcudp_create(RPC_ANYSOCK);
12    if (transp != NULL) {
13        svc_register(transp, RLSPROG, RLSVERS, rlsprog_1, IPPROTO_UDP);
14    }
15
16    /* Registrazione TCP */
17    transp = svctcp_create(RPC_ANYSOCK, 0, 0);
18    if (transp != NULL) {
19        svc_register(transp, RLSPROG, RLSVERS, rlsprog_1, IPPROTO_TCP);
20    }
21
22    svc_run(); /* Ciclo infinito di attesa richieste */
23    return 1;
24 }
25
26 static void rlsprog_1 (struct svc_req *rqstp, register SVCXPRT *transp) {
27     union { nametype readdir_1_arg; } argument;
28     char *result;
29     xdrproc_t _xdr_argument, _xdr_result;
30     char *(*local)(char *, struct svc_req *);
31 }
```

```

32     switch (rqstp->rq_proc) {
33         case NULLPROC:
34             svc_sendreply(transp, (xdrproc_t)xdr_void, (char *)NULL);
35             return;
36         case READDIR:
37             _xdr_argument = (xdrproc_t) xdr_nametype;
38             _xdr_result = (xdrproc_t) xdr_readdir_res;
39             local = (char *(*)(char *, struct svc_req *)) readdir_1_svc;
40             break;
41         default:
42             svcerr_noproc (transp);
43             return;
44     }
45
46     memset ((char *)&argument, 0, sizeof (argument));
47     if (!svc_getargs(transp, (xdrproc_t)_xdr_argument, (caddr_t)&argument)) {
48         svcerr_decode (transp);
49         return;
50     }
51
52     result = (*local)((char *)&argument, rqstp);
53
54     if (result != NULL && !svc_sendreply(transp, (xdrproc_t)_xdr_result, result))
55     {
56         svcerr_systemerr (transp);
57     }
58
59     if (!svc_freeargs (transp, (xdrproc_t)_xdr_argument, (caddr_t)&argument)) {
60         exit (1);
61     }
62 }

```

8.21.6 Implementazione del Client

Il client utilizza la funzione di alto livello `clnt_create` per localizzare il server e creare il gestore di trasporto. Successivamente, invoca la procedura remota come se fosse una funzione locale.

```

1  #include <stdio.h>
2  #include <rpc/rpc.h>
3  #include "rls.h"
4
5  int main(int argc, char *argv[]) {
6      CLIENT *cl;
7      namelist nl;
8      char *server;
9      char *dir;
10     readdir_res *result;
11
12     if (argc != 3) {
13         fprintf(stderr, "uso: %s <host> <dir>\n", argv[0]);
14         exit(1);
15     }
16
17     server = argv[1];
18     dir = argv[2];
19
20     /* Creazione del client handle utilizzando TCP */
21     cl = clnt_create(server, RLSPROG, RLSVERS, "tcp");

```

```

22     if (cl == NULL) {
23         clnt_pcreateerror(server);
24         exit(1);
25     }
26
27     /* Invocazione della procedura remota tramite lo stub */
28     result = readdir_1(&dir, cl);
29     if (result == NULL) {
30         clnt_perror(cl, server);
31         exit(1);
32     }
33
34     /* Controllo degli errori logici restituiti dal server */
35     if (result->errno != 0) {
36         fprintf(stderr, "Errore remoto nella directory %s\n", dir);
37         exit(1);
38     }
39
40     /* Scansione e stampa della lista concatenata ricevuta */
41     for (nl = result->readdir_res_u.list; nl != NULL; nl = nl->next) {
42         printf("%s\n", nl->name);
43     }
44
45     clnt_destroy(cl);
46     return 0;
47 }

```

8.21.7 Implementazione del Server

Il server implementa la funzione `readdir_1_svc`. È fondamentale l'uso della memoria `static` per il risultato e la pulizia della memoria tra una chiamata e l'altra per evitare leak.

```

1  #include <rpc/rpc.h>
2  #include <dirent.h>
3  #include "rls.h"
4
5  extern int errno;
6
7  readdir_res * readdir_1_svc(nametype *dirname, struct svc_req *rd) {
8      DIR *dirp;
9      struct dirent *d;
10     namelist nl;
11     namelist *nlp;
12     static readdir_res res;
13
14     /* Liberazione della memoria allocata nella chiamata precedente */
15     xdr_free((xdrproc_t)xdr_readdir_res, (caddr_t)&res);
16
17     /* Apertura della directory locale sul server */
18     dirp = opendir(*dirname);
19     if (dirp == NULL) {
20         res.errno = errno;
21         return &res;
22     }
23
24     /* Inizializzazione della lista risultati */
25     res.errno = 0;
26     nlp = &res.readdir_res_u.list;

```

```

27
28 while ((d = readdir(dirp)) != NULL) {
29     /* Allocazione di un nuovo nodo della lista */
30     nl = *nlp = (namenode*) malloc(sizeof(namenode));
31     if (nl == NULL) { /* Gestione errore allocazione */ }
32
33     /* Copia del nome del file */
34     nl->name = strdup(d->d_name);
35     nlp = &nl->next;
36 }
37
38 /* Chiusura della lista concatenata */
39 *nlp = NULL;
40
41 closedir(dirp);
42 return &res;
43 }

```

8.22 Utilizzo delle Variabili static nel Server

Nello sviluppo del server RPC è strettamente necessario l'utilizzo di variabili con specificatore **static** per la gestione dei risultati da restituire al client. Questo requisito deriva dalle caratteristiche fondamentali delle variabili statiche in C:

- **Visibilità (Scope):** Sono visibili solo all'interno della funzione o del modulo (file) in cui sono definite, garantendo l'incapsulamento dei dati.
- **Tempo di vita (Lifetime):** Vengono allocate nel **segmento dati globale** del processo. Il loro tempo di vita è pari a quello dell'intero programma.
- **Persistenza:** Una variabile statica interna a una funzione permane oltre la singola invocazione. Ogni chiamata successiva alla stessa procedura accede e utilizza il valore precedentemente memorizzato.

8.22.1 Necessità del valore di ritorno statico

Il valore di ritorno di una procedura RPC **deve necessariamente essere dichiarato static**. Il motivo risiede nel meccanismo di funzionamento del server:

1. Quando la procedura del server termina l'esecuzione, restituisce un **puntatore** alla variabile del risultato.
2. Dopo il ritorno dalla funzione, il controllo passa nuovamente allo *skeleton* (lo stub del server), il quale deve eseguire il **marshalling** XDR e la **spedizione** dei dati sulla rete.
3. Se la variabile non fosse **static**, essa verrebbe allocata nello **stack** della funzione e verrebbe distrutta (rimossa dallo stack) non appena la procedura termina, rendendo il puntatore non più valido.
4. Dichiarando la variabile **static**, il dato rimane integro e disponibile nel segmento dati globale anche dopo il termine della procedura, permettendo al gestore RPC di leggerlo correttamente per l'invio al client.

8.23 Dichiarazione tipi XDR

`rpcgen` non esegue sempre un controllo di tipo rigoroso. Se un tipo indicato non appartiene a quelli predefiniti, il compilatore assume che sia definito esternamente, delegando la risoluzione delle funzioni di conversione XDR alla fase di linking.

8.23.1 Vettori a lunghezza fissa

La sintassi segue lo standard C:

`type-ident variable-ident "[" value "]"`

Vengono mappati direttamente in array C di dimensione statica.

8.23.2 Matrici

In XDR non è possibile definire direttamente strutture innestate (array multidimensionali). È necessario definire strutture intermedie:

```
1 struct RigaMatrice { char riga[20]; };
2 struct MatriceCaratteri { RigaMatrice riga[10]; };
```

8.23.3 Vettori a lunghezza variabile

Per i vettori a lunghezza variabile si utilizzano le parentesi angolate. È possibile specificare una lunghezza massima o lasciarla arbitraria:

`type-ident variable-ident "<" [value] ">"`

In C, questa dichiarazione viene tradotta in una `struct` con due campi:

- `_len`: intero che contiene il numero di elementi correnti.
- `_val`: puntatore alla sequenza di dati allocata dinamicamente.

8.23.4 Puntatori

Dichiarati con l'asterisco (`type-ident *variable-ident`), sono fondamentali per trasferire strutture ricorsive (es. liste o alberi). `rpcgen` genera automaticamente le funzioni XDR (`xdr_pointer`) capaci di ricostruire i riferimenti e i dati puntati sul nodo remoto.

8.23.5 Struttura

```
1 struct struct-ident {
2     declaration;
3     /* ... */
4 };
```

8.23.6 Unione

A differenza del C, le unioni XDR sono **unioni discriminate** (simili a record varianti). Richiedono uno `switch` su una dichiarazione semplice (il "discriminante") per determinare quale braccio dell'unione è attivo:

```
1 union union-ident switch (simple-declaration) {
2     case value: declaration;
3     default: declaration;
4 };
```

8.23.7 Enumerazione e Costanti

Le enumerazioni seguono la sintassi C, permettendo di associare identificatori a valori interi. Le costanti sono definite globalmente e vengono tradotte in direttive `#define` nell'header file:

```
1 enum enum-ident { VAL1 = 1, VAL2 = 2 };
2 const const-ident = integer;
```

8.24 Modalità asincrona

Di default, il protocollo RPC prevede un comportamento **sincrono**: il client invia una richiesta e si blocca in attesa della risposta dal server. Tuttavia, è possibile configurare una modalità asincrona intervenendo sui parametri del client e del server, a patto di rispettare alcuni vincoli fondamentali:

- **Trasporto affidabile:** È indispensabile l'uso di **TCP**. Poiché il client non attende conferme a livello applicativo, la "fiducia" sull'arrivo del messaggio si sposta interamente sul livello di trasporto.
- **Timeout nullo:** Il client deve specificare un `timeout` pari a zero. Questo può essere fatto nella `clnt_call` o globalmente tramite `clnt_control`.
- **Assenza di risposta:** Il server non deve inviare alcun risultato. Nella procedura di risposta si deve dichiarare `xdr_void` come funzione di filtro e `NULL` (o `0`) come argomento.

L'impostazione del timeout avviene tramite la struttura `timeval`:

```
1 struct timeval timeout;  
2 timeout.tv_sec = 0;  
3 timeout.tv_usec = 0;  
4 clnt_control(clnt, CLSET_TIMEOUT, (char *)&timeout);
```

8.24.1 Batching

La modalità **Batch** rappresenta un'estensione del concetto asincrono applicata a flussi massivi di dati. In questa modalità, tutte le richieste di servizio del client non vengono inviate singolarmente ma poste nel buffer TCP.

- **Efficienza:** Il driver di trasporto gestisce l'invio del buffer in modo ottimale senza bloccare il processo generatore.
- **Caratteristiche:** Ogni chiamata non richiede risposta e ogni servizio non invia risultati.
- **Pipeline:** Le richieste vengono "incanalate" nel flusso TCP (*pipelining*), permettendo al server di elaborare una sequenza di comandi in modo continuo.

9 Sistemi di nomi

Nei sistemi distribuiti ci si trova in presenza di sistemi di nomi multipli, dove la gestione delle identità deve rispondere a diverse proprietà variabili che ne determinano l'efficacia e la scalabilità:

- **Generalità:** Capacità del sistema di identificare classi eterogenee di risorse (nodi, file, servizi, utenti) con un meccanismo uniforme.
- **Definizioni multiple:** Possibilità che la stessa entità fisica sia associata a più nomi o alias, richiedendo un coordinamento per evitare inconsistenze.
- **Distribuibilità:** Il sistema di nomi non deve essere centralizzato; le informazioni devono essere partizionate o replicate tra più nodi per garantire tolleranza ai guasti e prestazioni costanti.
- **User-friendliness:** La capacità di fornire nomi simbolici e mnemonici comprensibili agli esseri umani, che vengano poi tradotti in indirizzi numerici o identificativi unici gestibili dalle macchine.

9.1 Nomi

Il problema fondamentale nei sistemi distribuiti è la necessità di identificare in modo univoco le entità per permettere l'interazione. La presenza di entità diverse ed eterogenee porta alla definizione di molteplici livelli di nomi, ciascuno con specifici **contesti di visibilità** (scope).

I nomi si dividono principalmente in due categorie:

- **Nomi Esterni (Stringhe di caratteri):** Progettati per l'utente, sono mnemonici e leggibili (es. `www.esempio.it` o `/home/utente/file.txt`). Offrono un alto livello di astrazione ma richiedono una risoluzione per essere utilizzati.
- **Nomi Interni (Numeri binari):** Utilizzati dal sistema, sono identificativi a basso livello (es. indirizzi IP, UUID o file descriptor). Sono ottimizzati per l'efficienza computazionale e l'instradamento.

Le entità di un programma, come gli oggetti distribuiti, sono spesso associate simultaneamente a diversi nomi sia utente che di sistema. Questa stratificazione permette di mantenere il riferimento all'entità anche se la sua locazione fisica cambia, garantendo la trasparenza.

9.1.1 Livelli di nomi

L'identificazione di una risorsa non avviene tramite un singolo identificativo, ma attraverso una stratificazione su più livelli funzionali:

- **Nome logico esterno:** Identificativo mnemonico scelto dall'utente (es. un URL).
- **Indirizzo fisico:** Identificativo assegnato dal sistema (es. indirizzo IP e porta).
- **Route:** L'organizzazione dei passi necessari per garantire la raggiungibilità della risorsa attraverso l'infrastruttura di rete.

Il sistema deve rendere disponibili funzioni di **conversione o mapping** per navigare tra questi livelli. Mentre i nomi sono scelti dagli utenti e gli indirizzi sono gestiti dal sistema, le route devono essere calcolate dinamicamente dai protocolli di routing.

9.2 Globalità e Località

Ai fini della scalabilità, è fondamentale il coordinamento tra diverse porzioni del sistema. La distinzione tra modelli globali e locali determina la capacità di crescita della rete:

- **Modelli Globali:** Non impongono restrizioni alle interazioni. Tuttavia, le operazioni non sono scalabili poiché dipendono dal "diametro" del sistema: più la rete cresce, più aumentano i tempi di ricerca e i costi di gestione.

- **Modelli Locali (o ristretti):** Prevedono limiti alle interazioni basati sulla vicinanza o sull'appartenenza a un dominio. Questi modelli garantiscono operazioni scalabili perché il costo della ricerca è limitato all'ambito locale.

Per ottenere una reale scalabilità in ambienti vasti come Internet, l'architettura tende verso la **località**: il routing e la risoluzione dei nomi vengono gestiti in modo gerarchico, evitando che ogni nodo debba conoscere l'intera topologia globale del sistema.

9.3 Spazi di nomi

L'organizzazione degli identificativi può seguire diverse strutture logiche, ognuna adatta a specifiche esigenze di ricerca e gestione:

- **Piatto (Flat):** I nomi sono stringhe di bit o caratteri senza struttura interna (es. indirizzi MAC o UUID). Sono semplici e ideali per identificativi univoci globali, ma difficili da gestire su larga scala senza un indice centralizzato.
- **Partizionato (Gerarchico):** Lo spazio è diviso in domini e sottodomini (es. file system o DNS). Questa struttura favorisce la scalabilità poiché la gestione dei nomi può essere delegata ad autorità locali.
- **Descrittivo (Attribute-based):** Le entità sono identificate da un insieme di attributi (es. servizi di ricerca tipo "stampante a colori al piano 2"). La risoluzione richiede un processo di *matching* più complesso rispetto alla ricerca diretta.

9.3.1 Nomi di Gruppo

In questi scenari, sorge spesso la necessità di identificare collettivamente più entità tramite un **nome di gruppo**. Un nome di gruppo identifica una lista di nomi di entità (membri del gruppo).

Il problema della risoluzione di un nome di gruppo è concettualmente simile alla distinzione tra **multicast** e **comunicazione punto a punto**: il sistema deve decidere se inviare il messaggio a tutti i membri simultaneamente o gestire la consegna individualmente, mantenendo la coerenza della lista dei membri nel tempo.

9.4 Componenti

Un servizio di nomi si articola attraverso l'interazione tra diversi soggetti e moduli funzionali. I "clienti" del name server possono essere divisi in due categorie operative:

- **Clienti risolutori:** Entità (processi o utenti) che necessitano di tradurre un nome logico in un indirizzo utilizzabile per stabilire una comunicazione.
- **Entità risorse:** Risorse del sistema (servizi, file, nodi) che si registrano presso il sistema di nomi per rendersi visibili e rintracciabili.

Per garantire il corretto funzionamento, il supporto infrastrutturale deve gestire i seguenti aspetti critici:

- **Comunicazione client/server:** Definizione dei protocolli di trasporto e delle interfacce per l'invio delle richieste di risoluzione e ricezione delle risposte.
- **Infrastruttura dei Name Server:** Gestione della presenza di più server (replicati o distribuiti) per garantire alta disponibilità e tolleranza ai guasti.
- **Gestione dei nomi:** Definizione delle regole sintattiche, del ciclo di vita dei nomi e dei contesti di visibilità (namespace).
- **Gestione tabelle e coordinamento:** Mantenimento della coerenza dei dati tra i diversi server, aggiornamento delle tabelle di mapping e risoluzione di eventuali conflitti di denominazione.

9.4.1 Name server

I name server gestiscono tabelle di corrispondenza modellate come un insieme di **tuple di attributi**. Per garantire l'operatività, devono offrire le seguenti primitive:

- **Query:** Ricerca di un'entità in base a un nome o a un set di attributi.
- **Addtuple:** Registrazione di una nuova corrispondenza nome-indirizzo.
- **Modifytuple / Delete tuple:** Aggiornamento o rimozione di record esistenti.
- **Enumerate:** Elencazione di tutte le entità che appartengono a un determinato contesto o categoria.

Mentre sistemi semplici possono basarsi su un **unico servitore**, le architetture reali (come il DNS) utilizzano **agenti multipli** distribuiti e replicati per garantire scalabilità e tolleranza ai guasti.

9.4.2 Comunicazione

In presenza di molteplici name server, il servizio deve gestire il traffico di coordinamento interno tramite connessioni (TCP), datagrammi (UDP) o RPC. Il coordinamento deve minimizzare l'uso di risorse basandosi sulla struttura dello spazio:

- **Spazio piatto:** Richiede una partizione arbitraria dello spazio dei nomi tra i server disponibili.
- **Spazio partizionato:** La delega è naturale; i nomi sono gestiti dall'autorità preposta per quel dominio senza necessità di coordinamento globale costante.

9.4.3 Gestione nomi

La gestione si focalizza su due pilastri fondamentali: come i nomi sono sparsi nel sistema e come vengono ritrovati.

Distribuzione dei nomi I nomi risiedono presso server che ne detengono l'**autorità**. Le politiche di clustering per distribuire i record includono:

- **Algoritmico:** Il server responsabile è determinato da una funzione (es. hashing).
- **Sintattico:** La distribuzione segue la struttura del nome (es. tutti i ".it" su un server).
- **Basato su attributi:** Raggruppamento per caratteristiche comuni della risorsa.

Risoluzione dei nomi È il processo che guida la richiesta dal cliente al server autoritativo. Le fasi tipiche sono:

1. **Localizzazione dell'autorità:** Trovare il server che possiede il record cercato.
2. **Verifica autorizzazioni:** Controllo dei permessi del richiedente.
3. **Esecuzione:** Restituzione del valore o modifica della tupla.

Per limitare i costi di comunicazione, si adottano strategie di **caching** (memorizzazione locale dei risultati recenti), creazione di **vicinati** tra server e propagazione della conoscenza per ridurre i salti di rete necessari.

9.5 Sistema globale

Le strategie di coordinamento tra i server sono cruciali per determinare il comportamento e l'efficienza del sistema su scala globale. La struttura organizzativa influenza direttamente il modo in cui le richieste fluiscono tra i nodi:

- **DNS (Domain Name System):** I server sono organizzati in una **struttura gerarchica ad albero** ben definita (Root, TLD, Authoritative). Le comunicazioni seguono rigorosamente questa gerarchia: un server interagisce tipicamente con i propri "vicini" (padri o figli nell'albero) per risolvere i nomi. Questo garantisce un percorso deterministico e facilita il caching a vari livelli.
- **X.500 (Directory Service):** Al contrario del DNS, lo standard X.500 mira a fornire un servizio di directory globale dove l'organizzazione interna dei server è **completamente trasparente** e non visibile all'esterno. L'utente interagisce con il sistema come se fosse un'entità unica, mentre il sistema gestisce internamente il routing delle query tra i vari agenti (DSA - Directory System Agents) senza che il client debba conoscere la topologia della rete dei server.

La risoluzione dei nomi può essere gestita secondo tre schemi principali, che differiscono per il carico di lavoro distribuito tra client e server:

- **Risoluzione Ricorsiva:** Il client interroga un name server (solitamente il resolver locale) chiedendogli di risolvere l'intero nome. Se il server non conosce la risposta, si assume l'onere di interrogare gli altri server per conto del client, restituendo solo il risultato finale (o un errore). È onerosa per i server ma molto semplice per il client.
- **Risoluzione Iterativa:** Il server interrogato, se non possiede il record, non effettua altre chiamate, ma risponde al client con un riferimento (referral) al prossimo server più autorevole da contattare. Il client deve quindi farsi carico di "iterare" il processo contattando i server suggeriti fino a raggiungere quello autoritativo.
- **Risoluzione Transitiva:** Rappresenta una variante della ricorsiva in cui la catena di richieste si sposta progressivamente in avanti. Ogni server inoltra la richiesta al successivo; tuttavia, la risposta può tornare direttamente al chiamante originale o seguire il percorso inverso. Viene spesso utilizzata per ottimizzare il routing in contesti di vicinato tra server.

9.6 Altri sistemi di nomi

Oltre al DNS, che è focalizzato sulla risoluzione sintattica di domini, esistono sistemi che organizzano le entità attraverso **attributi** e permettono ricerche basate su di essi. Il principale esempio è lo standard **OSI X.500**.

X.500 è un servizio di direttorio (Directory Service) progettato per essere partizionato, decentralizzato e ad alta disponibilità. La definizione del CCITT (ora ITU-T) lo descrive come:

"Una collezione di sistemi aperti che cooperano per mantenere un database logico di informazioni sugli oggetti del mondo reale. Gli utenti della Directory possono leggere o modificare l'informazione, o parte di essa, solo se hanno i privilegi necessari."

9.7 Directory X.500

X.500 non è solo un protocollo, ma un ecosistema di nodi interconnessi progettato per mantenere elevati standard di **Quality of Service (QoS)** in ambienti distribuiti su larga scala.

- **Nodi e Attributi:** Ogni nodo (entry) è costituito da un insieme di attributi tipizzati. La struttura segue una gerarchia dove i nodi ereditano attributi comuni e aggiungono attributi specifici in base alla loro classe.
- **Granularità:** Il controllo degli accessi e le autorizzazioni alle operazioni sono estremamente granulari, permettendo di definire chi può leggere o modificare ogni singolo attributo.

Per garantire disponibilità e tolleranza ai guasti, il sistema adotta strategie come:

- **Trasparenza:** I nodi sono replicati tra diversi server in modo invisibile all'utente. Il sistema gestisce automaticamente l'instradamento verso la replica più vicina o disponibile.
- **Carico di lavoro:** A causa della complessità del mantenimento della coerenza tra repliche, X.500 è ottimizzato per carichi **read-intensive**. Le operazioni di lettura sono estremamente efficienti, mentre le scritture sono più costose e solitamente limitate in frequenza.

9.7.1 Organizzazione delle Informazioni

La struttura fondamentale si basa su due concetti chiave:

- **DIB (Directory Information Base):** L'insieme completo di tutte le informazioni contenute nella directory.
- **DIT (Directory Information Tree):** L'organizzazione logica ad albero delle informazioni del DIB.

La vera forza di X.500 risiede nella flessibilità delle ricerche. Oltre alla risoluzione del singolo nome (punto-punto), è possibile interrogare il sistema per attributi, ottenendo come risultato gruppi di entità, interi sottoalberi o "foreste" di dati che soddisfano i criteri di ricerca.

9.7.2 Caratteristiche distintive

Le caratteristiche distintive del sistema X.500 sono:

- **Modello a oggetti:** Ogni entità è un'istanza di una classe di oggetti con schema predefinito (es. *person*, *organizationUnit*).
- **Flessibilità di ricerca:** Supporto a filtri complessi.
- **Sicurezza integrata:** Meccanismi di autenticazione e liste di controllo degli accessi (ACL) nativi.
- **Interoperabilità standard:** Progettato per far cooperare sistemi eterogenei seguendo il modello di riferimento OSI.

9.7.3 Query e Ricerca

In X.500, le query di lettura sono progettate per essere estremamente flessibili, permettendo di ottenere:

- **Identificazione di un singolo nodo:** Quando la ricerca punta a un'entità specifica.
- **Identificazione di più nodi:** Quando la ricerca restituisce un insieme di entità che soddisfano determinati criteri (es. una ricerca per attributi).

I nodi ottenuti come risultato vengono forniti in modo completo, includendo tutti gli attributi per cui il richiedente possiede i permessi di lettura. Data la potenziale complessità delle ricerche su alberi di vaste dimensioni, l'operazione è intrinsecamente **parallelizzabile** e viene spesso eseguita in parallelo tra diversi Directory System Agents (DSA).

Ogni entry (o nodo) nel sistema è rintracciabile attraverso due notazioni fondamentali:

- **Distinguished Name (DN):** Rappresenta il percorso completo dalla radice dell'albero fino al nodo specifico. Identifica univocamente l'oggetto all'interno dell'intero DIT (es. **CN=Mario Rossi, OU=Ingegneria, O=UniBO, C=IT**).
- **Relative Distinguished Name (RDN):** Definisce univocamente un oggetto all'interno di un contesto specifico (il nodo padre). È solo l'ultima parte del DN (es. **CN=Mario Rossi**).

Sia il DN che l'RDN possono fungere da chiavi per identificare unicamente un nodo. Tuttavia, i nodi sono selezionabili anche attraverso il valore delle loro tuple (coppie **attributo = valore**).

Strategie di Ricerca Le ricerche possono essere eseguite in due modalità:

- **Ricerca contestuale:** Limitata a uno specifico sottoalbero identificato da un DN di partenza.
- **Ricerca globale:** Query basate su valori contenuti negli attributi ricercati su tutti i nodi dell'albero, permettendo di trovare gruppi di elementi anche molto numerosi che condividono le stesse caratteristiche.

9.7.4 Attributi in X.500

Gli attributi che compongono i nodi sono estremamente eterogenei ma rigorosamente ordinati secondo schemi predefiniti. Questa struttura permette di descrivere entità di diversa natura all'interno dello stesso albero logico, come:

- **Persone:** Attributi come nome, cognome, email, numero di telefono.
- **Organizzazioni:** Ragione sociale, indirizzo fisico, unità organizzative (OU).
- **Beni e Risorse:** Stampanti (risoluzione, velocità), server (indirizzo IP, carico), o risorse geografiche.

9.7.5 Filtri e Operazioni

I filtri di ricerca in X.500 offrono una capacità espressiva molto elevata, permettendo di costruire query complesse per isolare specifici sottoinsiemi di dati:

- **Condizioni logiche:** È possibile combinare più attributi (es. `CN=Corradi AND C=Italy`).
- **Espressioni regolari:** Supporto per ricerche parziali o pattern matching (es. `email=*hotmail*`).
- **Condizioni aritmetiche:** Valutazioni numeriche per attributi quantificabili (es. `(age > 18) AND (cookies < 10)`).

Le ricerche possono essere applicate a diversi **scope**: l'intero albero, un contesto specifico o solo i figli diretti di un nodo. L'interazione con il direttorio segue un protocollo preciso:

1. **Bind:** Operazione iniziale di autenticazione e stabilimento della sessione.
2. **Operazioni di lettura/ricerca:** Fase predominante e frequente.
3. **Cambiamenti:** Operazioni di scrittura o modifica, generalmente rare per mantenere l'efficienza della replicazione.
4. **Unbind:** Chiusura della sessione.

L'interfaccia di programmazione deve gestire la complessità derivante dalla potenziale **durata delle operazioni** (specialmente per query su foreste distribuite), spesso utilizzando meccanismi asincroni o timeout.

9.7.6 Organizzazione interna

L'operatività di X.500 si basa sulla cooperazione tra agenti software e protocolli specifici che separano l'interfaccia utente dalla gestione dei dati:

- **DUA (Directory User Agent):** Il software lato client che agisce per conto dell'utente, formulando le richieste di accesso.
- **DSA (Directory System Agent):** Il processo server che mantiene una porzione del DIB e risponde alle query.
- **DAP (Directory Access Protocol):** Il protocollo utilizzato per la comunicazione tra DUA e DSA (molto complesso nello standard originale).

- **DSP (Directory System Protocol):** Il protocollo utilizzato per il coordinamento e lo scambio di informazioni tra diversi DSA (necessario per risoluzioni ricorsive o transitive).

Dalla necessità di semplificare il protocollo DAP per l'ambiente Internet è nato **LDAP** (Lightweight Directory Access Protocol), che pur limitando alcune funzioni, è diventato lo standard per le infrastrutture di gestione delle identità e verifica dei certificati.

9.8 Differenza con Database

Mentre un database è ottimizzato per transazioni complesse e relazioni rigide tra tabelle, una directory ha l'obiettivo di mantenere informazioni su risorse eterogenee, minimizzando la ridondanza e massimizzando la reperibilità.

Le caratteristiche principali di una directory includono:

- **Capacità espressiva:** Lo schema è estendibile per accogliere nuovi tipi di oggetti e attributi.
- **Gestione multi-autorità:** Diverse parti dell'albero possono essere amministrate da autorità differenti (decentralizzazione).
- **Sicurezza granulare:** Permessi differenziati non solo per record, ma per singoli attributi.

Confronto Tecnico In sintesi, a differenza di un database tradizionale:

- **Eterogeneità:** Gli oggetti possono avere attributi diversi tra loro anche se appartengono alla stessa categoria logica.
- **Indipendenza:** Gli oggetti sono entità a sé stanti; la relazione principale è quella di **contenimento** definita dalla struttura ad albero (DIT).
- **Modello di accesso:** È progettata per un numero elevatissimo di **letture** e rarissime **scritture**.
- **Standardizzazione:** Necessita di un protocollo unificato (come LDAP) per garantire che client eterogenei possano estrarre informazioni in modo efficace da server di vendor diversi.

9.9 Uso

Le directory vengono utilizzate per il management delle risorse. In un sistema di gestione in cui consideriamo una molteplicità di gestori con autorità e responsabilità anche non completamente note e forse variabili lentamente nel tempo, si usano le directory per:

- localizzare i diversi gestori e le loro politiche
- trattare i problemi di domini incrociati
- trovare le proprietà delle risorse
- trovare le proprietà dei gestori delle risorse

I costi sono dipendenti dal numero dei nodi (e attributi) motivati dalle proprietà QoS offerte

9.10 Sviluppo

L'evoluzione dei sistemi di nomi ha portato a due approcci distinti, ottimizzati per la natura degli attributi delle entità (variazioni lente vs. variazioni veloci):

- **Directory (Nomi Globali):** Soluzioni progettate per servizi completi e complessi. Gestiscono attributi a variazione lenta (es. indirizzo ufficio, permessi). Il costo delle operazioni è elevato ma giustificato dalla persistenza e globalità dei dati.
- **Discovery (Nomi Locali):** Soluzioni per servizi essenziali e funzioni limitate. Gestiscono attributi a variazione rapida (es. disponibilità di una stampante, stato di un sensore). Il costo è limitato e il sistema è ottimizzato per la velocità in contesti locali.

9.10.1 Protocolli di directory

Un servizio di directory garantisce proprietà avanzate di **QoS** (replicazione, sicurezza, gestione multiserver) per carichi *read-intensive*.

- **Active Directory (Microsoft):** Evoluzione basata su LDAP, integrata nel sistema operativo per gestire utenti, risorse e permessi in reti aziendali.
- **Service Location Protocol (SLP - RFC 2165):** Standard che permette di registrare servizi divisi in località distinte, accessibili via URL e compatibili con interfacce web.
- **Implementazioni note:** Sistemi sviluppati da leader del settore come Cisco, Apple (Bonjour/mDNS) e Novell.

9.10.2 Protocolli di discovery

Il discovery è fondamentale per la computazione cooperativa in ambito locale (LAN), dove le entità devono trovarsi in modo economico e senza configurazione preventiva.

JINI (Java Service) è il protocollo di riferimento per il discovery di *appliance* in ambiente Java. Il processo si basa su:

1. **Multicast Request:** Un'entità (Client o Service) invia un pacchetto multicast sulla rete locale per individuare il **Lookup Service**.
2. **Lookup Service Response:** Il servizio di ricerca risponde fornendo il proprio indirizzo.
3. **Join / Lookup:** Il servizio si registra (Join) o il client cerca un servizio specifico (Lookup) scaricando il relativo codice proxy.

Il discovery risponde all'esigenza del "plug-and-work": un dispositivo entra in un contesto e diventa operativo immediatamente senza conoscenze predefinite dell'ambiente.